



UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y
DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica y Automática Industrial

TRABAJO FIN DE GRADO

BANCO DE ENSAYOS PARA UNA CARACOLA DE
RADIO VARIABLE: CONTROL DEL
SERVOACCIONAMIENTO, ADQUISICIÓN DE
FUERZA Y VALIDACIÓN DEL MODELO

Javier López [**COMPLETAR:** segundo apellido]

Cotutor (si lo hay):

[**COMPLETAR:** nombre y
apellidos]

Departamento:

[**COMPLETAR:** departamento]

Tutor: [**COMPLETAR:** nombre
y apellidos]

Departamento:

[**COMPLETAR:** departamento]

Madrid, **[COMPLETAR: mes]**, 2026



UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y
DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica y Automática Industrial

TRABAJO FIN DE GRADO

BANCO DE ENSAYOS PARA UNA CARACOLA DE
RADIO VARIABLE

Firma del autor: Javier López **[COMPLETAR: segundo apellido]**

Firma del cotutor (si lo hay)

Firma del tutor

Madrid, **[COMPLETAR: mes]**, 2026

Copyright © 2026. Javier López **[COMPLETAR: segundo apellido]**

Esta obra está licenciada bajo la licencia Creative Commons

Atribución-NoComercial-SinDerivadas 3.0 Unported (CC BY-NC-ND 3.0). Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-nc-nd/3.0/deed.es> o envíe una carta a Creative Commons, 444 Castro Street, Suite 900, Mountain View, California, 94041, EE.UU.

[COMPLETAR: confirmar la licencia que se quiere aplicar; ésta es la que propone la plantilla]

Todas las opiniones aquí expresadas son del autor, y no reflejan necesariamente las opiniones de la Universidad Politécnica de Madrid.

Título: Banco de ensayos para una caracola de radio variable: control del servoaccionamiento, adquisición de fuerza y validación del modelo

Autor: Javier López [**COMPLETAR:** segundo apellido]

Tutor: [**COMPLETAR:** nombre del tutor]

Cotutor: [**COMPLETAR:** nombre del cotutor]

EL TRIBUNAL

Presidente:

Vocal:

Secretario:

Realizado el acto de defensa y lectura del Trabajo Fin de Grado el día de de ... en, en la Escuela Técnica Superior de Ingeniería y Diseño Industrial de la Universidad Politécnica de Madrid, acuerda otorgarle la CALIFICACIÓN de:

VOCAL

SECRETARIO

PRESIDENTE

Agradecimientos

[**COMPLETAR:** Agradecimientos: tutoría, departamento o laboratorio que ha prestado el material, compañeros y familia.]

Resumen

Este Trabajo Fin de Grado desarrolla el banco de ensayos de un mecanismo de elevación basado en una *caracola*: un tambor de radio variable que, accionado por un servomotor paso a paso de lazo cerrado, enrolla un cable y levanta una barra articulada con una masa de 2 kg. La idea que se quiere contrastar es la del *fusee* de la relojería: si el radio del tambor crece donde la tensión del cable es pequeña y decrece donde es grande, el par que ve el motor se mantiene aproximadamente constante a lo largo del recorrido.

Para poder contrastarla se ha construido una plataforma completa: una placa adaptadora diseñada en KiCad alrededor de una Raspberry Pi Pico 2, con bus RS-485, entradas digitales protegidas, entradas analógicas acondicionadas, salidas de potencia y dos zócalos mikroBUS; un firmware en C++ que gobierna el accionamiento por Modbus RTU, lee la célula de carga a través de un acondicionador ZSC31014, hace el *homing* con dos finales de carrera y ejecuta ciclos automáticos B→A→B; un protocolo binario propio con CRC-8; y una aplicación web (FastAPI y WebSocket) con tres páginas: monitor en tiempo real, configuración del mecanismo con simulador, y visor de ensayos que superpone la curva teórica a cada medida.

El banco muestrea a 20,0 Hz con una dispersión del periodo de 0,06 ms, graba cada ensayo en cuentas brutas y repite el recorrido con una desviación típica de 11 mrev de motor. Se han analizado dos campañas de ensayos, la segunda con el ciclo de ensayo reorganizado para que arranque en el extremo inferior del recorrido.

El resultado principal es metodológico y, en parte, negativo. El modelo estático reproduce bien la forma de la señal de la célula de carga ($R^2 = 0,97$), pero se demuestra que esa comparación es insuficiente: al no estar calibrada la célula, el ajuste absorbe un cambio de signo y el mismo R^2 admite dos geometrías opuestas. El par del accionamiento, que no necesita calibración, deshace la ambigüedad, y al hacerlo contradice la configuración vigente del mecanismo: sobre el tramo del recorrido en que el cable trabaja con carga, el modelo predice que la tensión *baja* un 20 % mientras que el par gravitatorio medido *sube* un 56 %. El radio efectivo deducido crece un 93 %, cuando para mantener el par constante bastaría con que creciera un 24 %. La caracola, tal y como está descrita en la configuración, no compensa: sobrecompensa, y con el sentido invertido respecto del que pide el modelo.

El trabajo entrega, por tanto, un banco de ensayos reproducible, un modelo verificado en su formulación y un procedimiento de contraste, junto con un diagnóstico claro de lo que falta para cerrar la validación: medir las cotas reales del mecanismo y calibrar la célula con masas patrón.

Palabras clave: tambor de radio variable, caracola, compensación de par, servomotor paso a paso, Modbus RTU, RS-485, célula de carga, ZSC31014, Raspberry Pi Pico 2, adquisición de datos.

Abstract

This Bachelor's Thesis develops the test bench for a lifting mechanism based on a variable-radius spiral drum (a *fusee*-like drum, here called *caracola*): driven by a closed-loop stepper servomotor, it winds a cable that lifts a hinged bar carrying a 2 kg mass. The idea under test is the one behind the horological fusee: if the drum radius grows where the cable tension is small and shrinks where it is large, the motor torque stays roughly constant along the stroke.

A complete platform was built to test it: a KiCad-designed adapter board around a Raspberry Pi Pico 2, with an RS-485 bus, protected digital inputs, conditioned analogue inputs, high-side outputs and two mikroBUS sockets; C++ firmware that commands the drive over Modbus RTU, reads the load cell through a ZSC31014 conditioner, performs homing with two limit switches and runs automatic B→A→B cycles; a custom binary protocol with CRC-8; and a web application (FastAPI and WebSocket) with three pages: real-time monitor, mechanism configuration with a simulator, and a test viewer that overlays the theoretical curve on each measurement.

The bench samples at 20,0 Hz with a period spread of 0,06 ms, logs every test in raw counts and repeats the stroke with a standard deviation of 11 mrev of motor shaft. Two test campaigns were analysed, the second one with the test cycle reorganised to start at the lower end of the stroke.

The main result is methodological and partly negative. The static model reproduces the shape of the load-cell signal well ($R^2 = 0,97$), but that comparison is shown to be insufficient: since the cell is uncalibrated, the fit absorbs a sign change and the same R^2 admits two opposite geometries. The drive torque, which needs no calibration, resolves the ambiguity — and in doing so contradicts the current configuration of the mechanism: over the loaded part of the stroke the model predicts the tension to *fall* by 20 % while the measured gravitational torque *rises* by 56 %. The implied effective radius grows by 93 %, whereas a 24 % growth would suffice for constant torque. The spiral drum, as configured, does not compensate: it overcompensates, and in the opposite sense to the one the model requires.

The thesis therefore delivers a reproducible test bench, a model verified in its formulation and a contrast procedure, together with a clear diagnosis of what remains in order to close the validation: measuring the real dimensions of the mechanism and calibrating the load cell with reference masses.

Keywords: variable radius drum, fusee, torque compensation, closed-loop stepper servo, Modbus RTU, RS-485, load cell, ZSC31014, Raspberry Pi Pico 2, data acquisition.

Índice general

Agradecimientos	VII
Resumen	IX
Abstract	XI
Índice	XVI
Acrónimos y símbolos	XXI
1. Introducción	1
1.1. Motivación del proyecto	1
1.2. Objetivos	1
1.3. Materiales utilizados	2
1.4. Alcance y limitaciones	2
1.5. Metodología	2
1.6. Estructura del documento	3
2. Marco teórico	5
2.1. Transmisión por cable y radio variable	5
2.2. Accionamiento: servomotores paso a paso de lazo cerrado	5
2.3. Medida de fuerza con células de carga	6
2.3.1. Galgas extensométricas y puente de Wheatstone	6
2.3.2. Acondicionadores integrados	6
2.4. Microcontrolador	6
2.5. Buses de comunicación	6
2.5.1. RS-485 y Modbus RTU	6
2.5.2. I ² C y mikroBUS	7
2.6. Adquisición y supervisión en el ordenador	7
3. Estado del arte	9
3.1. Del <i>fusee</i> a los tambores de radio variable	9
3.2. Compensación de gravedad con poleas no circulares	9
3.3. Bancos de ensayo y validación experimental	9
3.4. Conclusiones del estado del arte	10
4. Modelo del mecanismo	11
4.1. Descripción del mecanismo	11
4.2. Longitud del cable	12
4.3. Tensión del cable	12
4.4. Par en el tambor	13
4.4.1. Cota inferior del par de pico	13
4.4.2. Perfil ideal de par constante	14
4.4.3. Comparación de perfiles	14
4.5. Método experimental: separación del par y radio efectivo	15

4.5.1. Por qué no basta con medir la fuerza	15
4.6. Limitaciones del modelo	15
5. Requisitos y arquitectura del banco	17
5.1. Requisitos	17
5.1.1. Requisitos funcionales	17
5.1.2. Requisitos no funcionales	17
5.2. Arquitectura general	18
5.3. Flujo de datos y temporización	19
5.4. Decisiones de diseño	19
5.5. Seguridad	20
6. Diseño hardware	23
6.1. Placa adaptadora: objetivos	23
6.2. Revisiones del diseño	23
6.3. Microcontrolador	24
6.4. Interfaz RS-485	24
6.5. Entradas digitales	25
6.6. Entradas analógicas	25
6.7. Salidas de potencia	25
6.8. Célula de carga	25
6.9. Alimentación	26
6.10. Placa de circuito impreso	26
6.11. Validación de la placa	26
7. Firmware del microcontrolador	29
7.1. Entorno y organización	29
7.2. Lazo principal	29
7.3. Máquina de estados del servo	30
7.4. Lectura de la célula de carga	30
7.4.1. Árbitro del bus I ² C	31
7.5. Posición integrada	31
7.6. <i>Homíng</i>	31
7.7. Ciclo de ensayo B→A→B	32
7.8. Configuración del acondicionador de la célula	33
7.8.1. Escritura verificada	33
7.8.2. Mantenimiento del reinicio para medida	33
7.9. Evolución del firmware	33
8. Comunicaciones	35
8.1. Protocolo binario entre el microcontrolador y el ordenador	35
8.1.1. Formato de trama	35
8.1.2. Recepción y resincronización	35
8.1.3. Paquetes del microcontrolador al ordenador	36
8.1.4. Órdenes del ordenador al microcontrolador	36
8.2. Modbus RTU con el accionamiento	37
8.2.1. Registros utilizados	37
8.2.2. Secuencia de configuración	38
8.2.3. Transacciones por periodo	38
8.3. Interfaz entre el servidor y el navegador	38
8.3.1. WebSocket	38
8.3.2. Servicios HTTP	39

9. Software de supervisión y análisis	41
9.1. Visión general	41
9.2. Servidor	41
9.2.1. Estructura concurrente	41
9.2.2. Configuración del mecanismo	42
9.2.3. Grabación de ensayos	42
9.3. Modelo compartido	42
9.4. Monitor en tiempo real	42
9.5. Configuración del mecanismo y simulador	43
9.6. Visor de ensayos	43
9.7. Herramientas auxiliares	44
9.8. Interfaz de usuario	44
9.9. Observaciones de calidad	44
10. Problemas técnicos y soluciones	45
10.1. UART sobre PIO y latencia del enlace Modbus	45
10.2. Configuración del ZSC31014: la ventana del modo comando	45
10.2.1. Primer obstáculo: provocar el reinicio	46
10.2.2. Segundo obstáculo: alimentación parásita por el bus	46
10.2.3. Tercer obstáculo: acertar la ventana	46
10.2.4. Cuarto obstáculo: integridad de la EEPROM	46
10.3. Lecturas de la célula siempre “antiguas”	46
10.4. Depuración del ciclo de ensayo	46
10.5. El modelo que no cerraba: la salida del cable desplazada	47
10.6. Lo que la fuerza no puede decidir	47
10.7. Tramo inicial sin carga y calibraciones incoherentes	47
10.8. Segunda célula de carga en el mismo bus	48
10.9. Discrepancias con la hoja de datos	48
10.10. Otros problemas	48
10.11. Lecciones aprendidas	49
11. Resultados y discusión	51
11.1. Campañas de ensayos	51
11.2. Temporización del lazo de control	51
11.3. Ensayo tipo	52
11.4. Repetibilidad	52
11.5. Tensión y par a lo largo del recorrido	52
11.6. Contraste con el modelo: cilindro frente a caracola	54
11.6.1. Lo que dice la fuerza: nada concluyente	54
11.6.2. Lo que dice el par: el tambor no es un cilindro	56
11.6.3. Cuánto compensa, y en qué sentido	56
11.7. Ajuste de los parámetros del mecanismo	57
11.8. Comparación con la campaña preliminar	57
11.9. Limitaciones de las medidas	57
11.10. Trabajo pendiente para cerrar la validación	58
12. Gestión del proyecto	59
12.1. Ciclo de vida	59
12.2. Planificación	59
12.2.1. Planificación real	59
12.3. Presupuesto	60
12.3.1. Personal	60
12.3.2. Material	61
12.3.3. Equipos y software	61
12.3.4. Resumen de costes	62
12.4. Impacto	62

12.4.1. Impacto técnico y económico	62
12.4.2. Objetivos de Desarrollo Sostenible	63
12.4.3. Aspectos éticos, legales y de seguridad	63
13. Conclusiones	65
13.1. Conclusión	65
13.1.1. Sobre el banco de ensayos	65
13.1.2. Sobre el modelo del mecanismo	65
13.1.3. Sobre la validación experimental	65
13.1.4. Grado de cumplimiento de los objetivos	66
13.2. Desarrollos futuros	66
13.2.1. Para cerrar la validación	66
13.2.2. Sobre el mecanismo	67
13.2.3. Sobre el sistema de medida	67
13.2.4. Sobre la seguridad y la explotación	67
13.3. Valoración personal	67
A. Asignación de pines	69
A.1. Raspberry Pi Pico 2 (revisión B de la placa)	69
A.2. Bornas de campo	70
B. Especificación del protocolo binario	71
B.1. Paquete 0x02: telemetría por lotes	71
B.2. Paquete 0x03: estado de <i>homíng</i> (11 bytes)	72
B.3. Paquete 0x04: resultado de la célula (13 bytes)	72
B.4. Paquete 0x05: estado del ciclo (18 bytes)	72
B.5. Órdenes	73
B.6. Tablas de códigos	73
C. Registros Modbus del accionamiento	75
C.1. Secuencia de configuración	75
D. Formato de los ficheros de ensayo	77
D.1. Conversión y análisis	77
D.2. Notas sobre las cabeceras	78
E. Manual de uso	79
E.1. Instalación	79
E.1.1. Firmware	79
E.1.2. Aplicación de supervisión	79
E.2. Configuración del mecanismo	79
E.3. Procedimiento de ensayo	79
E.4. Programación de la célula de carga	80
E.5. Diagnóstico	80
E.6. Precauciones	80
F. Estructura del repositorio	81
F.1. Reproducción de los resultados	82
Bibliografía	83

Índice de figuras

4.1. Esquema del mecanismo. El punto de salida del cable D está desplazado dx respecto de la vertical de la articulación O.	11
4.2. Izquierda: par medido (normalizado) frente a las dos orientaciones posibles del perfil. Derecha: par en el tambor para el cilindro equivalente, la caracola configurada y el perfil ideal.	14
5.1. Arquitectura del banco de ensayos.	19
6.1. Esquema de la placa adaptadora, revisión B.	27
6.2. PCB de la revisión A: capas de cobre, máscara y serigrafía superiores.	27
6.3. Vista 3D de la PCB de la revisión A generada con KiCad.	28
7.1. Máquina de estados del enlace con el accionamiento.	30
7.2. Máquina de estados del <i>homíng</i>	32
7.3. Máquina de estados del ciclo de ensayo.	32
8.1. Formato común de trama. El CRC-8 cubre los LEN bytes de la carga útil (ID + datos).	35
9.1. Estructura del servidor de supervisión.	41
9.2. Página de configuración del mecanismo con el simulador.	43
11.1. Izquierda: periodo entre muestras (escala logarítmica). Derecha: tiempo de trabajo por vuelta del lazo.	52
11.2. Ensayo tipo de la campaña del 21 de septiembre: avance, velocidad, señal de la célula con la tensión del modelo superpuesta, y par del motor. En gris, el tramo inicial sin carga.	53
11.3. Señal de la célula (izquierda) y par del motor (derecha) frente al avance, en los tres ensayos limpios. En gris, el tramo sin carga.	54
11.4. Arriba: señal de la célula en cada sentido, con la tensión del modelo. Centro: par en cada sentido, componente gravitatoria τ_g y pérdidas τ_f . Abajo: radio efectivo medido frente a las dos orientaciones posibles de la caracola.	55
11.5. Contraste de las tres hipótesis. Izquierda: tensión del modelo. Centro: par normalizado de cada hipótesis frente al par medido. Derecha: radio de cada hipótesis frente al radio efectivo deducido de la medida.	56
12.1. <i>Commits</i> por semana en el repositorio del proyecto.	59
12.2. Diagrama de Gantt reconstruido (semanas desde el 2 de marzo de 2026).	60

Índice de tablas

2.1. Características del microcontrolador RP2350 en la Raspberry Pi Pico 2 [27].	6
4.1. Parámetros del mecanismo empleados en el modelo [VERIFICAR: medir las cotas reales del prototipo y rehacer el ajuste; de ellas dependen todas las cifras de este capítulo].	13
4.2. Perfiles de tambor sobre el recorrido con carga ($\varphi_{\text{tot}} = 0,209$ rev de tambor, $W = 1,58$ J).	14
5.3. Decisiones de diseño y alternativas consideradas.	19
6.1. Problemas detectados en la revisión A y corrección en la revisión B.	24
6.2. Pruebas de la placa adaptadora.	26
7.1. Versiones del firmware conservadas en el repositorio.	34
8.1. Enlaces de comunicación del sistema.	35
8.2. Paquetes de telemetría y estado.	36
8.3. Órdenes aceptadas por el firmware.	37
8.4. Parámetros configurables con SET_PARAM.	37
8.5. Servicios HTTP del servidor.	39
11.1. Ensayos de la campaña del 21 de septiembre con ciclo completo.	51
11.2. Contraste de las tres hipótesis de tambor sobre el tramo con carga. El R^2 del par no distingue el signo; la columna de la pendiente sí: un valor negativo significa que el modelo y la medida evolucionan en sentidos opuestos.	56
12.1. Dedicación estimada y coste de personal.	61
12.2. Material del sistema.	61
12.3. Amortización de equipos.	62
12.4. Presupuesto total.	62
12.5. Relación del trabajo con los Objetivos de Desarrollo Sostenible [21].	63
A.1. Asignación de GPIO según el esquema de la revisión B y el firmware.	69
A.2. Bornas de la placa (revisión B).	70
B.1. Cabecera del lote (14 bytes).	71
B.2. Muestra (14 bytes), repetida n veces.	71
B.3. Estados de la máquina.	73
B.4. Códigos de error.	73
B.5. Resultados de ModbusMaster.	73
B.6. Resultados de las órdenes de la célula.	74
C.1. Registros utilizados por el firmware.	75
D.1. Columnas del fichero de ensayo.	77

Acrónimos y símbolos

Acrónimos

Sigla	Significado
A/D, ADC	Convertidor analógico-digital (<i>Analog-to-Digital Converter</i>)
API	Interfaz de programación de aplicaciones
CDC	Clase de dispositivo USB de comunicaciones (puerto serie virtual)
CRC	Código de redundancia cíclica (<i>Cyclic Redundancy Check</i>)
CSV	Valores separados por comas (<i>Comma-Separated Values</i>)
DE / RE	Habilitación del emisor / receptor de un transceptor RS-485
DI / DO	Entrada digital / salida digital
EEPROM	Memoria no volátil borrrable eléctricamente
ESD	Descarga electrostática (<i>Electrostatic Discharge</i>)
FC	Final de carrera
GPIO	Entrada/salida de propósito general
GUI	Interfaz gráfica de usuario
HTTP	Protocolo de transferencia de hipertexto
I ² C	Bus serie síncrono de dos hilos (<i>Inter-Integrated Circuit</i>)
LC	Célula de carga (<i>Load Cell</i>); LC1 y LC2 en el firmware
MISR	Registro de firma de la EEPROM del ZSC31014 (<i>Multiple-Input Signature Register</i>)
NC	Contacto normalmente cerrado
ODS	Objetivos de Desarrollo Sostenible
PCB	Placa de circuito impreso (<i>Printed Circuit Board</i>)
PIO	E/S programable del RP2350 (<i>Programmable I/O</i>)
POR	Reinicio por encendido (<i>Power-On Reset</i>)
RPM, rpm	Revoluciones por minuto
RS-485	Estándar TIA/EIA-485 de transmisión diferencial multipunto
RTU	Modo binario de Modbus sobre línea serie (<i>Remote Terminal Unit</i>)
SMD	Componente de montaje superficial
STO	Desconexión segura del par (<i>Safe Torque Off</i>)
TFG	Trabajo Fin de Grado
UART	Transmisor-receptor asíncrono universal
USB	Bus serie universal
VRD	Tambor de radio variable (<i>Variable Radius Drum</i>)
WS	WebSocket

Símbolos principales

Símbolo	Descripción	Unidad
O	Articulación de la barra	–

Símbolo	Descripción	Unidad
D	Punto por el que sale el cable hacia el tambor	—
A	Punto de la barra donde tira el cable	—
L_2	Altura de D sobre la articulación O	mm
dx	Desplazamiento horizontal de D respecto de O	mm
$ OD $	Distancia de O a D, $\sqrt{L_2^2 + dx^2}$	mm
L_3	Distancia de O al anclaje del cable en la barra	mm
L_4	Distancia del anclaje al punto de suspensión de la masa	mm
m	Masa suspendida	kg
g	Aceleración de la gravedad	m/s ²
α	Ángulo en O entre la línea O→D y la barra, de 0° a 180°	°
α_0	Valor de α en B, donde arranca el ensayo	°
θ	Ángulo de la barra respecto a la horizontal	°
c	Longitud libre de cable entre D y A	mm
T	Tensión del cable	N
φ	Giro del tambor contado desde B	rad
$r(\varphi)$	Radio instantáneo de enrollamiento del tambor	mm
$s(\varphi)$	Cable pagado por el tambor, $s = \int r d\varphi$	mm
r_0, r_1	Radios de la caracola al principio y al final del barrido	mm
τ	Par en el eje del tambor	N · m
N	Reducción motor:tambor (vueltas de motor por vuelta de tambor)	—
p	Avance del eje del motor contado desde B	rev
τ_g, τ_f	Componentes gravitatoria y de pérdidas del par medido	u. a.
r_{ef}	Radio efectivo estimado, $\propto \tau_g/T$	u. a./N
u. a.	Unidades internas del accionamiento (registro 16387)	—

Capítulo 1

Introducción

1.1. Motivación del proyecto

Muchos mecanismos de elevación no presentan una carga constante al actuador. En una barra articulada que se levanta tirando de un cable, la tensión necesaria para sostener la carga depende del ángulo que forman el cable y la barra; al girar la barra, esa geometría cambia y con ella la tensión. Si el cable se enrolla en un tambor cilíndrico, el par que el motor debe aportar, producto de la tensión por el radio, sigue fielmente esas variaciones, y el actuador hay que dimensionarlo para el peor punto del recorrido aunque la mayor parte del tiempo trabaje muy por debajo.

Una solución clásica es cambiar la relación de transmisión a lo largo del recorrido haciendo que el radio del tambor no sea constante. Es la idea del *fusee* de la relojería del siglo XV, un cono con una ranura helicoidal que compensaba la pérdida de par del muelle real a medida que se desenrollaba [14]. La misma idea reaparece hoy en los tambores de radio variable de la robótica de cables [29, 1] y en los sistemas pasivos de compensación de gravedad [30, 3]. En este trabajo, al tambor en espiral se le llama *caracola* por su forma.

El mecanismo objeto del proyecto es precisamente una caracola que, accionada por un servomotor paso a paso de lazo cerrado, enrolla un cable y levanta una barra articulada con una masa de 2 kg. La hipótesis de partida es que el perfil de radio variable aplanará el par que ve el motor frente a la variación de la tensión del cable. Comprobarlo exige dos cosas: un modelo que diga qué cabe esperar y un sistema de medida capaz de registrar a la vez la posición del eje, la tensión del cable y el par del motor en ensayos repetibles. Construir ese sistema, y usarlo para contrastar el modelo, es el contenido de este Trabajo Fin de Grado.

1.2. Objetivos

El objetivo general es diseñar, construir y validar un banco de ensayos para el mecanismo de caracola: la electrónica de accionamiento, control y adquisición, el software de supervisión y el modelo con el que se interpretan las medidas.

De él se derivan los siguientes objetivos específicos:

- O1.** Desarrollar un modelo estático del mecanismo que relacione la geometría, la tensión del cable y el par en el tambor, y deducir de él el perfil de radio que hace constante el par.
- O2.** Diseñar una placa de circuito impreso que adapte el microcontrolador al entorno de la máquina: bus RS-485, entradas digitales protegidas, entradas analógicas acondicionadas, salidas de potencia y conexión de células de carga.
- O3.** Desarrollar el firmware del microcontrolador, capaz de gobernar el servoaccionamiento por Modbus RTU, leer la célula de carga, ejecutar el *homing* y los ciclos de ensayo, y vigilar los límites de seguridad con una temporización determinista.
- O4.** Definir un protocolo de comunicación robusto entre el microcontrolador y el ordenador, que transporte la telemetría por lotes y las órdenes con detección de errores.

- 05. Desarrollar una aplicación de supervisión web para operar la máquina, visualizar los datos en tiempo real, grabar cada ensayo en un fichero reproducible, configurar la geometría del mecanismo y comparar cada ensayo con el modelo.
- 06. Caracterizar el banco (temporización, repetibilidad y comportamiento de la medida) y analizar los ensayos para contrastar el modelo y cuantificar el efecto de la caracola.
- 07. Documentar los problemas técnicos encontrados y las soluciones adoptadas, así como la planificación, el presupuesto y el impacto del trabajo.

1.3. Materiales utilizados

- **Mecanismo:** caracola de radio variable, cable, barra articulada con una masa de 2 kg y dos finales de carrera **[COMPLETAR: indicar si la caracola y la estructura se han diseñado y fabricado en este TFG o proceden de un trabajo previo]**.
- **Accionamiento:** servomotor paso a paso de lazo cerrado con accionamiento Modbus RTU sobre RS-485 **[COMPLETAR: marca y modelo]**.
- **Electrónica:** placa adaptadora propia diseñada en KiCad [13] alrededor de un módulo Raspberry Pi Pico 2 [27], con transceptor THVD1400 [32], interruptor de potencia TPS4H000-Q1 [33] y amplificadores MCP6294 [16].
- **Medida de fuerza:** célula de carga con acondicionador ZSC31014 [28] montado en una placa *Load Cell 2 Click* [17].
- **Software:** entorno Arduino con el núcleo Arduino-Pico [10] y la biblioteca ModbusMaster [34] para el firmware; Python con FastAPI [26], pySerial [15] y Chart.js [2] para la supervisión; matplotlib [8] para el análisis, y git para el control de versiones.

1.4. Alcance y limitaciones

Entra en el alcance todo el sistema electrónico y de software (placa, firmware, protocolo, supervisión y análisis), el modelo del mecanismo y la campaña de ensayos. El diseño mecánico se toma como dado.

Las limitaciones conocidas, que se discuten en los capítulos correspondientes, son:

- La posición del eje no se lee del codificador del accionamiento, sino que se obtiene integrando la velocidad que devuelve el propio servo (sección 7.5).
- La célula de carga no está calibrada con masas patrón; los ensayos se guardan en cuentas brutas y la conversión a newtons es provisional (sección 11.9).
- El par se obtiene en las unidades internas del accionamiento, cuya escala física no se ha confirmado, de modo que los resultados se expresan en variaciones relativas.
- Las cotas del mecanismo no se han medido: los parámetros geométricos proceden de un ajuste sobre los propios ensayos (capítulo 11).
- Los finales de carrera y el límite de fuerza se implementan por software; no hay una cadena de seguridad cableada (sección 5.5).

1.5. Metodología

El desarrollo ha sido incremental y guiado por pruebas. Cada subsistema (bus RS-485, célula de carga, finales de carrera, entradas y salidas de la placa) se validó primero con un programa de prueba aislado, que se conserva en el repositorio (anexo F), antes de integrarlo en el firmware principal. El código se ha versionado con git desde el primer día, y ese historial ha servido además para reconstruir la planificación real (capítulo 12). Los ensayos se guardan en crudo, con las constantes de conversión en la cabecera de cada fichero, de modo que cualquier resultado puede recalcularse sin

repetir el ensayo; las figuras de resultados de esta memoria se generan con programas reproducibles a partir de esos ficheros.

1.6. Estructura del documento

- El capítulo 2 resume los conceptos necesarios: accionamientos, medida de fuerza con galgas, buses de campo y microcontroladores.
- El capítulo 3 sitúa el trabajo frente a los tambores de radio variable y los mecanismos de compensación publicados.
- El capítulo 4 desarrolla el modelo del mecanismo y el método experimental con el que se contrasta.
- El capítulo 5 recoge los requisitos y la arquitectura del banco.
- Los capítulos 6 a 9 describen el hardware, el firmware, las comunicaciones y el software de supervisión.
- El capítulo 10 reúne los problemas técnicos y sus soluciones.
- El capítulo 11 presenta la caracterización del banco y los resultados de las dos campañas de ensayos.
- El capítulo 12 trata el ciclo de vida, la planificación y el presupuesto, y el capítulo 13 las conclusiones y el trabajo futuro.

Los anexos contienen la asignación de pines, el protocolo, los registros Modbus, el formato de los ficheros de ensayo, el manual de uso y la estructura del repositorio.

Capítulo 2

Marco teórico

Este capítulo resume los conceptos necesarios para seguir el resto de la memoria: cómo se acciona el eje, cómo se mide la fuerza, con qué microcontrolador y por qué buses viaja la información.

2.1. Transmisión por cable y radio variable

En una transmisión por cable, el par que hay que aplicar al tambor es el producto de la tensión del cable por el radio con el que ese cable sale del tambor:

$$\tau = T r. \tag{2.1}$$

Si el radio es constante, el par reproduce exactamente la forma de la tensión. Si el radio cambia con el giro, $r(\varphi)$, se dispone de un grado de libertad para moldear el par: basta con que el radio sea grande donde la tensión es pequeña, y pequeño donde es grande, para que el producto se mantenga aproximadamente constante. Ésa es la idea del *fusee* y de los tambores de radio variable, y el capítulo 4 la desarrolla para la geometría concreta del prototipo.

Conviene separar dos cosas que se confunden con facilidad. Un mecanismo de compensación *pasiva* (un muelle con una polea no circular) reduce la energía que el actuador tiene que aportar. Una caracola accionada por un motor no reduce esa energía: el trabajo de subir la masa es el mismo. Lo que hace es repartirlo de manera uniforme a lo largo del giro, lo que reduce el par de pico y, con él, el tamaño del motor necesario.

2.2. Accionamiento: servomotores paso a paso de lazo cerrado

Un motor paso a paso convencional funciona en lazo abierto: el controlador impone la secuencia de fases y se da por hecho que el rotor la sigue. Es sencillo y tiene buen par a baja velocidad, pero si la carga supera el par disponible se pierden pasos sin que el sistema lo detecte [12]. Los servomotores paso a paso de lazo cerrado añaden un codificador y un accionamiento que regula la corriente en función del error de posición: conservan el par a baja velocidad, no pierden pasos y ofrecen modos de control en posición, velocidad o par, además de magnitudes de supervisión como la velocidad real o el par entregado.

El accionamiento de este trabajo pertenece a esa familia **[COMPLETAR: marca y modelo del motor y del accionamiento, par nominal, tensión de alimentación y resolución del codificador]**. Integra una interfaz RS-485 con protocolo Modbus RTU, por la que se configura (modo de control, rigidez, rampas y resistencia de frenado), se habilita, se envía la consigna de velocidad y se leen la velocidad y el par. Tenerlo todo en un bus de dos hilos simplifica el cableado frente a la alternativa de señales de paso y dirección más entradas analógicas.

2.3. Medida de fuerza con células de carga

2.3.1. Galgas extensométricas y puente de Wheatstone

Una célula de carga convierte la fuerza en una deformación elástica, que se mide con galgas extensométricas dispuestas en un puente de Wheatstone [7]. La salida del puente es una tensión diferencial de unos pocos milivoltios por voltio de excitación a plena escala, que hay que amplificar con alta ganancia y bajo ruido antes de digitalizarla; además hay que compensar el desequilibrio de cero del puente y su deriva con la temperatura [25]. De ahí que la cadena de medida necesite, ante todo, una calibración con masas patrón: sin ella se conoce la forma de la señal, pero no su escala ni su cero.

2.3.2. Acondicionadores integrados

Existen circuitos integrados que agrupan excitación, amplificación, conversión A/D y corrección digital. En este trabajo se han evaluado dos:

- **ZSC31014** de Renesas [28], con salida digital I²C, preamplificador de ganancia programable, conversor de 14 bits y corrección almacenada en EEPROM. Cada lectura incluye dos bits de estado que indican si el dato es nuevo. Su configuración sólo puede modificarse en un *modo comando* al que se accede durante una ventana corta tras el encendido, lo que ha condicionado el diseño (sección 10.2). Se usa en una placa *Load Cell 2 Click* [17].
- **NAU7802** de Nuvoton [23], conversor sigma-delta de 24 bits con amplificador programable y regulador interno. Se probó como segunda célula y se retiró del firmware final.

2.4. Microcontrolador

El proyecto empezó con una placa basada en ESP32-S3 (Seeed XIAO) y se migró a una Raspberry Pi Pico 2. La tabla 2.1 resume las características del RP2350 [27]. El motivo del cambio y sus consecuencias se explican en la sección 10.1: los pines que impone el trazado de la placa no admiten una UART hardware, pero el RP2350 dispone de periféricos PIO capaces de implementar una UART en cualquier pin.

Tabla 2.1: Características del microcontrolador RP2350 en la Raspberry Pi Pico 2 [27].

Característica	Raspberry Pi Pico 2 (RP2350)
Núcleos	2 × Arm Cortex-M33 o 2 × RISC-V Hazard3, a 150 MHz
Memoria	520 kB SRAM, 4 MB flash QSPI
Periféricos serie	2 × UART, 2 × SPI, 2 × I ² C
E/S programable	12 máquinas de estado PIO
Entradas analógicas	3 canales ADC
Entorno usado	Arduino-Pico [10]

2.5. Buses de comunicación

2.5.1. RS-485 y Modbus RTU

RS-485 [31] es un estándar de capa física diferencial y multipunto, extendido en la industria por su inmunidad al ruido, su alcance y la posibilidad de conectar varios equipos al mismo par trenzado. En modo semidúplex cada nodo activa su emisor sólo mientras transmite, y ese control de dirección (líneas DE y $\overline{RE}$ del transceptor) corresponde al maestro.

Modbus [20] es un protocolo maestro-esclavo de petición y respuesta que modela los datos del esclavo como registros de 16 bits. En su variante RTU sobre línea serie [19], cada trama lleva la dirección del esclavo, el código de función, los datos y un CRC-16, y las tramas se delimitan por

silencios de al menos 3,5 caracteres. Aquí se usan la función 0x03 (lectura de registros) y la 0x06 (escritura de un registro), a través de la biblioteca ModbusMaster [34].

2.5.2. I²C y mikroBUS

I²C [24] es un bus síncrono de dos hilos con resistencias de *pull-up*, pensado para comunicar circuitos dentro de una misma placa; es el que usan los acondicionadores de célula de carga. Las placas Click siguen el estándar mikroBUS [18], que define un zócalo de 16 pines con I²C, SPI, UART, alimentación y señales auxiliares.

2.6. Adquisición y supervisión en el ordenador

La supervisión de máquinas se ha hecho tradicionalmente con paneles HMI o paquetes SCADA comerciales. Para un banco de laboratorio, una aplicación web local es una alternativa ligera: no requiere licencias, funciona en cualquier navegador y admite varios clientes a la vez. En este trabajo se emplea FastAPI [26] con pySerial [15] para el puerto USB, WebSocket [4] para el envío continuo de datos al navegador y Chart.js [2] para las gráficas.

Dos conceptos de adquisición son importantes para interpretar los resultados. El primero es la *base de tiempos*: si las muestras se marcan con el reloj del ordenador, la latencia del enlace se confunde con el fenómeno medido; por eso aquí cada muestra lleva la marca de tiempo del microcontrolador. El segundo es la distinción entre *dato bruto* y *dato calibrado*: guardar las cuentas del conversor, y no los newtons, permite recalibrar un ensayo sin repetirlo, que es justo lo que ha hecho falta en este trabajo.

Capítulo 3

Estado del arte

Este capítulo sitúa el trabajo frente a lo publicado sobre tambores de radio variable y mecanismos de compensación, y frente a las plataformas con las que se ensayan.

3.1. Del *fusee* a los tambores de radio variable

El problema de convertir una fuerza variable en un par constante es antiguo. En los relojes de muelle, el par del muelle real disminuye a medida que se desenrolla, y eso altera la marcha. El *fusee* lo resolvió con un cono de perfil curvo y una ranura helicoidal por la que se enrolla una cadena: con el muelle tenso la cadena tira del radio pequeño, y al debilitarse pasa al radio grande, de manera que el par entregado al tren se mantiene aproximadamente constante [14]. La condición de diseño es simplemente que el producto fuerza por radio se conserve a lo largo del recorrido.

La misma idea se ha formalizado en robótica con el nombre de tambores de radio variable (*Variable Radius Drums*, VRD). Seriani y Gallina [29] plantean la síntesis del perfil de un tambor para que la carga siga una ley prescrita y obtienen soluciones analíticas cerradas para varios casos. En los robots paralelos accionados por cables, Bieber *et al.* [1] diseñan el perfil a partir del perfil de carga máxima del espacio de trabajo, de modo que el par del motor no supere un valor dado para ninguna longitud de cable. La fabricación aditiva ha abaratado mucho estos perfiles, lo que explica el interés reciente.

3.2. Compensación de gravedad con poleas no circulares

Un segundo campo de aplicación es la compensación estática de peso. Herder [6] estudia de forma sistemática los mecanismos estáticamente equilibrados, en los que la energía potencial total se mantiene constante y el actuador sólo vence las pérdidas. Endo *et al.* [3] proponen una polea no circular combinada con un muelle para compensar el par gravitatorio de un brazo, y deducen numéricamente la forma de la polea. Shen y Yano [30] refinan ese cálculo teniendo en cuenta la variación de la tensión del cable a lo largo del recorrido, y mejoran así la calidad de la compensación.

El mecanismo de este trabajo comparte el principio, con una diferencia importante: aquí no hay muelle que almacene energía, sino un motor que aporta todo el trabajo de elevación. La caracola no ahorra energía, sino que reparte el par.

3.3. Bancos de ensayo y validación experimental

En los trabajos citados, la validación experimental suele consistir en medir el par o la corriente del actuador a lo largo del recorrido y compararlos con el perfil previsto. Esa comparación exige tres medidas sincronizadas: posición del eje, fuerza en el cable y par del actuador. Los equipos comerciales de adquisición que resuelven esto tienen un coste muy superior al del propio prototipo, y los accionamientos industriales modernos ya publican velocidad y par por bus de campo, de modo que buena parte de la instrumentación puede sustituirse por software.

Ésa es la oportunidad que aprovecha este TFG: construir, con componentes comerciales de bajo coste y herramientas libres, un banco que registre las tres magnitudes con la misma base de tiempos, guarde los datos en bruto y permita contrastar el modelo del mecanismo con la medida. Un resultado colateral del trabajo, que se detalla en el capítulo 11, es que esa instrumentación completa resulta *necesaria*: la medida de fuerza por sí sola no basta para determinar el perfil del tambor, porque la calibración desconocida de la célula admite soluciones con el signo invertido; es el par del accionamiento el que deshace la ambigüedad.

3.4. Conclusiones del estado del arte

Los tambores de radio variable están bien estudiados como principio y existen métodos de síntesis de su perfil. Lo que falta, y es lo que motiva este trabajo, es una plataforma asequible que permita medir en un prototipo real cómo se reparte el par y contrastarlo con el modelo. Para ello hay que integrar un accionamiento industrial por Modbus, un acondicionador de célula de carga con reglas de configuración estrictas y una cadena de adquisición con marcas de tiempo fiables, que es lo que desarrollan los capítulos siguientes.

Capítulo 4

Modelo del mecanismo

Este capítulo desarrolla el modelo estático del mecanismo: relaciona el giro del tambor con el ángulo de la barra, la longitud del cable y su tensión, y de ahí obtiene el par. Con ese modelo se comparan tres perfiles de tambor y se establece el método con el que, en el capítulo 11, se contrastan las medidas. Las mismas ecuaciones están implementadas en `static/mecanismo.js`, de modo que el simulador y el visor de ensayos calculan exactamente lo mismo que aquí se escribe.

4.1. Descripción del mecanismo

La figura 4.1 muestra el esquema. Una barra rígida articulada en O lleva suspendida una masa m a una distancia $L_3 + L_4$ de la articulación. El cable se ancla a la barra en el punto A, a una distancia L_3 de O, y sube hasta el punto de salida D, desde donde se enrolla en la caracola. El tambor gira solidario al eje de salida de la transmisión, con una reducción N de vueltas de motor por vuelta de tambor.

El punto D *no* está sobre la vertical de O: está a una altura L_2 y desplazado dx hacia el lado contrario a la barra. Ese detalle, que en una primera versión del modelo se pasó por alto, cambia la física y no sólo el dibujo, como se ve en la sección 4.3.

Como variable angular se usa α , el ángulo en O entre la línea O→D y la barra, que recorre de 0° (barra plegada sobre esa línea) a 180° (barra en su prolongación). El ángulo respecto a la horizontal es $\theta = \beta - \alpha$, siendo β la inclinación de O→D. Se prefiere α porque a cada longitud de cable le corresponde un único α , sin la ambigüedad que tenía medir desde la horizontal.

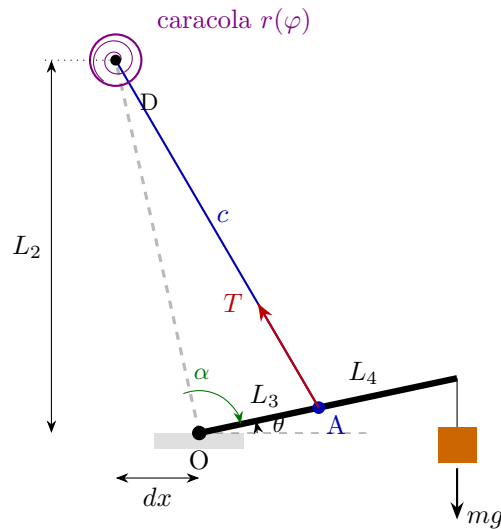


Figura 4.1: Esquema del mecanismo. El punto de salida del cable D está desplazado dx respecto de la vertical de la articulación O.

El modelo parte de las siguientes hipótesis:

- H1. Régimen cuasiestático: a velocidad constante se desprecian las fuerzas de inercia.
- H2. Cable inextensible y sin masa, y punto de salida D fijo.
- H3. Se desprecia la masa de la barra frente a la masa suspendida.
- H4. Las pérdidas por rozamiento no entran en el modelo estático: se tratan aparte en la sección 4.5.

4.2. Longitud del cable

Con $|OD| = \sqrt{L_2^2 + dx^2}$, la ley del coseno en el triángulo O–D–A da

$$c(\alpha)^2 = |OD|^2 + L_3^2 - 2|OD| L_3 \cos \alpha. \quad (4.1)$$

El coseno es monótono en $[0^\circ, 180^\circ]$, de modo que la correspondencia entre α y c es biunívoca: cada longitud de cable define un único ángulo, y al revés. Los extremos geométricos son $c_{\min} = ||OD| - L_3|$ con la barra plegada y $c_{\max} = |OD| + L_3$ con la barra alineada. Despejando,

$$\cos \alpha = \frac{|OD|^2 + L_3^2 - c^2}{2|OD| L_3}. \quad (4.2)$$

4.3. Tensión del cable

El momento del peso respecto a O es $mg(L_3 + L_4) \cos \theta$. El brazo de la tensión es la distancia de O a la recta D–A, que vale

$$d = \frac{|OD| L_3 \sin \alpha}{c}. \quad (4.3)$$

Igualando momentos, $T d = mg(L_3 + L_4) \cos \theta$, resulta

$$\boxed{T = \frac{m g (L_3 + L_4) c \cos \theta}{|OD| L_3 \sin \alpha}} \quad \theta = \beta - \alpha, \quad \beta = \arctan \frac{L_2}{-dx}. \quad (4.4)$$

Conviene detenerse en el caso particular $dx = 0$, que fue el primer modelo del proyecto. Entonces $\beta = 90^\circ$, $\theta = 90^\circ - \alpha$, $\cos \theta = \sin \alpha$, y los dos factores se cancelan:

$$T|_{dx=0} = \frac{m g (L_3 + L_4)}{L_2 L_3} c = K c, \quad (4.5)$$

es decir, *la tensión sería proporcional a la longitud libre de cable*. Es un resultado elegante, y por eso costó abandonarlo: con D desplazada la cancelación desaparece y la tensión deja de ser proporcional al cable. Además, en $\alpha \rightarrow 180^\circ$ el brazo del cable se anula mientras la pesa sigue haciendo momento, así que la tensión crece sin límite: ese punto es una singularidad del mecanismo y no un punto de trabajo.

La tabla 4.1 recoge los valores usados. No son cotas medidas, sino los parámetros con los que trabaja la configuración del banco, ajustados sobre los propios ensayos (sección 11.7).

Tabla 4.1: Parámetros del mecanismo empleados en el modelo [VERIFICAR: medir las cotas reales del prototipo y rehacer el ajuste; de ellas dependen todas las cifras de este capítulo].

Parámetro	Valor	Comentario
L_2	290 mm	altura de D sobre O
dx	65 mm	desplazamiento horizontal de D
$ OD $	297,2 mm	línea O→D, a 12,6° de la vertical
L_3	95 mm	anclaje del cable en la barra
L_4	110 mm	resto de barra hasta la masa
m	2 kg	masa suspendida
α_0	114°	ángulo en B, donde arranca el ensayo
c en B	346,9 mm	de (4.1)
$c_{\text{mín}} - c_{\text{máx}}$	202,2 mm a 392,2 mm	topes geométricos
$r_0 \rightarrow r_1$	45 mm a 25 mm	radios de la caracola configurados
Barrido	0,3 rev de tambor	recorrido en que el radio pasa de r_0 a r_1
N	5,5	reducción motor:tambor

4.4. Par en el tambor

El tambor paga cable según su radio instantáneo. Tomando como origen el punto B, donde arranca el ensayo, y contando el giro φ positivo en el sentido en que el ciclo avanza (de B hacia A):

$$s(\varphi) = \int_0^\varphi r(u) du, \quad c(\varphi) = c_B - s(\varphi), \quad \varphi = \frac{2\pi p}{N}, \quad (4.6)$$

donde p es el avance del eje del motor en revoluciones. El par en el tambor y en el motor son

$$\tau(\varphi) = T(c(\varphi)) r(\varphi), \quad \tau_m = \frac{\tau}{N\eta}, \quad (4.7)$$

con η el rendimiento de la transmisión. Para la caracola se modela un radio lineal en el giro (espiral de Arquímedes) entre r_0 y r_1 a lo largo del barrido φ_b :

$$r(\varphi) = r_0 + (r_1 - r_0) \frac{\varphi}{\varphi_b}, \quad 0 \leq \varphi \leq \varphi_b, \quad (4.8)$$

y fuera de ese tramo el radio se queda en el extremo que corresponda.

4.4.1. Cota inferior del par de pico

El trabajo que el tambor entrega a lo largo de un recorrido es el que gana la masa en energía potencial, y no depende del perfil:

$$W = \int_0^{\varphi_{\text{tot}}} \tau d\varphi = \int_{c_{\text{fin}}}^{c_{\text{ini}}} T(c) dc. \quad (4.9)$$

Como la integral del par está fijada, su máximo no puede ser menor que su media:

$$\max_{\varphi} \tau(\varphi) \geq \frac{W}{\varphi_{\text{tot}}}, \quad (4.10)$$

y la igualdad sólo se alcanza si el par es constante. *El perfil de par constante es, por tanto, el que minimiza el par de pico para un recorrido y un giro de tambor dados.* Este resultado no depende de la geometría: vale igual con $dx = 0$ que sin él.

4.4.2. Perfil ideal de par constante

Imponiendo $\tau = T(c)r = \tau_0$ y usando $dc/d\varphi = -r$:

$$r(\varphi) = \frac{\tau_0}{T(c(\varphi))}, \quad \frac{dc}{d\varphi} = -\frac{\tau_0}{T(c)}, \quad \tau_0 = \frac{W}{\varphi_{\text{tot}}}. \quad (4.11)$$

Con $dx = 0$, donde $T = Kc$, la ecuación se integra en forma cerrada y da $c(\varphi) = \sqrt{c_0^2 - 2(\tau_0/K)\varphi}$, con el radio inversamente proporcional al cable. Con $dx \neq 0$ no hay forma cerrada y la integración se hace numéricamente, que es como se ha calculado la curva de la figura 4.2.

La regla práctica se mantiene en ambos casos: *el radio debe ser grande donde la tensión es pequeña y pequeño donde la tensión es grande*.

4.4.3. Comparación de perfiles

La comparación se hace sobre el recorrido que los ensayos hacen realmente con carga en el cable (sección 11.1): un avance del motor de 0,65 a 1,80 rev, que son 0,209 rev de tambor y llevan el cable de 316,4 a 276,6 mm. En ese tramo el modelo da una tensión que *baja* de 44,2 a 35,5 N, porque el brazo del cable mejora más deprisa de lo que crece el momento del peso, y un trabajo $W = 1,58$ J.

Tabla 4.2: Perfiles de tambor sobre el recorrido con carga ($\varphi_{\text{tot}} = 0,209$ rev de tambor, $W = 1,58$ J).

Perfil	r inicial (mm)	r final (mm)	τ mín. – máx. (N · m)	Rizado $\frac{\text{máx} - \text{mín}}{\text{media}}$
Cilindro equivalente	30,3	30,3	1,06 – 1,35	24 %
Caracola configurada	41,2	27,9	0,88 – 1,65	65 %
Ideal (par constante)	27,2	33,8	1,20 – 1,20	0 %

El resultado es llamativo y conviene subrayarlo: con esta geometría, y en este tramo, *la caracola tal y como está configurada empeora el reparto del par* respecto a un cilindro que recorriese lo mismo (rizado del 65 % frente al 24 %). La razón es que la tensión baja a lo largo del recorrido, de modo que para compensarla el radio tendría que *crecer* un 24 %, y el perfil configurado lo hace decrecer un 32 %. Es decir: la caracola está descrita —o montada— al revés respecto de lo que pide la compensación.

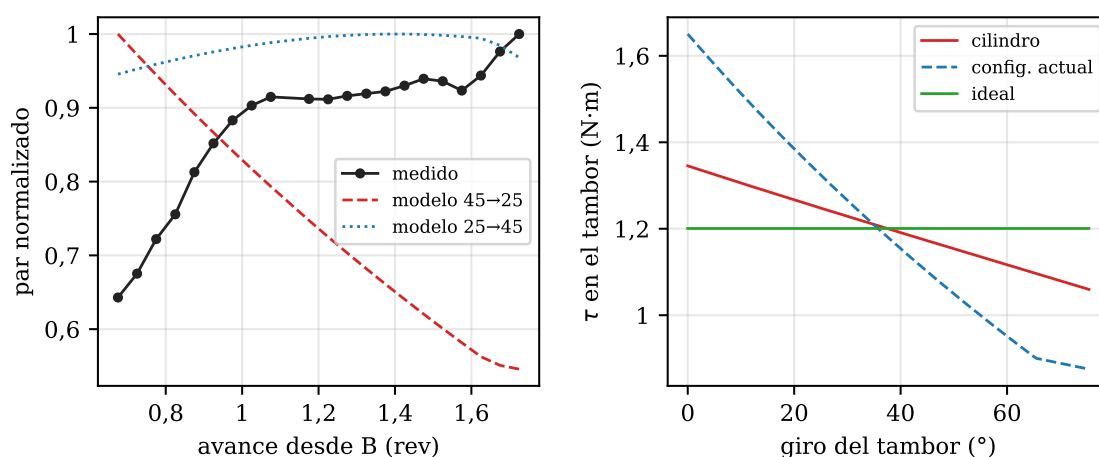


Figura 4.2: Izquierda: par medido (normalizado) frente a las dos orientaciones posibles del perfil. Derecha: par en el tambor para el cilindro equivalente, la caracola configurada y el perfil ideal.

4.5. Método experimental: separación del par y radio efectivo

El accionamiento entrega el par en unidades internas de escala desconocida, y el par medido incluye las pérdidas por rozamiento. Para comparar con el modelo sin depender de esa escala se recorre el mismo punto en los dos sentidos, a la misma velocidad. Las pérdidas se oponen siempre al movimiento, mientras que el par gravitatorio conserva el signo:

$$|\tau_{\uparrow}(p)| = \tau_g(p) + \tau_f(p), \quad |\tau_{\downarrow}(p)| = \tau_g(p) - \tau_f(p), \quad (4.12)$$

$$\tau_g = \frac{|\tau_{\uparrow}| + |\tau_{\downarrow}|}{2}, \quad \tau_f = \frac{|\tau_{\uparrow}| - |\tau_{\downarrow}|}{2}. \quad (4.13)$$

El mismo promedio aplicado a la señal de la célula elimina en primera aproximación el retardo de su filtro, que desplaza en sentidos opuestos las curvas de ida y de vuelta.

Con τ_g ya limpio de pérdidas, la ecuación (4.7) permite estimar el *radio efectivo*

$$r_{\text{ef}}(p) \propto \frac{\tau_g(p)}{T(p)}, \quad (4.14)$$

donde T se toma del modelo, que no necesita la célula de carga: sólo la geometría y la masa. En un tambor cilíndrico r_{ef} sería constante; su forma es, por tanto, una medida directa del perfil del tambor.

4.5.1. Por qué no basta con medir la fuerza

Podría pensarse que basta con comparar la señal de la célula con la tensión del modelo. No es así, y esto condiciona todo el capítulo de resultados. Como la célula no está calibrada, la comparación se hace ajustando una recta $T \approx a(\text{cuentas}) + b$, y el coeficiente de determinación de ese ajuste es el cuadrado del coeficiente de correlación:

$$R^2 = \rho^2. \quad (4.15)$$

El valor de R^2 *no cambia si se invierte el signo de a* . Una geometría que prediga una tensión creciente y otra que la prediga decreciente pueden dar exactamente el mismo R^2 frente a los mismos datos, y sólo difieren en el signo de la sensibilidad de la célula, que es precisamente lo que no se conoce. La medida de fuerza, por sí sola, no puede decidir en qué sentido trabaja el mecanismo.

El par del accionamiento sí puede: su signo y su escala son conocidos y no dependen de ninguna calibración pendiente. Por eso el criterio que se usa en el capítulo 11 es comparar τ_g con el par del modelo, y no sólo la fuerza.

4.6. Limitaciones del modelo

- Los parámetros de la tabla 4.1 no son cotas medidas, sino un ajuste sobre los ensayos; el propio capítulo 11 muestra que no son consistentes con el par medido.
- El punto de salida del cable se desplaza con el radio de enrollamiento, lo que introduce un error de algunos milímetros en L_2 y dx .
- Se desprecian la elasticidad del cable y la masa de la barra.
- El rendimiento y las pérdidas se suponen iguales en los dos sentidos, en valor absoluto, para poder aplicar (4.13).
- El modelo es estático: no describe el tramo inicial en que el cable está destensado, ni los transitorios de arranque e inversión.

Capítulo 5

Requisitos y arquitectura del banco

5.1. Requisitos

Los requisitos salen de lo que exige el ensayo de la caracola (capítulo 4). El eje debe moverse a velocidad constante y conocida en los dos sentidos, y hay que registrar simultáneamente, con la misma base de tiempos, la posición, la tensión del cable y el par del motor. Todo ello de forma segura y repetible.

5.1.1. Requisitos funcionales

Id.	Requisito
RF1	Configurar y habilitar el accionamiento y gobernar su velocidad por Modbus RTU sobre RS-485.
RF2	Leer la velocidad y el par del motor en cada periodo de control.
RF3	Leer la tensión del cable con una célula de carga y un acondicionador digital.
RF4	Detectar los finales de carrera A (arriba) y B (abajo) y detener el movimiento hacia un final activado.
RF5	Ejecutar una rutina de <i>homing</i> que localice A, mida el recorrido hasta B y fije el origen en un desplazamiento configurable.
RF6	Ejecutar ciclos de ensayo B→A→B: colocarse en el extremo inferior, subir hasta A y volver a bajar, con velocidad y número de repeticiones configurables.
RF7	Enviar al ordenador la telemetría de cada periodo con su marca de tiempo del dispositivo.
RF8	Recibir órdenes y parámetros desde el ordenador con detección de errores.
RF9	Consultar y programar la ganancia y el cero del acondicionador de la célula, sólo con el eje parado.
RF10	Mostrar el estado de la máquina y los datos en tiempo real en un navegador.
RF11	Grabar automáticamente cada ensayo en un fichero con datos brutos y constantes de conversión.
RF12	Visualizar los ensayos grabados y compararlos con el modelo teórico.
RF13	Mantener una única configuración de la geometría del mecanismo, editable desde la interfaz y compartida por el simulador y el visor de ensayos.

5.1.2. Requisitos no funcionales

Id.	Requisito
RNF1	Determinismo: periodo de control y muestreo fijo de 50 ms, con dispersión inferior a 1 ms.
RNF2	Latencia de parada: una orden de parada, un final de carrera o el límite de fuerza deben anular la consigna en el mismo periodo en que se detectan.
RNF3	Robustez de la comunicación: una trama corrupta no debe ejecutar ninguna orden, y el receptor debe resincronizarse solo.
RNF4	Recuperación: la aplicación debe reconectarse sola si se desconecta o reinicia el microcontrolador.
RNF5	Reproducibilidad: cualquier resultado debe poder recalcularse a partir de los ficheros grabados.
RNF6	Coste y disponibilidad: componentes comerciales y herramientas libres.
RNF7	Mantenibilidad: validación de cada subsistema con un programa de prueba propio y código documentado.
RNF8	Coherencia del modelo: una sola implementación de la geometría para todas las páginas, para que el simulador y el análisis no puedan discrepar.

5.2. Arquitectura general

La figura 5.1 muestra la arquitectura, organizada en tres niveles:

1. **Nivel de campo:** el servomotor paso a paso con su accionamiento, la transmisión, la caracola, el cable y la barra con la masa; la célula de carga intercalada en el cable y los dos finales de carrera.
2. **Nivel de control:** la placa adaptadora con la Raspberry Pi Pico 2. Ejecuta el lazo de 50 ms y es el único elemento que actúa sobre la máquina.
3. **Nivel de supervisión:** un ordenador con el servidor `web_monitor.py`, que traduce entre el protocolo binario del puerto USB y mensajes JSON por WebSocket, graba los ensayos, guarda la configuración del mecanismo y sirve las tres páginas web.

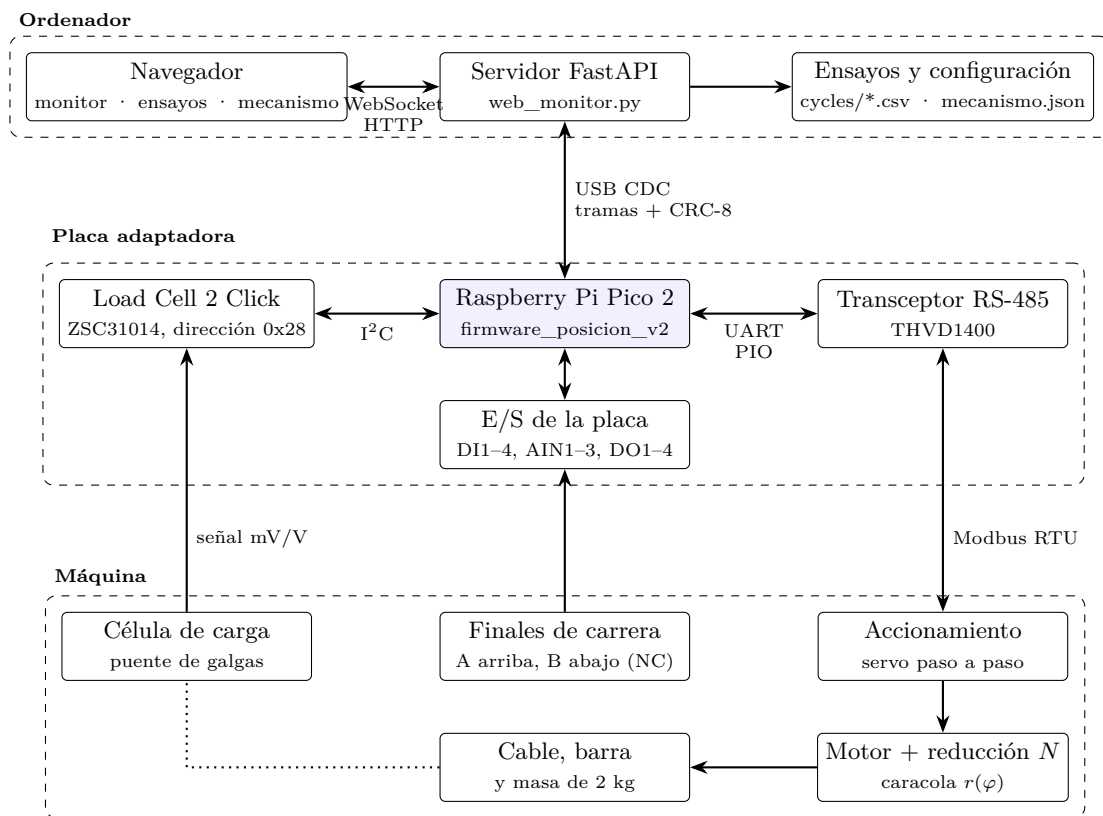


Figura 5.1: Arquitectura del banco de ensayos.

5.3. Flujo de datos y temporización

El microcontrolador ejecuta un lazo de periodo fijo (sección 7.2). En cada vuelta lee el estado del servo y de la célula, integra la posición, procesa las órdenes recibidas, actualiza las máquinas de estado de *homing* y de ciclo, aplica las protecciones y escribe la nueva consigna; además guarda una muestra de la vuelta en un búfer. Cada 200 ms (o antes, si el búfer se llena) el búfer se envía en una sola trama, con una marca de tiempo por muestra. El servidor desempaqueta las muestras, las reenvía al navegador y, si hay un ensayo en marcha, las escribe en su fichero. Toda la cadena usa el reloj del microcontrolador, de modo que las gráficas y los ficheros reflejan los instantes reales de medida y no los de llegada al ordenador.

La geometría del mecanismo sigue el camino contrario: se edita en la página de configuración, se valida y se guarda en el servidor (`mecanismo.json`), y desde ahí la leen tanto el simulador como el visor de ensayos, de modo que el modelo con el que se compara una medida es siempre el mismo.

5.4. Decisiones de diseño

Tabla 5.3: Decisiones de diseño y alternativas consideradas.

Aspecto	Elección	Justificación / alternativa descartada
Microcontrolador	Raspberry Pi Pico 2	Sustituye al ESP32-S3 inicial. La placa se diseñó para el módulo Pico, y su PIO permite una UART en los pines que imponía el trazado.

Aspecto	Elección	Justificación / alternativa descartada
Modo del servo	Velocidad	Los primeros firmwares usaban el modo par con un regulador sobre la célula. El ensayo cuasiestático y el método de separación del par (sección 4.5) exigen velocidad constante e igual en ambos sentidos.
Organización del firmware	Un único dueño de cada bus en el núcleo 0	La versión de doble núcleo dedicaba el núcleo 1 a Modbus con variables compartidas. El lazo único de periodo fijo elimina las condiciones de carrera y hace predecible la temporización.
Origen del ciclo	Arranca en B, el extremo inferior	El ensayo tiene sentido físico desde la extensión completa del mecanismo. Como el <i>homing</i> deja el eje en A, el ciclo se coloca primero en B, y ese tramo ni cuenta como repetición ni se graba.
Extremos del ciclo	Finales de carrera	Reconocer A y B por los finales, y no por la posición integrada, hace que el ciclo funcione esté donde esté el cero y sin depender de la escala de velocidad del accionamiento.
Protocolo con el PC	Binario con CRC-8 y telemetría por lotes	Un protocolo de texto ocupa más y es ambiguo de delimitar. Las tramas binarias con cabecera, longitud y CRC se resincronizan solas.
Posición	Integración de la velocidad del servo	No requiere cablear el codificador. Su deriva se acota con el <i>homing</i> y los finales de carrera (sección 7.5).
Células de carga	Una sola (ZSC31014)	El NAU7802 se evaluó como segunda célula y se retiró: compartían bus y el ZSC31014 no tolera transacciones ajenas entre la petición y la recogida del dato.
Supervisión	Aplicación web local	No requiere licencias, admite varios clientes y separa la operación en vivo del análisis.
Modelo del mecanismo	Una implementación única en el servidor	Cada página tenía su copia de las fórmulas y llegaron a discrepar: el monitor seguía con un cilindro mientras el visor usaba la caracola. Ahora la geometría vive en <code>mecanismo.json</code> y las fórmulas en <code>mecanismo.js</code> .
Datos grabados	Cuentas brutas + constantes en la cabecera	Permite recalibrar un ensayo sin repetirlo, que es justo lo que ha hecho falta.

5.5. Seguridad

El prototipo mueve una masa pequeña a baja velocidad, pero el sistema incluye varias medidas de seguridad, todas implementadas en el firmware:

- **Finales de carrera normalmente cerrados** conectados entre la entrada y masa, con *pull-up* interna. En reposo la entrada lee nivel bajo, y un final pulsado o un cable roto dan nivel alto: la rotura de cable se interpreta como final activado (comportamiento a prueba de fallos).
- **Parada hacia el final activado:** en cada periodo se anula cualquier consigna que empuje hacia un final pulsado, sea cual sea el modo (manual, *homing* o ciclo).
- **Límite de fuerza** configurable: si el valor absoluto de la célula tarada supera el umbral, la consigna se anula. Sólo actúa con una lectura válida reciente (menos de 150 ms).
- **Temporizadores de tramo** de 30 s en el *homing* y en los ciclos.

- **Bloqueo durante la programación de la célula**, que la deja sin medida durante cientos de milisegundos: se rechaza si el eje no está parado y, mientras dura, se fuerza la consigna a cero en cada vuelta.
- **Orden de parada** que corta también el *homing* y los ciclos, y orden de apagado que deshabilita el accionamiento.

Estas medidas dependen del software y del enlace Modbus. En una máquina industrial, las normas de seguridad funcional exigirían una cadena de parada independiente del control [11], por ejemplo cableando los finales de carrera a la entrada de desconexión segura del par (STO) del accionamiento [9]. Se recoge como trabajo futuro en el capítulo 13.

Capítulo 6

Diseño hardware

6.1. Placa adaptadora: objetivos

El módulo Raspberry Pi Pico 2 trabaja a 3,3 V y no puede conectarse directamente al entorno de la máquina. La placa adaptadora (proyecto KiCad `tfg_adaptador`) resuelve esa adaptación y reúne en un solo circuito:

- un transceptor RS-485 semidúplex para el bus Modbus del accionamiento;
- cuatro entradas digitales protegidas para finales de carrera y otros contactos;
- entradas analógicas de 0 V a 5 V acondicionadas para el ADC del microcontrolador;
- cuatro salidas de potencia de lado alto con diagnóstico;
- dos zócalos mikroBUS para placas Click de célula de carga;
- bornas de tornillo para todo el cableado de campo y taladros de fijación M3.

6.2. Revisiones del diseño

El repositorio contiene dos estados del diseño, que conviene distinguir:

- **Revisión A:** los ficheros `tfg_adaptador.kicad_sch` y `.kicad_pcb`, creados entre el 12 y el 17 de marzo de 2026. Tienen el esquema, la colocación de los componentes y los planos de cobre (figuras 6.2 y 6.3).
- **Revisión B:** el esquema de la figura 6.1, incorporado al repositorio como imagen. Es la asignación de pines que usan todos los programas de prueba de la placa y el firmware final **[VERIFICAR: añadir al repositorio los ficheros KiCad de la revisión B, que no están versionados, y regenerar las figuras de la PCB]**.

El análisis de la lista de conexiones de la revisión A, extraída con `kicad-cli`, muestra una serie de problemas que la revisión B corrige (tabla 6.1). Documentarlos es útil, porque son errores típicos de un primer diseño.

Tabla 6.1: Problemas detectados en la revisión A y corrección en la revisión B.

Bloque	Revisión A (lista de conexiones)	Revisión B
RS-485	La salida RO del transceptor está en la misma red que 3,3 V y GPIO13. DI, DE y $\overline{RE}$ están unidas entre sí y a GPIO14 y GPIO15. El transceptor no puede funcionar en semidúplex.	DE y $\overline{RE}$ a GPIO12 con <i>pull-down</i> de 10 k Ω ; RO a GPIO13 con <i>pull-up</i> de 10 k Ω ; DI a GPIO14.
Entradas analógicas	Las salidas de los seguidores llegan a GPIO16–19, que no tienen ADC.	Salidas a GPIO26–28 (ADC0–2).
Amplificadores	Cuatro MCP6001 sin alimentación: pines V+ y V– sin conectar.	Un MCP6294 cuádruple alimentado a 5 V con desacoplo de 100 nF.
Salidas de potencia	El TPS4H000 se alimenta de VBUS (5 V del USB); diagnóstico ($\overline{ST}$), DIAG_EN, CL y THER sin conectar; entradas directas.	Alimentación desde la borna VS_IN; $\overline{ST1}$ – $\overline{ST4}$ unidas a DST con <i>pull-up</i> ; DIAG_EN controlado; resistencias de 10 k Ω en serie con las entradas.
Células de carga	Cada zócalo en un bus I ² C distinto (GPIO0/1 y GPIO2/3), sin control de EN ni lectura de INT.	Ambos zócalos en I ² C1 (GPIO2/3), habilitación EN1 (GPIO1) e interrupciones INT1 (GPIO0) e INT2 (GPIO4).
Entradas digitales	Llegan a GPIO4–7.	Llegan a GPIO16–19.
Rutado	El fichero de PCB no contiene pistas (0 segmentos): sólo colocación, 23 vías y planos.	[COMPLETAR: indicar cómo se fabricó la placa definitiva: PCB de revisión B, placa perforada o cableado manual]

6.3. Microcontrolador

La placa aloja el módulo Raspberry Pi Pico 2 en sus dos filas de 20 pines, de modo que puede sustituirse sin desoldar. El módulo se alimenta y comunica por USB. De él se toman 5 V (VBUS) para los amplificadores y 3,3 V (salida 3V3 del regulador del módulo) para el transceptor y las placas Click. La asignación completa de pines está en el anexo A.

6.4. Interfaz RS-485

Se usa el transceptor THVD1400 de Texas Instruments [32]: semidúplex, alimentación de 3 V a 5,5 V, hasta 500 kbit/s, 1/8 de carga unitaria (hasta 256 nodos), protección ESD de ± 16 kV HBM y ± 12 kV IEC 61000-4-2 por contacto, y protección ante fallos de bus de ± 16 V. Esta robustez es adecuada para un cable que convive con los de potencia del motor.

Las señales DE y $\overline{RE}$ están unidas y se controlan desde GPIO12. Una resistencia de 10 k Ω a masa deja el transceptor en recepción durante el arranque, cuando el pin aún no está configurado, para que no ocupe el bus. La salida RO lleva una *pull-up* de 10 k Ω para que la línea de recepción no quede flotando. El bus sale por la borna de tres polos J5 (A, B y masa). El firmware activa el emisor antes de cada trama y lo desactiva 200 μ s después del último byte (sección 10.1). No hay terminación en la placa: el enlace es punto a punto y corto **[VERIFICAR: longitud del cable y si el accionamiento incorpora terminación de 120 Ω]**.

6.5. Entradas digitales

Cada entrada digital (DI1–DI4) pasa por una resistencia serie de $1\text{ k}\Omega$ y un diodo zéner BZX384 a masa [22] antes de llegar al GPIO (figura 6.1). La resistencia limita la corriente y el zéner recorta las sobretensiones y las descargas que entran por el cable. Los finales de carrera, de contacto normalmente cerrado, se conectan entre la borna y masa, y el firmware activa la *pull-up* interna del GPIO. En el banco, DI1 es el final A (arriba, donde queda el *homíng*) y DI2 el final B (abajo, donde arranca el ensayo).

La tensión que admite la entrada de forma continua está limitada por la potencia de la resistencia. Con un zéner de tensión V_Z y una resistencia de $0,125\text{ W}$ (encapsulado 0805), la tensión de entrada máxima es

$$V_{\text{in,max}} = V_Z + \sqrt{P_R R} \approx V_Z + \sqrt{0,125 \cdot 1000} \approx V_Z + 11,2\text{ V}. \quad (6.1)$$

Para contactos secos es más que suficiente. Si se quisieran conectar sensores de 24 V , habría que subir la resistencia a unos $10\text{ k}\Omega$ [COMPLETAR: tensión nominal del zéner BZX384 montado].

6.6. Entradas analógicas

Las entradas AIN1–AIN3 admiten 0 V a 5 V . Cada canal tiene un divisor resistivo de $17\text{ k}\Omega$ y $33\text{ k}\Omega$, con relación $33/50 = 0,66$, que lleva 5 V a $3,3\text{ V}$, seguido de un amplificador operacional en seguidor de tensión. El seguidor presenta al ADC una impedancia de fuente baja, necesaria para cargar el condensador de muestreo en cada conversión sin error, y aísla el divisor del ADC. Se usa el MCP6294 [16], cuádruple, *rail-to-rail* en entrada y salida y alimentado a 5 V .

Con el ADC de 12 bits del RP2350 y referencia de $3,3\text{ V}$, la resolución referida a la entrada es

$$\Delta V_{\text{in}} = \frac{3,3\text{ V}}{4095 \cdot 0,66} \approx 1,22\text{ mV}. \quad (6.2)$$

El cuarto amplificador se usa como comparador con un umbral de $2,5\text{ V}$, obtenido de 5 V con un divisor de dos resistencias de $10\text{ k}\Omega$. Su salida se adapta a $3,3\text{ V}$ con otro divisor $17/33$ y llega a una entrada digital (COMP).

El valor de $17\text{ k}\Omega$ no pertenece a la serie E24. Para la fabricación conviene usar $17,4\text{ k}\Omega$ (E96), con relación $0,655$, o recalibrar el factor en el firmware.

6.7. Salidas de potencia

Las cuatro salidas usan el interruptor inteligente de lado alto TPS4H000-Q1 [33]: cuatro canales, alimentación de $3,4\text{ V}$ a 40 V , $1000\text{ m}\Omega$ de resistencia en conducción, límite de corriente ajustable entre $0,1$ y $1,3\text{ A}$, detección de sobrecorriente, cortocircuito y carga abierta, protección térmica y capacidad para cargas inductivas. Las entradas IN1–IN4 se gobiernan desde GPIO7–10 a través de resistencias de $10\text{ k}\Omega$. Las salidas de estado, en drenador abierto, se unen en una línea común de fallo (DST) con *pull-up*, y la habilitación del diagnóstico se controla con DIAG_EN. El firmware final no usa todavía estas salidas; su funcionamiento se verificó con el programa `pcbtest/test_digital_outputs` [COMPLETAR: uso previsto de las salidas y resistencia de límite de corriente elegida].

6.8. Célula de carga

La célula de carga se conecta a una placa *Load Cell 2 Click* de MikroElektronika [17], basada en el acondicionador ZSC31014 [28], que se inserta en el zócalo mikroBUS MD1. El chip responde en la dirección I²C `0x28`, admite alimentación de $2,7\text{ V}$ a $5,5\text{ V}$ y ofrece ocho ganancias de preamplificador ($\times 1,5$ a $\times 192$). Su señal EN, controlada desde GPIO1 con la corriente de salida del pin configurada a 12 mA , permite apagarlo y encenderlo para forzar un reinicio y abrir la ventana del modo comando. Esta posibilidad resultó imprescindible, y también problemática (sección 10.2).

El segundo zócalo (MD2) comparte el bus y se usó para evaluar una *Load Cell 4 Click* con NAU7802 [23] **[COMPLETAR: modelo, capacidad nominal y sensibilidad (mV/V) de la célula de carga, y forma de montaje en el cable]**.

6.9. Alimentación

La lógica se alimenta del USB del ordenador a través del módulo Pico. Las salidas de potencia toman su tensión de la borna VS_IN y el accionamiento tiene su propia fuente **[COMPLETAR: tensión y potencia de la fuente del accionamiento]**. Las masas se unen en la placa, y la masa del bus RS-485 se lleva también a la borna J5 para dar al transceptor una referencia común con el accionamiento.

6.10. Placa de circuito impreso

La PCB de la revisión A es de dos capas, mide 123 mm × 75 mm y tiene cuatro taladros M3 en las esquinas. Los componentes pasivos son SMD 0805, los diodos SOD-323, los amplificadores SOT-23-5, el transceptor SOIC-8 y el interruptor de potencia HTSSOP-20 con almohadilla térmica. Las bornas de campo están agrupadas en el lado izquierdo, el módulo Pico en el lado derecho con el conector USB hacia el borde, y los dos zócalos mikroBUS arriba a la derecha, de modo que las placas Click quedan accesibles.

6.11. Validación de la placa

Cada bloque se validó con un programa de prueba independiente antes de la integración (tabla 6.2).

Tabla 6.2: Pruebas de la placa adaptadora.

Programa	Procedimiento y criterio	Resultado
pcbtest/test_analog	Llevar AIN a GND, 3,3 V y 5 V; la tensión reconstruida debe ser 0, 3,3 y 5,0 V.	[COMPLETAR: lecturas]
pcbtest/test_din	Cambiar el estado de DIN1–4 y comprobar que se detecta cada flanco.	[COMPLETAR:]
pcbtest/test_digital_outputs	Conmutar SIN1 cada segundo y medir la salida.	[COMPLETAR:]
test_limit_switches	Pulsar cada final de carrera: nivel bajo en reposo y alto activado.	Correcto (usado en todos los ensayos)
test_rs485_loopback	Puente GP14–GP13 sin transceptor: debe recibirse la trama enviada a 9600 y 115200 baudios.	Correcto
test_rs485_tx/rx	Transmisión y recepción a través del transceptor.	Correcto
test_modbus_scan	Lectura periódica del registro 0 del esclavo 1 a 115200 baudios.	Correcto (código 0x00)
test_loadcell	Detección del ZSC31014 en 0x28 y lectura del puente con estado.	Correcto

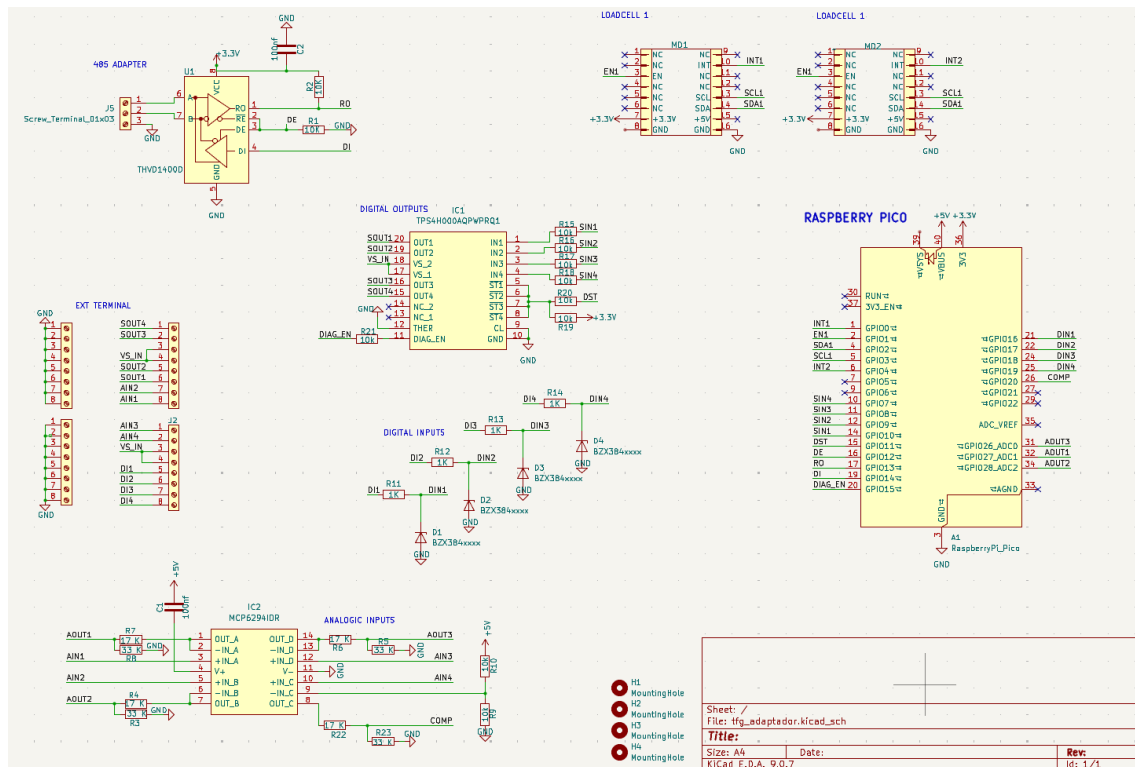


Figura 6.1: Esquema de la placa adaptadora, revisión B.

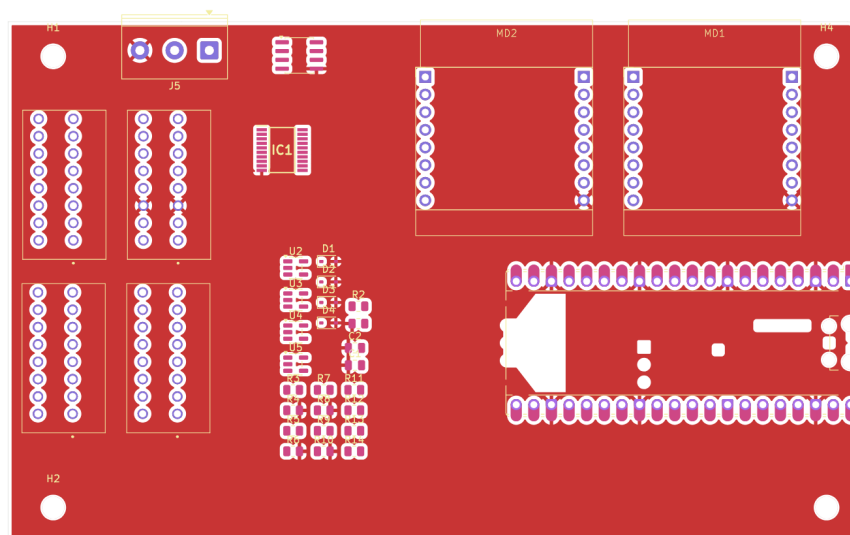


Figura 6.2: PCB de la revisión A: capas de cobre, máscara y serigrafía superiores.

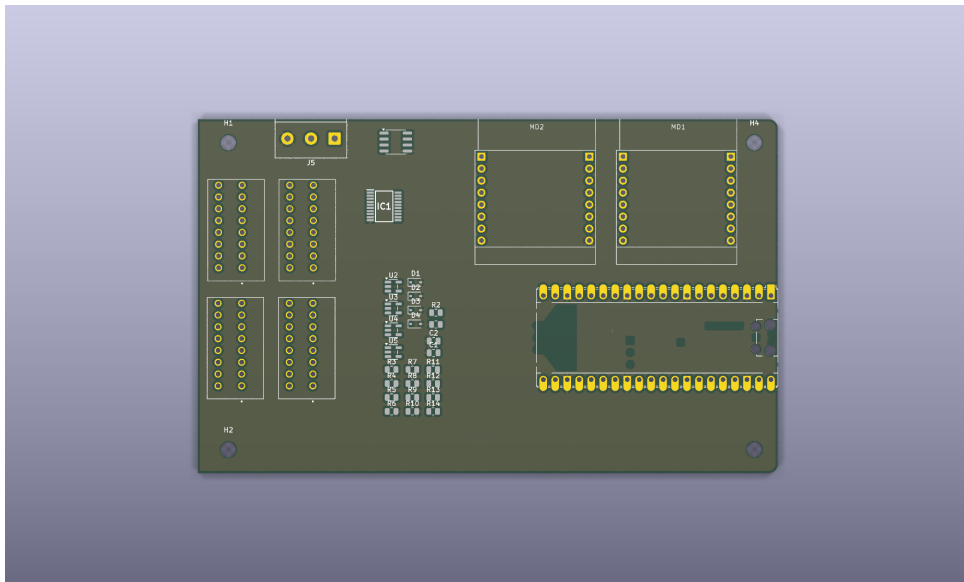


Figura 6.3: Vista 3D de la PCB de la revisión A generada con KiCad.

Capítulo 7

Firmware del microcontrolador

7.1. Entorno y organización

El firmware está escrito en C++ con el entorno Arduino y el núcleo Arduino-Pico [10], que da acceso a los periféricos del RP2350 y a su SDK. Usa las bibliotecas `Wire` (I²C), `SerialPIO` (UART implementada con PIO) y `ModbusMaster` [34]. El código final está en `firmware_posicion_v2/`: un fichero principal de unas 1500 líneas y la cabecera `i2c_bus.h` con el árbitro del bus I²C.

El diseño sigue un principio que se repite en todo el código: *cada bus tiene un único dueño y un único punto del lazo donde se usa*. Sólo dos funciones tocan el bus RS-485 (`servoReadState` y `servoAction`) y sólo dos tocan el I²C durante el funcionamiento (`i2cUpdate` y `lcRequestService`). El intérprete de órdenes (`processCommand`) nunca accede a un bus: se limita a modificar variables de estado, que el lazo lee en el punto que corresponde. Así, la llegada de una orden no puede interrumpir una transacción a medias.

El estado vive en dos estructuras: `ControlState ctrl` (modo, errores, parámetros, célula, posición, *homing* y ciclo) y `Servo sv` (fase del enlace, dirección, conexión, contadores de fallos, consigna pendiente y últimas lecturas).

7.2. Lazo principal

El núcleo 0 ejecuta un lazo con periodo mínimo de 50 ms (`LOOP_MS`). Cada vuelta tiene cinco fases (código 7.1).

```
1 void loop(){
2   unsigned long now = millis(); uint32_t t0us = micros();
3
4   // == LECTURA ==
5   servoReadState();      // RS-485: velocidad y par (2 transacciones)
6   lcRequestService();    // consulta/programación de la célula si se ha pedido
7   i2cUpdate();           // I2C: una lectura de la célula
8   positionUpdate();      // integra rpm -> posición (sin buses)
9
10  // == LÓGICA == (sin acceso a buses)
11  checkCommands();        // órdenes USB -> variables de estado
12  if (ctrl.lcProgBusy)    { sv.speedCmd = 0; ctrl.cycPhase = CYC_IDLE; }
13  else if (homing en curso) homingUpdate();
14  else                    { homingUpdate(); cycleUpdate(); }
15  // protecciones: límite de fuerza y finales de carrera anulan la consigna
16  // derivación del estado de la aplicación para la telemetría
17
18  // == ACTUACIÓN ==
19  servoAction();          // RS-485: máquina de estados del servo y escritura de consigna
20
21  // == REGISTRO ==
22  recordSample(now, micros() - t0us);
23  if (sampleCount >= MAX_SAMPLES || now - ctrl.lastTelemetryTime >= 200)
24    telemetryFlush();     // + tramas de homing y de ciclo si están activos
25}
```

```

26 // == RITMO ==
27 unsigned long dt = millis() - now;
28 if (dt < LOOP_MS) delay(LOOP_MS - dt);
29 }

```

Código 7.1: Estructura del lazo principal (simplificada).

Este orden tiene consecuencias deliberadas:

- Las protecciones se evalúan *después* de la lógica y *antes* de la escritura: ninguna consigna generada por el *homing*, el ciclo o una orden manual llega al accionamiento sin pasar por ellas.
- Una orden recibida se aplica en la misma vuelta, porque la escritura de la consigna es posterior a la lectura de órdenes.
- El tiempo de trabajo (*work_us*) se mide antes del retardo de ritmo y viaja con cada muestra. Es la base de la caracterización temporal de la sección 11.2.
- El retardo final hace que el periodo no dependa de la duración del trabajo mientras ésta no supere 50 ms.

El núcleo 1 no se usa (*loop1()* sólo espera). La evolución desde la versión de doble núcleo se explica en la sección 10.1.

7.3. Máquina de estados del servo

La figura 7.1 muestra la máquina de estados del enlace con el accionamiento, que ejecuta *servoAction*.

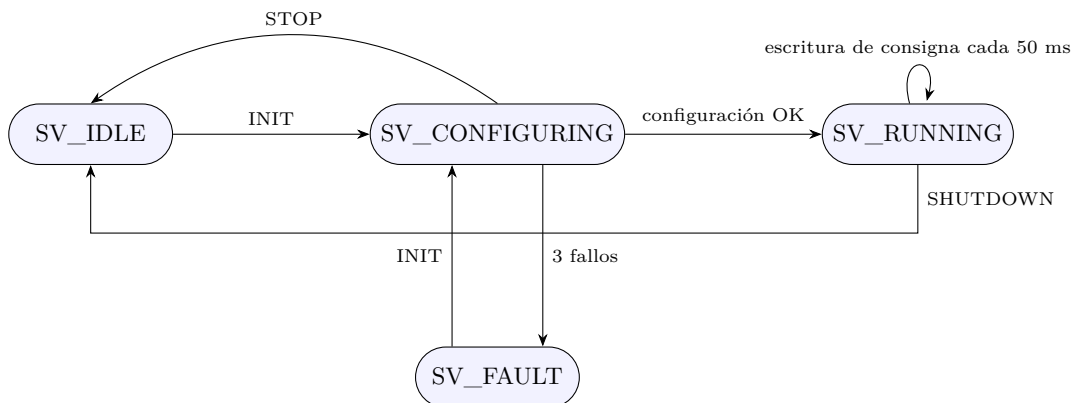


Figura 7.1: Máquina de estados del enlace con el accionamiento.

En *SV_CONFIGURING* se ejecuta la secuencia de configuración del capítulo 8. En *SV_RUNNING* cada vuelta escribe la consigna: un fallo marca el servo como desconectado y pone a cero las lecturas, pero no provoca un nuevo escaneo del bus, que es bloqueante y lento (hasta un segundo por dirección vacía). El escaneo sólo se hace bajo demanda con la orden *SCAN*.

La orden *INIT*, además de reiniciar la posición, invalida el recorrido A→B medido: poner el cero donde esté el eje deja sin sentido la referencia anterior, y sin esa invalidación el ciclo podría creer que A es el punto actual.

7.4. Lectura de la célula de carga

El ZSC31014 trabaja en modo de conversión continua: convierte a su propio ritmo, fijado por la palabra *ZMDI_Config1*, de 0,5 a 125 ms según el reloj y la tasa elegidos [28], y guarda el último resultado. Una lectura no dispara una conversión: devuelve el último dato con dos bits de estado que indican si es nuevo o si ya se había leído.

Por ello, `loadCellUpdate` hace *una sola lectura de dos bytes por vuelta*:

1. Pide el bus al árbitro. Si está ocupado, pierde el turno y lo intenta en la vuelta siguiente.
2. Lee 14 bits de dato y 2 de estado. Si el estado es “dato válido”, lo introduce en una media móvil de 8 posiciones y calcula la lectura tarada `baseRead = media - zeroOffset`.
3. Si pasan más de 150 ms sin un dato válido, marca la célula como no válida y el límite de fuerza deja de actuar, en lugar de hacerlo con un dato viejo.

7.4.1. Árbitro del bus I²C

La cabecera `i2c_bus.h` implementa un árbitro mínimo: una variable con el dueño actual (`I2C_FREE`, `I2C_LC1`, `I2C_LC2` o `I2C_CFG`) y dos funciones, `i2cAcquire` e `i2cRelease`. La reserva se deniega si el bus tiene otro dueño o si se pide desde el núcleo 1, y cada denegación se cuenta para el diagnóstico. El árbitro nació al compartir el bus con el NAU7802 (sección 10.8) y se conserva para reservar el bus en exclusiva durante la programación de la EEPROM. Nadie espera con el bus ocupado, así que el árbitro no puede bloquear el lazo.

7.5. Posición integrada

La posición del eje del motor se obtiene integrando la velocidad que devuelve el accionamiento:

$$p_k = p_{k-1} + \frac{n_k \Delta t_k}{60000}, \quad [p] = \text{rev}, [n] = \text{rpm}, [\Delta t] = \text{ms}. \quad (7.1)$$

Es una integración rectangular con la última velocidad leída y el intervalo real medido con `millis()`. Hay tres aspectos que tener en cuenta:

- **Rebase tras un hueco.** Si el lazo se detiene, por ejemplo durante la programación de la célula, la siguiente vuelta vería un Δt de casi un segundo y lo multiplicaría por la última velocidad, inventando un desplazamiento que no ha ocurrido. Por eso se anula la base de tiempo (`lastPosTime = 0`) tras INIT, al iniciar el *homing* y tras cualquier operación bloqueante.
- **Resolución.** El registro de velocidad tiene resolución de 1 rpm: si el accionamiento redondea, el error es de media nula; si trunca, a 30 rpm introduciría un sesgo de hasta el 3 %.
- **Acotación.** La deriva no se acumula entre ensayos, porque el ciclo reconoce sus extremos por los finales de carrera y no por la posición integrada.

Esta última decisión es la que hace que el banco siga siendo utilizable pese a la incertidumbre de la posición: el recorrido del ensayo queda definido por dos contactos físicos, y la posición integrada sólo sirve como eje de referencia dentro del ensayo.

7.6. Homing

La rutina de *homing* (figura 7.2) establece un origen repetible y mide el recorrido disponible:

1. **Buscar A:** avanza hacia A (arriba) a la velocidad de *homing* hasta que se activa el final A, y fija ahí $p = 0$.
2. **Buscar B:** avanza hacia B (abajo) hasta el final B y guarda el recorrido p_B .
3. **Ir al origen:** se mueve hasta el desplazamiento configurado, acotado a $[0, p_B]$, con tolerancia de $\pm 0,02$ rev.
4. Fija ese punto como nuevo origen ($p = 0$) y sigue informando del resultado durante 1,5 s para que la interfaz lo muestre.

Cualquier tramo que dure más de 30 s, o la pérdida del servo, aborta con fallo. Durante el *homing* se ignoran las órdenes de movimiento manual.

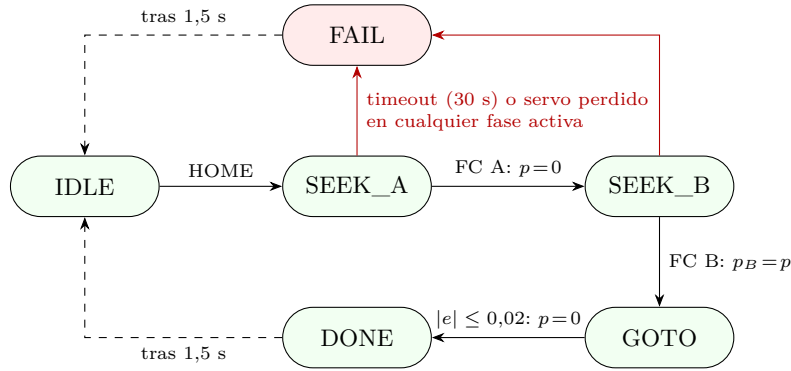


Figura 7.2: Máquina de estados del homing.

7.7. Ciclo de ensayo B→A→B

El ensayo tiene sentido físico empezando por el extremo inferior, con el mecanismo en extensión completa y la masa en su punto más bajo. Como el *homing* deja el eje en A, el ciclo se organiza en tres fases (figura 7.3):

CYC_POS_B — colocación

Baja hasta B. Este tramo no cuenta como repetición y el servidor no lo graba: el ensayo empieza cuando el eje ya está en B. Al llegar se guarda *cycPosB*, la posición integrada en ese punto, que será el origen de medida del ensayo.

CYC_TO_A — ida

Sube hasta A. Es el tramo cargado del ensayo.

CYC_TO_B — vuelta

Baja de nuevo hasta B y cierra la repetición; si quedan repeticiones, se vuelve a CYC_TO_A.

Por convenio, la consigna negativa lleva el eje hacia A (arriba) y la positiva hacia B (abajo).

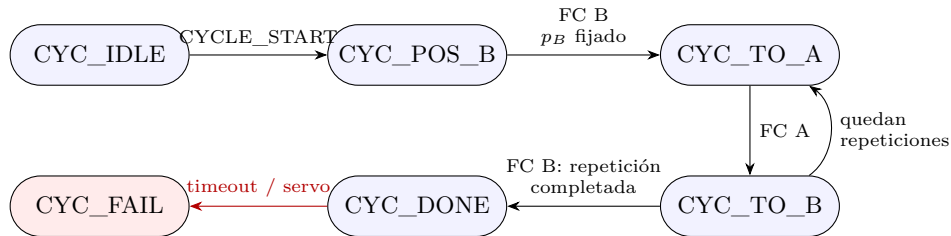


Figura 7.3: Máquina de estados del ciclo de ensayo.

Los extremos se reconocen por los *finales de carrera*, no por la posición integrada. Así el ciclo funciona esté el cero donde esté —el *homing* puede haberlo dejado en A, en B o en medio— y sin depender de que la escala de velocidad del accionamiento sea exacta, que es de donde sale la posición. Si se pide un recorrido parcial, la distancia se mide *desde B*:

```

1 static bool cycleAtB(bool limB){           // B: el final de abajo
2     if(ctrl.cycToLimitB) return limB;
3     return limB || (ctrl.posRev >= ctrl.cycPosB - HOMING_TOL_REV);
4 }
5 static bool cycleAtA(bool limA){           // A: el final de arriba, o la distancia pedida
6     if(ctrl.cycToLimitB) return limA;
7     return limA || ((ctrl.cycPosB - ctrl.posRev) >= (ctrl.cycTargetRev - HOMING_TOL_REV));
8 }
  
```

Código 7.2: Reconocimiento de los extremos del ciclo.

El ciclo sólo se acepta con el servo habilitado, sin *homing* en curso y sin programación de la célula. Los temporizadores y la pérdida del servo lo abortan, y los finales de carrera y el límite de fuerza siguen actuando en el lazo como red de seguridad. La prioridad es: programación de la célula > *homing* > ciclo > movimiento manual.

7.8. Configuración del acondicionador de la célula

La consulta y la programación de la EEPROM del ZSC31014 son las operaciones más delicadas del firmware. Por las restricciones que se analizan en la sección 10.2, obligan a reiniciar el chip y dejan la célula sin medida durante cientos de milisegundos. Se ejecutan así:

1. `processCommand` sólo *anota* la petición y un plazo de 1 s.
2. En su punto fijo del lazo, `lcRequestService` comprueba que el eje está en reposo: estado IDLE o STOPPED, consigna nula y sin *homing*. Si no lo está, responde “rechazado”.
3. Reserva el bus en exclusiva (I2C_CFG). Si no lo consigue antes del plazo, responde “bus ocupado”.
4. Pone la consigna a cero, pasa al estado PROGRAMMING y envía la telemetría pendiente para que la interfaz vea el silencio que viene.
5. Ejecuta la consulta o la programación, devuelve el bus a 400 kHz, libera el árbitro, reinicializa la célula, anula la base de la integración de posición y responde con el paquete 0x04.

7.8.1. Escritura verificada

`loadCellProgram` aplica dos reglas de la hoja de datos [28]: la palabra ZMDI_Config1 sólo se carga tras un ciclo de alimentación, y la firma MISR de la EEPROM (palabra 0x12) se regenera al salir del modo comando con Start_NOM si hubo escrituras. La función hace dos pasadas:

1. **Primera pasada:** entra en modo comando, lee B_Config (0x0F) y ZMDI_Config1 (0x01) y calcula las palabras deseadas. Si coinciden con las actuales, termina sin escribir, porque la EEPROM admite 100.000 ciclos de escritura. Si no, escribe las que cambian, espera 20 ms por escritura (típico 15 ms, peor caso 17,3 ms), envía Start_NOM y espera 150 ms antes de cortar la alimentación.
2. **Segunda pasada:** desde un reinicio limpio, vuelve a leer ambas palabras. Si coinciden con lo pedido, el resultado es “grabado y verificado”; si no, “fallo”.

Las palabras deseadas se construyen a partir de las actuales, protegiendo los bits que la interfaz no debe tocar: en B_Config se conservan los reservados [15:13]; en ZMDI_Config1 se conservan [15:9] (signos de las correcciones de segundo orden y de temperatura, calibrados en fábrica) y se fuerzan el interfaz I²C (bit 4 = 0) y el campo reservado [2:0] = 001.

7.8.2. Mantenimiento del reinicio para medida

La orden LC_HOLD deja la célula sin alimentación durante 8 s, con las líneas del bus en el estado elegido, para medir con un polímetro la tensión de la placa Click. A diferencia de la programación, esta espera *no* bloquea: el lazo sigue girando, leyendo el servo, vigilando los finales de carrera y enviando telemetría, y en cada vuelta sólo comprueba si ha vencido el plazo. Mientras dura se fuerza la consigna a cero en cada vuelta: alguien tiene las manos dentro de la máquina, y una orden de movimiento que llegue por USB no debe poder arrancar el eje.

7.9. Evolución del firmware

Tabla 7.1: Versiones del firmware conservadas en el repositorio.

Versión	Placa	Características
old/speedtorque_mode	XIAO	Escaneo Modbus y selección de modo par o velocidad.
firmware	ESP32-S3	
	XIAO	Protocolo binario v2, parámetros del accionamiento y regulador de fuerza en modo par con zona muerta, umbral y ganancia sobre la célula filtrada.
firmware	ESP32-S3	
	XIAO	
firmwarerasp	Pico 2	Migración con SerialPIO. Doble núcleo: el núcleo 1 escribe la consigna y lee la supervisión Modbus, comunicado con el núcleo 0 por variables <i>volatile</i> .
firmware_posicion	Pico 2	Modo velocidad sin lazo de fuerza, integración de posición, telemetría por lotes y diseño de dueño único en el núcleo 0.
firmware_posicion_v2	Pico 2	Versión final: <i>homing</i> , ciclo B→A→B con extremos por finales de carrera, programación verificada de la EEPROM, árbitro I ² C y diagnóstico en la telemetría.

Capítulo 8

Comunicaciones

El sistema usa cuatro enlaces (tabla 8.1). Este capítulo describe el protocolo binario propio entre el microcontrolador y el ordenador, el uso de Modbus RTU con el accionamiento y la interfaz entre el servidor y el navegador. La especificación byte a byte está en el anexo B.

Tabla 8.1: Enlaces de comunicación del sistema.

Enlace	Extremos	Capa física	Protocolo
USB	PC ↔ Pico	USB CDC (puerto serie virtual)	Binario propio + CRC-8
RS-485	Pico ↔ accionamiento	UART PIO, 115200 8N1	Modbus RTU
I ² C1	Pico ↔ ZSC31014	400 kHz (100 kHz en modo comando)	Propio del chip
Red local	Navegador ↔ servidor	TCP, puerto 8080	HTTP + WebSocket (JSON)

8.1. Protocolo binario entre el microcontrolador y el ordenador

8.1.1. Formato de trama

Todas las tramas, en ambos sentidos, tienen el mismo formato (figura 8.1): dos bytes de sincronía 0xAA 0x55, un byte de longitud, la carga útil y un CRC-8 calculado sobre la carga útil. El primer byte de la carga útil identifica el tipo de paquete o la orden. Los enteros multibyte van en orden *little-endian*.

0xAA	0x55	LEN	ID	datos (LEN-1)	CRC-8
------	------	-----	----	---------------	-------

Figura 8.1: Formato común de trama. El CRC-8 cubre los LEN bytes de la carga útil (ID + datos).

El CRC usa el polinomio $x^8 + x^2 + x + 1$ (0x07), valor inicial 0x00, sin reflexión ni XOR final (la variante conocida como CRC-8/SMBUS). Se implementa bit a bit en el firmware y en Python con el mismo algoritmo. Para tramas cortas como éstas detecta todos los errores de uno y dos bits y todas las ráfagas de hasta 8 bits.

8.1.2. Recepción y resincronización

El firmware decodifica las órdenes con una máquina de cuatro estados (RX_SYNC0, RX_SYNC1, RX_LEN, RX_PAYLOAD). Una longitud nula o mayor de 60 devuelve el decodificador a la búsqueda de sincronía, y una trama con CRC incorrecto se descarta sin ejecutarse. Si se pierden bytes, el receptor vuelve a encontrar el patrón AA 55 en la siguiente trama. El servidor hace lo mismo con la telemetría, con un tiempo de espera de 100 ms por lectura.

Para depurar desde un terminal, el firmware acepta también órdenes de una letra (i, a, b, x, h, q), pero *sólo* cuando el decodificador está esperando el primer byte de sincronía. Antes se comprobaban en cualquier estado, y eso provocaba un error difícil de ver: la orden de programar la célula con ganancia 7 y cero 8 genera la trama AA 55 03 09 07 08 69, cuyo CRC 0x69 coincide con el carácter i. El atajo se comía el último byte, la orden se perdía y además se reiniciaba el servo. Con ganancia 6 el CRC vale 0x7C y la misma orden funcionaba. Ambos valores se han comprobado con la implementación del CRC.

8.1.3. Paquetes del microcontrolador al ordenador

Tabla 8.2: Paquetes de telemetría y estado.

ID	Nombre	Tamaño de carga útil	Cuándo se envía
0x02	Telemetría por lotes	$14 + 14n$ B, $n \leq 8$	cada 200 ms o con 8 muestras
0x03	Estado de <i>homíng</i>	11 B	junto al lote, mientras hay <i>homíng</i>
0x04	Resultado de la célula	13 B	una vez por consulta o programación
0x05	Estado del ciclo	18 B	junto al lote, mientras hay ciclo

Telemetría por lotes. Cada vuelta del lazo añade al búfer una muestra de 14 bytes: desfase en milisegundos respecto al instante base del lote, velocidad, par, consigna, fuerza tarada, estado de los finales de carrera, estado de la máquina y tiempo de trabajo de la vuelta. Los datos comunes (instante base de 32 bits, lectura bruta de la célula y su estado, indicadores, código de error, dirección del servo y códigos de resultado de las dos lecturas Modbus) viajan una sola vez en una cabecera de 14 bytes.

Agrupar las muestras reduce las escrituras por USB a cinco por segundo sin perder resolución temporal, porque cada muestra conserva su instante exacto. Con el lazo de 50 ms cada lote lleva típicamente cuatro muestras: 70 bytes de carga útil, 74 de trama y unos 370 B/s, muy por debajo de la capacidad del enlace. El precio es un retardo de visualización de hasta 200 ms y que la lectura bruta de la célula sólo se refresque una vez por lote (sección 11.9).

Estado del ciclo. El paquete 0x05 creció de 14 a 18 bytes al reorganizar el ciclo: además de la fase, las repeticiones y la posición, transporta *cycPosB*, la posición en la que ha quedado fijado el punto B. Sin ese dato no se puede saber desde fuera contra qué está midiendo el ciclo un recorrido parcial. Las fases son ahora seis: reposo, colocándose en B, hacia A, volviendo a B, completado y abortado.

Para que un firmware y un servidor desaparejados no se queden mudos, el servidor acepta también la trama de 14 bytes de la versión anterior y asume *cycPosB* = 0. Esa compatibilidad se añadió precisamente porque un desajuste de versiones despistó durante la depuración del ciclo: el firmware enviaba una trama que el servidor rechazaba entera, y la interfaz no mostraba ninguna fase.

Modelo pregunta-respuesta de la célula. Cada orden de consulta, programación o mantenimiento de la célula produce exactamente un paquete 0x04, también cuando se rechaza. El paquete incluye el código de la orden a la que responde, para que la interfaz nunca se quede esperando, y lleva información de diagnóstico: el primer byte que devolvió el chip, si el pin de reinicio corta de verdad la alimentación y en qué intento del barrido de retardos se consiguió entrar en modo comando (sección 10.2).

8.1.4. Órdenes del ordenador al microcontrolador

Tabla 8.3: Órdenes aceptadas por el firmware.

Código	Nombre	Datos	Efecto
0x01	INIT	[dirección Modbus] opcional	Reinicia la posición, invalida el recorrido medido y configura y habilita el servo.
0x02	STOP	–	Consigna a cero; aborta <i>homing</i> y ciclo.
0x03	SET_PARAM	id (1 B), valor int16	Modifica un parámetro (tabla 8.4).
0x04	MOVE_A	–	Movimiento manual hacia A (arriba).
0x05	MOVE_B	–	Movimiento manual hacia B (abajo).
0x06	SHUTDOWN	–	Deshabilita el accionamiento.
0x07	SCAN	–	Búsqueda bloqueante del esclavo (1–247).
0x08	HOME	–	Inicia el <i>homing</i> .
0x09	LC_PROGRAM	B_Config, ZMDI_Config1 (uint16)	Graba la configuración de la célula (sólo parado).
0x0A	LC_QUERY	–	Lee la configuración de la célula (sólo parado).
0x0B	LC_HOLD	modo de aparcado (1 B)	Mantiene la célula sin tensión 8 s para medirla.
0x0C	CYCLE_START	modo, recorrido (centi-rev), velocidad, repeticiones	Inicia el ciclo B→A→B, empezando por la colocación en B.
0x0D	CYCLE_STOP	–	Detiene el ciclo donde esté.

Tabla 8.4: Parámetros configurables con SET_PARAM.

Id	Parámetro	Unidad	Defecto
1	Velocidad de movimiento manual	rpm	10
2	Límite de fuerza (0 = desactivado)	cuentas	0
3	Tara de la célula	cuentas	8210
4	Aceleración (C03.22)	u. del accionamiento	5
5	Deceleración (C03.24)	u. del accionamiento	5
6	Rigidez (C00.05)	nivel	10
7	Calibración de fuerza (sólo informativa)	cuentas/N	0
8	Desplazamiento del origen tras el <i>homing</i>	centi-rev	0
9	Velocidad de <i>homing</i>	rpm	30

8.2. Modbus RTU con el accionamiento

8.2.1. Registros utilizados

El firmware sólo usa las funciones 0x03 y 0x06 sobre un conjunto reducido de registros (anexo C). Los parámetros del accionamiento se nombran Cgg.ii, y todas las direcciones cumplen la regla

$$\text{dirección} = 256 \cdot gg + 0xii, \quad (8.1)$$

es decir, el índice se interpreta en hexadecimal. Por ejemplo, C04.11 (habilitación) es $1024 + 17 = 1041$ y C03.22 (aceleración) es $768 + 34 = 802$. Las magnitudes de supervisión están en el grupo 0x40: velocidad en 16385 (0x4001) y par en 16387 (0x4003). Tener esta regla evita errores al añadir registros a partir del manual.

8.2.2. Secuencia de configuración

Al recibir INIT, el firmware configura el accionamiento así: deshabilitar ($C04.11 = 0$), seleccionar el modo velocidad ($C00.00 = 1$), poner la consigna a cero ($C03.21 = 0$), escribir la rigidez y las rampas, configurar la resistencia de frenado externa ($C00.10$ a $C00.13$) y habilitar ($C04.11 = 1$). Si una escritura esencial falla se reintenta la secuencia completa, y tras tres fallos el sistema pasa a estado de error. Entre escrituras se deja un silencio de $700\text{ }\mu\text{s}$.

8.2.3. Transacciones por periodo

En régimen normal, cada vuelta del lazo realiza tres transacciones: lectura de la velocidad, lectura del par y escritura de la consigna. Escribir la consigna en cada vuelta, aunque no cambie, sirve además de supervisión del enlace: si la escritura falla, se marca el servo como desconectado.

A 115200 baudios con formato $8N1$, cada byte ocupa $86,8\text{ }\mu\text{s}$. Las tres transacciones suman 46 bytes, unos $4,0\text{ ms}$ de línea. A eso se añaden $0,6\text{ ms}$ de retardos de conmutación del transceptor y $0,7\text{ ms}$ de silencio entre las dos lecturas. El tiempo de trabajo medido del lazo es $10,97\text{ ms}$ (sección 11.2); la diferencia, unos $5,7\text{ ms}$, corresponde principalmente al tiempo de respuesta del accionamiento, aproximadamente $1,9\text{ ms}$ por transacción.

La guía de implementación de Modbus serie recomienda, por encima de 19200 baudios, silencios fijos de $1,75\text{ ms}$ entre tramas [19]. El firmware usa $700\text{ }\mu\text{s}$ y el accionamiento lo acepta sin errores, pero con otros esclavos convendría respetar el valor recomendado.

8.3. Interfaz entre el servidor y el navegador

8.3.1. WebSocket

El servidor mantiene una conexión WebSocket en `/ws` con cada navegador. En sentido servidor → navegador envía objetos JSON de cinco tipos:

- **muestra de telemetría** (sin campo `type`): una por muestra del lote, con los valores brutos, los nombres de estado y error ya traducidos, la posición integrada, los finales de carrera, la frecuencia de muestreo medida y el estado del *homing*;
- **home**: fase del *homing*, posición y recorrido medido;
- **cycle**: fase, repeticiones hechas y pedidas, posición, recorrido pedido, posición de B y fichero en grabación;
- **lc_config**: resultado de la consulta o programación de la célula;
- **cmd_ack**: confirmación de *toda* orden recibida, con indicación de éxito y un texto explicativo. Sin ella, una orden que no llegaba al puerto era indistinguible de una que sí se había ejecutado.

En sentido navegador → servidor, las órdenes son objetos `{cmd": nombre, ...parámetros}`. El servidor las valida, limita los parámetros a rangos seguros (por ejemplo, velocidad del ciclo entre 0 y 300 rpm), construye la trama binaria y la escribe en el puerto.

8.3.2. Servicios HTTP

Tabla 8.5: Servicios HTTP del servidor.

Método	Ruta	Función
GET	/	Monitor en tiempo real
GET	/ensayos	Visor de ensayos grabados
GET	/sim	Configuración del mecanismo y simulador
GET	/api/mecanismo	Geometría actual (JSON)
PUT	/api/mecanismo	Guarda la geometría, validada contra rangos
GET	/cycles	Lista de los 50 ensayos más recientes (JSON)
GET	/cycles/{nombre}	Descarga de un ensayo (CSV)
DELETE	/cycles/{nombre}	Borrado de un ensayo (rechazado si se está grabando)
GET	/mecanismo.js, /tema.css, /tema.js, /nav.js	Recursos comunes a las tres páginas
/fui (montaje estático)		Kit de interfaz (hoja de estilo y tipografías)

Las rutas con nombre de fichero reducen el parámetro a su último componente antes de usarlo, de modo que una petición con `../` no puede leer ni borrar ficheros fuera de la carpeta de ensayos. La escritura de la geometría se valida en el servidor contra los mismos rangos que comprueba el navegador, de forma que una petición construida a mano tampoco puede dejar una configuración imposible.

Capítulo 9

Software de supervisión y análisis

9.1. Visión general

El software del ordenador consta de un servidor en Python (`web_monitor.py`), tres páginas web y varias herramientas auxiliares. El servidor se ejecuta con

```
python web_monitor.py [puerto_serie] # http://localhost:8080
```

y detecta solo el puerto del microcontrolador si no se indica. Las tres páginas son el *monitor* en tiempo real (`/`), el *visor de ensayos* (`/ensayos`) y la *configuración del mecanismo* con simulador (`/sim`), enlazadas entre sí por una barra de teclas programables común.

9.2. Servidor

9.2.1. Estructura concurrente

El servidor combina un hilo del sistema operativo para el puerto serie con el bucle asíncrono de FastAPI (figura 9.1).

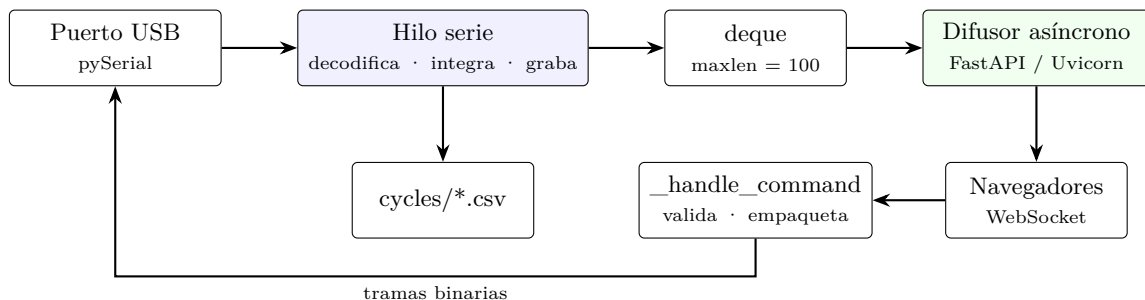


Figura 9.1: Estructura del servidor de supervisión.

Hilo serie. Abre el puerto (probando `/dev/ttyACM*`, `/dev/ttyUSB*`, `/dev/cu.usbmodem*` y `COM*`), vacía el búfer y entra en un bucle que lee tramas, comprueba el CRC y las decodifica con `struct`. Cualquier excepción cierra el puerto y provoca un reintento al cabo de un segundo, lo que cubre la desconexión física y el reinicio del microcontrolador al reprogramarlo (RNF4). Por cada muestra de telemetría:

- mide la frecuencia de muestreo contando las muestras del último segundo *con el reloj del dispositivo*, no con el del ordenador;
- detecta que el reloj del microcontrolador ha vuelto a cero (reinicio) y reinicia entonces la ventana de medida y la integración;
- integra la posición a partir de la velocidad, descartando saltos de tiempo anómalos;

- si hay un ensayo en grabación, escribe la fila correspondiente;
- deposita un mensaje JSON en la cola.

Cola y difusor. La comunicación entre el hilo y el bucle asíncrono usa una **deque** de la biblioteca estándar con un único productor y un único consumidor, segura sin cerrojos en CPython para **append** y **popleft**. La longitud máxima de 100 mensajes acota la memoria si no hay ningún navegador conectado. El difusor envía cada mensaje a todos los clientes, elimina los que han cerrado la conexión y, cuando la cola está vacía, duerme 20 ms.

Registro. Todas las tramas, órdenes y confirmaciones se registran con marca de tiempo en consola y en **web_monitor.log**. Al depurar el ciclo se reforzó ese registro, porque era insuficiente: ahora cada línea de telemetría incluye la posición, los dos finales de carrera y la fase del ciclo, y cada cambio de fase escribe una línea propia. Sin eso, un ciclo que arrancaba y terminaba en 200 ms no dejaba rastro de por dónde había pasado.

9.2.2. Configuración del mecanismo

La geometría vive en **mecanismo.json**, con dos servicios: **GET /api/mecanismo** devuelve la configuración y **PUT /api/mecanismo** la guarda. El servidor valida cada clave contra un rango (por ejemplo, α_0 entre 0° y 180° , o radios estrictamente positivos) y rechaza lo que no encaje, de modo que ni una petición construida a mano puede dejar una geometría imposible. Los ficheros guardados con la convención anterior, que medían el ángulo desde la horizontal, se convierten al leerlos mediante $\alpha = 90^\circ - \theta_0$.

9.2.3. Grabación de ensayos

La grabación es automática y la dirigen los paquetes de ciclo: cuando la fase pasa a “hacia A” o “volviendo a B” y no hay fichero abierto, se crea **cycles/AAAAMDD-HHMMSS_ciclo.csv**; cuando la fase pasa a completado, abortado o reposo, se cierra. La fase de colocación en B no se graba, porque el ensayo empieza cuando el eje ya está en B. Cada fichero lleva una cabecera con las constantes del ensayo y una fila por muestra (formato en el anexo D).

Se graban deliberadamente las *cuentas brutas* de la célula, no newtons: la calibración puede cambiar después del ensayo —de hecho ha cambiado varias veces— y con los datos brutos basta con aplicar la nueva constante (RNF5).

9.3. Modelo compartido

Las fórmulas del capítulo 4 están implementadas una sola vez, en **static/mecanismo.js**, y las usan por igual el simulador y el visor de ensayos. El módulo ofrece la geometría del tambor (**tambor**), la longitud de cable (**cable**), el ángulo (**barraAng**), el estado completo del mecanismo para un avance dado (**estado**) y la inversa, el avance necesario para llevar la barra a un ángulo (**revParaAngulo**), resuelta por bisección.

Esta unificación corrigió un problema real: cada página llevaba su copia de las fórmulas y habían divergido, hasta el punto de que el monitor seguía calculando con un cilindro y la barra horizontal mientras el visor ya usaba la caracola. Los cambios se propagan al instante entre pestañas del mismo navegador mediante **BroadcastChannel**, y al volver a una pestaña se relea la configuración por si se cambió desde otro equipo.

9.4. Monitor en tiempo real

La página principal (**index.html**) se organiza en tres zonas:

1. **Estado:** indicador de conexión, estado de la máquina y código de error, modo, estado del servo y de la célula, tiempo del dispositivo y frecuencia de muestreo, finales de carrera (A

arriba, B abajo) con la fase del *homing* y el recorrido medido, y una tarjeta destacada con la fuerza.

2. **Controles en pestañas:** *Servo* (dirección Modbus, inicialización, movimiento manual, *homing*, parada, puesta a cero y apagado; velocidades y desplazamiento del origen); *Célula 1* (calibración, tara y, en configuración avanzada, las palabras B_Config y ZMDI_Config1 con el botón de programar); *Ciclo* $B \rightarrow A \rightarrow B$ (hasta dónde sube, velocidad y repeticiones); y *Ensayos grabados*.
3. **Gráficas:** velocidad de consigna y real con los finales de carrera y marcas verticales en los instantes en que se enviaron órdenes, y fuerza frente al tiempo.

9.5. Configuración del mecanismo y simulador

La página `/sim` (figura 9.2) reúne la edición de la geometría y un simulador:

- **Formulario** con el tipo de tambor (caracola o cilindro), radios y barrido, reducción, sentido, cotas L_2 , dx , L_3 , L_4 , masa y α_0 . Los cambios no se aplican hasta pulsar *Guardar y propagar*, y se pueden descartar.
- **Comprobaciones** automáticas: distancia e inclinación de la línea $O \rightarrow D$, ángulo y cable en B, ángulo en el extremo del recorrido, recorrido angular total y avisos cuando la geometría pide a la barra pasar de 0° o de 180° , que es donde la tensión se hace singular. También calcula la inversa: cuántas revoluciones de motor hacen falta para llevar la barra a un ángulo dado, que es el recorrido $B \rightarrow A$ que tendría que hacer el ciclo.
- **Dibujo a escala** del mecanismo en un *canvas*, con un cursor que recorre el ciclo y un botón de reproducción, y la posibilidad de cargar un ensayo grabado para que el cursor siga sus datos.
- **Gráficas** de tensión, par y ángulo frente al avance del motor.

[COMPLETAR: captura de pantalla de la página `/sim` con el mecanismo dibujado, para incluirla como figura 9.2]

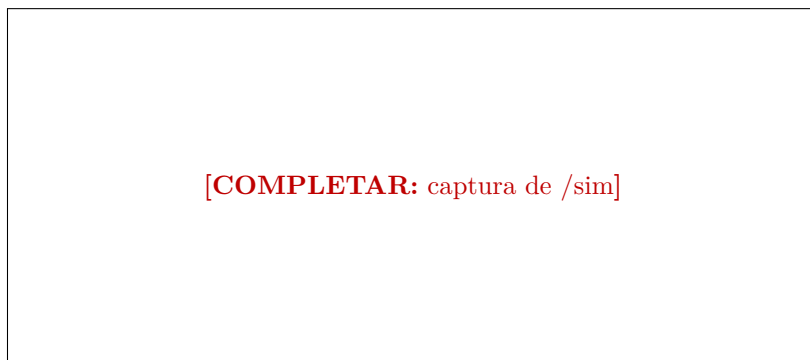


Figura 9.2: Página de configuración del mecanismo con el simulador.

9.6. Visor de ensayos

El visor (`ensayos.html`) lista los ensayos grabados y, al abrir uno, muestra un resumen (muestras, duración, cadencia, repeticiones y rangos), la fuerza frente a la posición con una traza por repetición, la fuerza y la posición frente al tiempo, el par frente a la posición y al tiempo, y la velocidad con los finales de carrera.

Sobre la gráfica de fuerza puede superponerse la curva teórica, calculada con la geometría configurada. El avance del ciclo se toma del propio ensayo: se resta la posición de la primera

muestra —que es B, donde arranca— y el sentido se deduce del signo de la muestra más alejada, de modo que los ensayos antiguos, que iban de A hacia B, siguen siendo interpretables.

El visor comprueba además dos consistencias que han resultado muy útiles:

1. *Escala*: compara el cable que paga el tambor durante el ensayo con el que el mecanismo puede absorber en ese sentido; si se pide más del que cabe, avisa y sugiere la reducción o los radios que lo harían cuadrar.
2. *Ángulo*: calcula el ángulo de la barra en B, en el extremo y al final del ensayo, y avisa si el modelo exigiría pasar de los topes o si el cierre del ciclo no cuadra.

Por último, ofrece un ajuste por mínimos cuadrados de la recta que convierte cuentas en newtons, $T \approx a(\text{cuentas}) + b$, con su R^2 y su cobertura. Conviene leerlo junto con la sección 4.5.1: un R^2 alto no valida la geometría, porque el mismo valor se obtiene con la sensibilidad de signo contrario.

9.7. Herramientas auxiliares

- `leer_ciclo.py`: resumen de un ensayo en consola, sólo con la biblioteca estándar.
- `monitor.py` y `monitor_posicion.py`: monitores de terminal de versiones anteriores del protocolo.
- `test_servo_modbus.py`: barrido de velocidades y direcciones Modbus con un adaptador USB-RS485, útil para identificar la configuración del accionamiento sin el microcontrolador.
- `memoria/figuras/generar_figuras.py` y `generar_figuras21.py`: generan las figuras y las cifras de esta memoria a partir de los ficheros de ensayo, junto con `modelo_mecanismo.py`, que es el modelo del capítulo 4 portado a Python.

9.8. Interfaz de usuario

Las tres páginas comparten el aspecto de un panel de control industrial, con un kit de interfaz externo (hoja de estilo y tipografías) publicado en `/fui` y copiado sin modificar, y los ajustes propios en `static/tema.css`. Los colores de las gráficas se leen de las variables de esa hoja de estilo (`tema.js`), de modo que hay un único sitio donde cambiarlos, y la navegación entre páginas se genera desde `nav.js`.

9.9. Observaciones de calidad

Durante la redacción de esta memoria se revisó el software y se identificaron estos puntos de mejora, que no afectan a los resultados:

- `requirements.txt` sólo declara `pyserial`; faltan `fastapi` y `uvicorn[standard]`.
- La cabecera del CSV decide si el punto de parada es “posición” o “final de carrera” según el recorrido pedido sea mayor que cero, no según el modo realmente usado. Como la interfaz envía siempre un recorrido, todos los ensayos aparecen como “posición” aunque se hicieron contra los finales de carrera. La corrección es incluir el modo en el paquete de ciclo.
- El servidor escucha en todas las interfaces (0.0.0.0) sin autenticación, así que cualquier equipo de la red puede mover la máquina.
- La posición se integra dos veces, en el firmware y en el servidor, con el mismo método; el CSV guarda la del servidor. Sería más limpio transmitir la del firmware en cada muestra.
- La cabecera del firmware todavía describe la versión de doble núcleo.

Capítulo 10

Problemas técnicos y soluciones

Este capítulo reúne los problemas más costosos del desarrollo. Para cada uno se describen el síntoma, la causa y la solución, tal como quedaron documentados en el código y en el repositorio. Muchos no se deben a un error de programación, sino a detalles de los componentes o del montaje que no son evidentes en una primera lectura.

10.1. UART sobre PIO y latencia del enlace Modbus

Síntoma. Tras migrar el firmware del ESP32-S3 a la Pico 2, las órdenes tardaban entre 3 y 4 s en tener efecto.

Causa. En el ESP32-S3 la UART es un periférico hardware: los bytes de la respuesta llegan a una FIFO por interrupción y, cuando ModbusMaster los pide, ya están disponibles, así que cada transacción dura 1 a 3 ms. En la Pico, el trazado lleva la transmisión a GP14 y la recepción a GP13, y se usó `SerialPIO`, una UART implementada con una máquina de estado PIO que funciona en cualquier pin [\[VERIFICAR: confirmar si el RP2350 permite la función UART en GP14 mediante su función alternativa\]](#). Con `SerialPIO`, el bucle de espera de ModbusMaster agotaba su tiempo límite, de unos 20 ms, en muchas transacciones: diez escrituras consecutivas bloqueaban el núcleo 200 ms, y con las lecturas de supervisión se llegaba a varios segundos.

Un segundo detalle afectaba al cambio de sentido del transceptor: llamar a `flush()` antes de desactivar el emisor, como se haría con una UART hardware, rompía la recepción inmediata de la respuesta. La solución fue esperar un tiempo fijo de 200 μ s tras el último byte.

Primera solución: doble núcleo. La versión `firmwarerasp` dedicó el núcleo 1 al bus Modbus, con la consigna publicada en variables `volatile` y un protocolo de relevo para que el núcleo 0 pudiera tomar el bus. Funcionaba, pero con variables compartidas sin atomicidad, esperas de relevo y lecturas cuyo instante no estaba ligado al de la muestra de telemetría.

Solución final: dueño único con lazo de periodo fijo. Ajustados los tiempos de espera y el cambio de sentido, cada transacción volvió a durar pocos milisegundos y se pudo prescindir del segundo núcleo: un único lazo de 50 ms con exactamente tres transacciones por vuelta. El resultado es un tiempo de trabajo de 11,0 ms con 0,2 ms de dispersión (sección 11.2), sin condiciones de carrera y con lecturas, lógica y escritura en la misma vuelta.

10.2. Configuración del ZSC31014: la ventana del modo comando

Contexto. Para cambiar la ganancia o el cero del acondicionador hay que escribir en su EEPROM, y para ello entrar en modo comando. Según la hoja de datos [28], tras un reinicio por

encendido (POR) el chip espera la orden `Start_CM` durante 6 ms si la EEPROM no está bloqueada, o 1,5 ms si lo está. Pasada la ventana, ignora `Start_CM` sin dar ninguna señal de error.

10.2.1. Primer obstáculo: provocar el reinicio

Tras reprogramar la Pico, la configuración nunca se aplicaba, porque la placa Click seguía alimentada y sin POR no se abre la ventana. La solución fue controlar su alimentación con el pin EN (GPIO1) y hacer un ciclo completo de apagado y encendido antes de cada intento.

10.2.2. Segundo obstáculo: alimentación parásita por el bus

Síntoma. Con EN a nivel bajo, el chip seguía respondiendo por I²C, y la tensión de la Click se quedaba en 2,5 V a 2,7 V en lugar de caer a cero.

Causa. Las resistencias de *pull-up* de SDA, SCL e INT cuelgan del 3,3 V permanente del zócalo mikroBUS, no del carril gobernado por EN. Con el chip sin alimentar, la corriente entraba por esos pines, atravesaba los diodos de protección internos y alimentaba el carril. La hoja de datos sitúa el umbral de POR entre 1,8 y 2,5 V: con 2,5 V a 2,7 V parásitos, el chip nunca veía un reinicio.

Diagnóstico y solución. Se añadió la orden `LC_HOLD`, que mantiene la célula sin alimentación 8 s sin bloquear el lazo, con dos modos para las líneas del bus. Las medidas con polímetro mostraron que sólo forzándolas a nivel bajo la tensión cae a cero, y así se dejó por defecto. Hubo además un error en el propio diagnóstico: la primera versión de la prueba dejaba las líneas en alto mientras medía, de modo que siempre “detectaba” realimentación y acusaba al pin EN sin motivo.

10.2.3. Tercer obstáculo: acertar la ventana

La orden tiene que llegar después de que el chip arranque y antes de que la ventana se cierre, y ese instante depende de los condensadores de la placa Click, así que los retardos copiados de otros controladores no servían. En lugar de adivinarlo, el firmware lo mide: sondea la dirección hasta que el chip responde, y a partir de ahí prueba retardos de 0 a 1050 µs en pasos de 150 µs a lo largo de hasta ocho ciclos de alimentación, confirmando la entrada con la lectura de `B_Config`. El intento que funciona se devuelve a la interfaz para poder fijarlo más adelante.

10.2.4. Cuarto obstáculo: integridad de la EEPROM

La firma MISR que protege la EEPROM se recalcula al salir del modo comando con `Start_NOM` y se comprueba en el siguiente POR: cortar la alimentación antes deja la firma a medias y el chip arranca desde entonces en estado de diagnóstico. Por eso, tras cualquier escritura se envía `Start_NOM` y se espera diez veces el tiempo típico antes de cortar. Como `ZMDI_Config1` sólo se carga tras un POR, la verificación se hace en una segunda pasada desde un reinicio limpio.

10.3. Lecturas de la célula siempre “antiguas”

En una versión del firmware no se registraba ni una sola muestra válida. La lectura se hacía en dos fases separadas 2 ms, quedándose con la segunda; como en modo de conversión continua leer no dispara una conversión y el chip no había producido un dato nuevo en ese tiempo, la segunda lectura venía siempre marcada como repetida. Una versión anterior funcionaba por casualidad, porque sus dos lecturas caían en vueltas distintas del lazo. La solución fue una única lectura por vuelta, aceptada sólo si el dato es nuevo.

10.4. Depuración del ciclo de ensayo

Al reorganizar el ciclo para que empezara en el extremo inferior (sección 7.7) aparecieron tres problemas encadenados:

El ciclo terminaba sin hacer nada

Con el eje ya en el final de carrera, la condición de llegada se cumplía de inmediato y el ciclo recorría todas sus fases en un par de periodos. No era un fallo de lógica, sino de arranque: faltaba la fase de colocación en B.

No quedaba rastro en el registro

Un ciclo que empieza y acaba en 200 ms no aparecía en el registro, porque sólo se escribía una línea por lote de telemetría y ninguna al cambiar de fase. Ahora cada cambio de fase escribe su propia línea, y la línea de telemetría incluye posición, finales de carrera y fase.

La interfaz no mostraba la fase

El firmware había pasado a enviar una trama de ciclo de 18 bytes y el servidor, que esperaba 14, la descartaba entera. Se añadió la compatibilidad con la trama antigua para que un desajuste de versiones degrade la información en lugar de hacerla desaparecer.

Además, el ciclo pasó a reconocer los extremos por los finales de carrera en vez de por la posición integrada. Con la posición, cualquier error de escala del accionamiento o cualquier cero mal colocado desplazaba el recorrido; con los finales, el ensayo recorre siempre lo mismo.

10.5. El modelo que no cerraba: la salida del cable desplazada

Síntoma. Con el primer modelo, que situaba la salida del cable justo encima de la articulación, la curva teórica no encajaba con los ensayos: el recorrido angular que predecía no era el que hacía la barra, y el ajuste sólo cuadraba forzando valores poco creíbles de la reducción.

Causa. La salida del cable no está sobre la vertical de la articulación, sino desplazada horizontalmente. Ese desplazamiento no es un detalle de dibujo: con la salida en la vertical, el coseno del ángulo de la barra se cancela con el seno del ángulo del cable y la tensión resulta proporcional a la longitud de cable, que es el resultado elegante de la ecuación (4.5). Con la salida desplazada esa cancelación desaparece (sección 4.3).

Solución. Se reescribió el modelo con el desplazamiento dx como parámetro y con el ángulo medido entre la línea que une articulación y salida y la propia barra, que además elimina la ambigüedad que tenía medir desde la horizontal. El modelo pasó a estar implementado una sola vez y a alimentarse de una configuración guardada en el servidor.

10.6. Lo que la fuerza no puede decidir

Al ajustar los parámetros del mecanismo contra la señal de la célula se obtuvieron coeficientes de determinación altos ($R^2 \approx 0,93$ en los ensayos limpios), lo que parecía confirmar la geometría. No lo confirmaba. Como la célula no está calibrada, el ajuste incluye la sensibilidad como incógnita, y R^2 es el cuadrado de la correlación: no distingue una sensibilidad positiva de una negativa. Dos geometrías que predicen tensiones con tendencias opuestas dan exactamente el mismo R^2 (sección 4.5.1).

La consecuencia práctica llegó al añadir el par del accionamiento al contraste: con los mismos parámetros, el par medido y el del modelo resultaron *anticorrelados* (capítulo 11). El análisis se reorganizó entonces alrededor del par, que no depende de ninguna calibración pendiente, y la señal de la célula quedó como comprobación secundaria.

10.7. Tramo inicial sin carga y calibraciones incoherentes

Los ensayos de la segunda campaña muestran que la primera media revolución de motor transcurre sin tensión en el cable y con un par casi nulo: el mecanismo recoge holgura antes de empezar

a levantar la masa. No es un fallo, pero invalida ese tramo para el análisis y obliga a definir explícitamente el recorrido con carga.

Por otra parte, la constante que relaciona cuentas y newtons ha resultado distinta en cada intento: 14,41 cuentas/N en la configuración de la interfaz, unas 20 al ajustar contra el modelo y 27 a 28 en los ensayos limpios; y en la campaña del 15 de septiembre el ajuste salía incluso con el signo contrario. Todo ello confirma que la célula necesita una calibración con masas patrón antes de dar valores absolutos de fuerza.

10.8. Segunda célula de carga en el mismo bus

Se evaluó añadir una segunda célula con una Load Cell 4 Click (NAU7802) en el mismo bus. Surgieron dos problemas. El primero, de convivencia: el ZSC31014 no toleraba transacciones ajenas entre la petición y la recogida del dato, y de ahí nació el árbitro de `i2c_bus.h`. El segundo, de ruido: con ganancia 128 y en reposo se midieron unas 18.000 cuentas pico a pico con el regulador interno a 4,5 V, unas 3.500 bajándolo a 3,0 V con calibración interna de cero, y unas 350 al activar el *chopper*, equivalentes a 0,49 μ V en la entrada. La segunda célula se retiró del firmware final.

10.9. Discrepancias con la hoja de datos

Al contrastar el firmware con la hoja de datos del ZSC31014 se encontraron dos discrepancias que no afectan a los ensayos, pero conviene corregir:

1. **Bits de estado.** Según la hoja de datos, 00 = dato válido, 01 = modo comando, 10 = dato repetido y 11 = diagnóstico. El firmware define `LC_STATUS_STALE = 1` y `LC_STATUS_COMMAND = 2`, intercambiados. En la lectura normal sólo se acepta 00, así que no hay efecto; en la confirmación del modo comando la respuesta real (0x5A) se acepta por la segunda condición del código.
2. **Codificación de la ganancia.** El campo `PreAmp_Gain` no es lineal: 000 = $\times 1,5$, 100 = $\times 3$, 001 = $\times 6$, 101 = $\times 12$, 010 = $\times 24$, 110 = $\times 48$, 011 = $\times 96$ y 111 = $\times 192$, es decir, los tres bits en orden inverso. El selector de la interfaz supone un código lineal, de modo que para los códigos 1 a 6 la ganancia grabada no es la mostrada. Los códigos 0 y 7 coinciden. **[VERIFICAR: comprobar qué código de ganancia estaba grabado durante los ensayos]**

10.10. Otros problemas

Órdenes perdidas por los atajos de teclado

Un byte de CRC igual a una letra de atajo hacía que se perdiera la orden y se reiniciara el servo. Se resolvió aceptando los atajos sólo con el decodificador en reposo.

Frecuencia de muestreo imposible

La interfaz llegó a mostrar 3813 Hz: tras un reinicio del microcontrolador su reloj volvía a cero y la ventana de medida no se podaba nunca. Se detecta el salto de reloj y se reinicia la ventana.

Desplazamientos fantasma

Tras la programación de la célula, que bloquea el lazo cerca de un segundo, la integración multiplicaba ese hueco por la última velocidad. Se anula la base de tiempo tras cada operación bloqueante.

Órdenes que “no hacían nada”

Una acción mal escrita en la interfaz se ignoraba sin avisar. Se añadió una confirmación para toda orden, con el motivo del fallo cuando lo hay.

Revisión A de la placa

Conflictos de red en el bloque RS-485, entradas analógicas en pines sin ADC y amplificadores sin alimentar (tabla 6.1).

10.11. Lecciones aprendidas

- **Leer la hoja de datos entera:** la ventana del modo comando, la firma de la EEPROM, el umbral de POR y la codificación no lineal de la ganancia están en secciones distintas y todas han resultado críticas.
- **Medir antes de suponer:** la alimentación parásita sólo se descubrió con un polímetro, y el retardo de la ventana, con un barrido.
- **Un ajuste bueno no es una validación:** un R^2 alto puede esconder una ambigüedad de signo. Hace falta una segunda magnitud, medida de forma independiente, para cerrar el contraste.
- **Cada operación debe responder:** las confirmaciones de órdenes y el registro por cambio de fase convirtieron fallos silenciosos en diagnósticos claros.
- **Conservar los datos brutos:** ha permitido recalibrar y reanalizar todos los ensayos sin repetirlos, incluso después de cambiar el modelo por completo.

Capítulo 11

Resultados y discusión

Este capítulo presenta primero la caracterización del banco (temporización y repetibilidad) y después el contraste del modelo con las medidas. El contraste se organiza alrededor de una pregunta concreta: *¿se comporta el tambor como un cilindro o como una caracola, y en qué medida?* Todas las figuras y cifras se generan con los programas de **memoria/figuras/** a partir de los ficheros de **cycles/**, sin edición manual.

11.1. Campañas de ensayos

Se han realizado dos campañas, separadas por el rediseño del ciclo y del modelo.

Campaña preliminar (14 y 15 de septiembre)

Quince ficheros con el ciclo anterior, que arrancaba en el origen y bajaba hasta el final de carrera ($A \rightarrow B \rightarrow A$), y con el modelo que situaba la salida del cable en la vertical de la articulación. Catorce ensayos válidos, 2744 muestras, doce de ellos a 30 rpm.

Campaña del 21 de septiembre

Quince ficheros con el ciclo reorganizado ($B \rightarrow A \rightarrow B$, sección 7.7) y la calibración provisional ya escrita en la cabecera. De ellos, seis son ciclos completos y **tres** son ensayos limpios y mutuamente consistentes (18:06:37, 18:07:09 y 18:08:46), que son los que se usan para el análisis cuantitativo. El resto quedaron interrumpidos o con el cable mal asentado, y se descartan de forma explícita en la tabla 11.1.

Tabla 11.1: Ensayos de la campaña del 21 de septiembre con ciclo completo.

Ensayo	Muestras	$p_{\text{máx}}$ (rev)	R^2 fuerza	Observaciones	Uso
18:06:37	180	1,789	0,932	ciclo limpio	sí
18:07:09	180	1,806	0,931	ciclo limpio	sí
18:08:46	180	1,810	0,930	ciclo limpio	sí
18:16:41	172	1,847	0,187	la señal de la célula apenas varía	no
18:21:20	176	1,867	0,165	ídem, con par de bajada muy bajo	no
18:34:24	180	1,864	0,956	rango de fuerza distinto: tara cambiada	no

11.2. Temporización del lazo de control

La figura 11.1 muestra la distribución del periodo entre muestras y del tiempo de trabajo del lazo, medidos por el propio firmware. En la campaña del 21 de septiembre:

- el periodo medio es 50,004ms con una desviación típica de 0,059ms, y el 99,7% de los periodos son exactamente de 50 ms (el resto, de 51 ms, el grano de `millis()`). La frecuencia de muestreo es de 20,0Hz y cumple RNF1 con holgura;

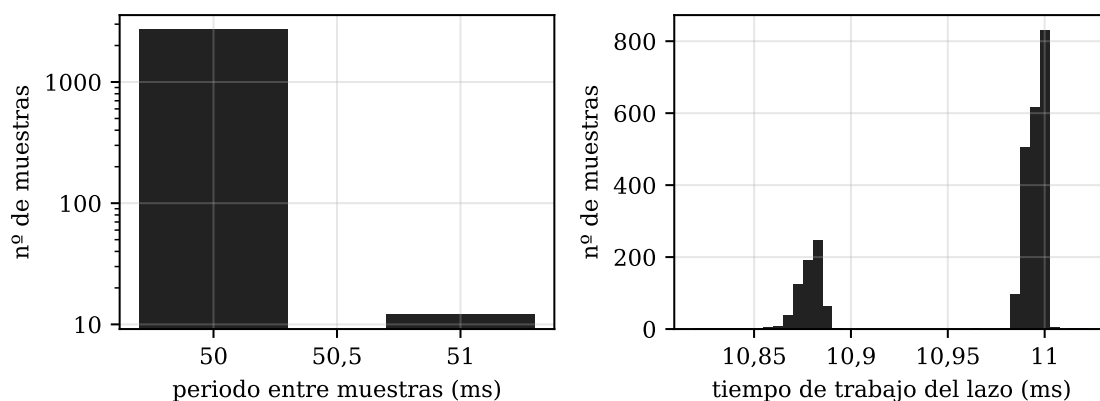


Figura 11.1: Izquierda: periodo entre muestras (escala logarítmica). Derecha: tiempo de trabajo por vuelta del lazo.

- el tiempo de trabajo está entre 10,84 y 11,00 ms, con media de 10,97 ms: el 22 % del periodo, con 39 ms de margen. Su dispersión, de apenas 0,2 ms, confirma que el lazo está dominado por las tres transacciones Modbus.

Las cifras coinciden con las de la campaña preliminar, lo que indica que la temporización no depende del ensayo.

11.3. Ensayo tipo

La figura 11.2 muestra el ensayo de las 18:08:46. La secuencia es:

1. El eje arranca en B y sube hacia A. Durante la primera media revolución (zona sombreada) la señal de la célula permanece plana y el par del motor es de unas 3 u.a.: el mecanismo está recogiendo holgura y todavía no levanta la masa.
2. A partir de 0,56 rev de avance el par crece con rapidez y la señal de la célula empieza a moverse: el cable ya trabaja. Desde ahí hasta A el par sigue creciendo.
3. Al activarse el final A el ciclo invierte la consigna y el eje baja de vuelta a B, con un par menor que en la subida por efecto de las pérdidas.
4. La repetición se cierra al volver a tocar B. La duración total es de 8,9 s.

11.4. Repetibilidad

Los tres ensayos limpios alcanzan el final de carrera A tras 1,789, 1,806 y 1,810 rev de avance, con una desviación típica de 11 mrev, y vuelven a B con un avance final de -14, 5 y 1 mrev. Es decir, el recorrido entre los dos finales de carrera se repite dentro del 0,6 %, y el ciclo cierra prácticamente sobre el punto de partida. El tramo inicial sin carga también es repetible: el par supera las 10 u.a. en 0,55, 0,56 y 0,58 rev.

La dispersión combina la repetibilidad de los propios finales de carrera con la deriva de la posición integrada a lo largo de casi dos revoluciones, de modo que es una cota superior de ambas.

11.5. Tensión y par a lo largo del recorrido

La figura 11.3 superpone las muestras de los tres ensayos limpios en las que el eje se mueve de forma estable. Se aprecian tres hechos: los ensayos son muy repetibles, la señal de la célula recorre el mismo camino en subida y bajada (una vez descontado el retardo de su filtro), y el par

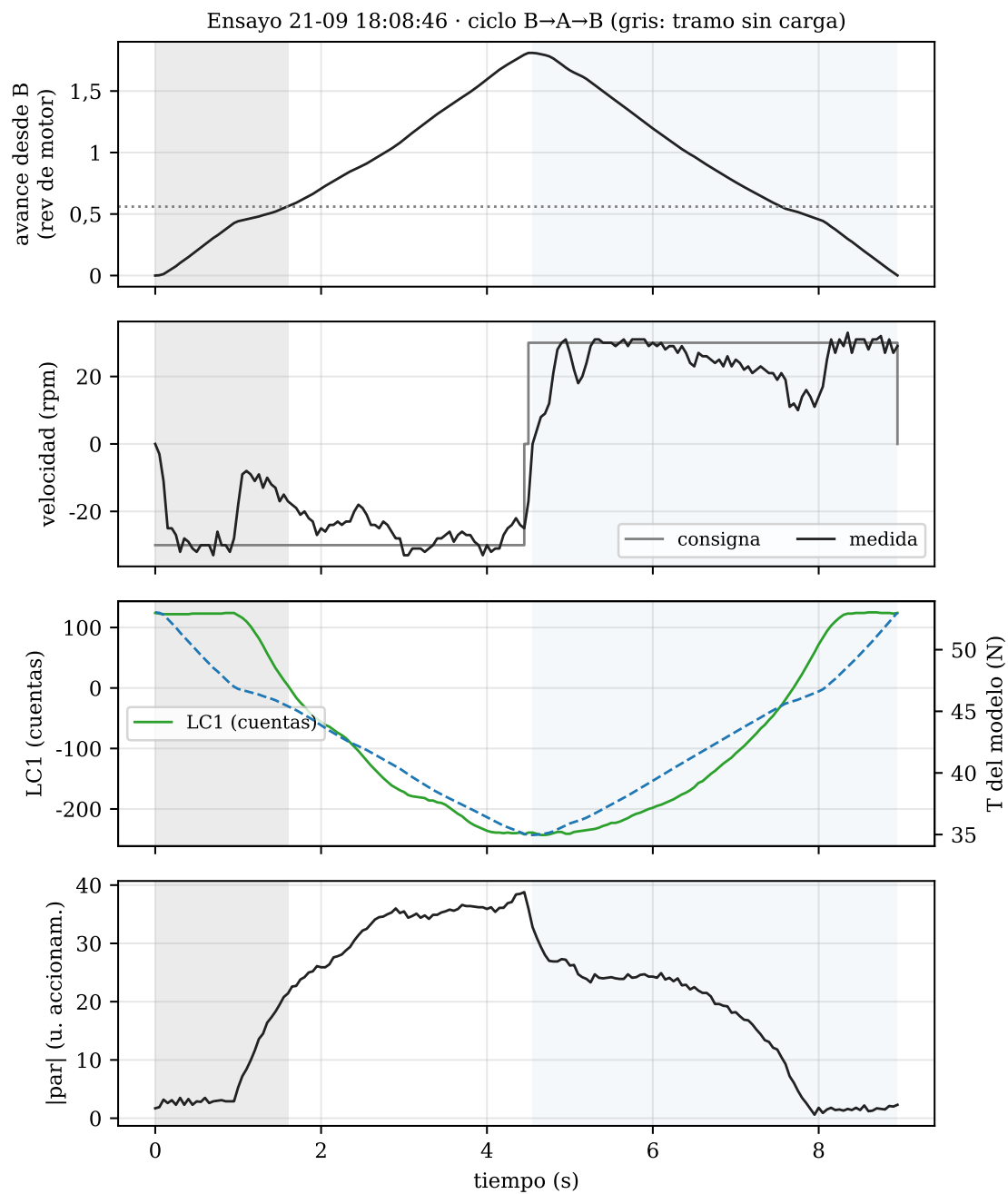


Figura 11.2: Ensayo tipo de la campaña del 21 de septiembre: avance, velocidad, señal de la célula con la tensión del modelo superpuesta, y par del motor. En gris, el tramo inicial sin carga.

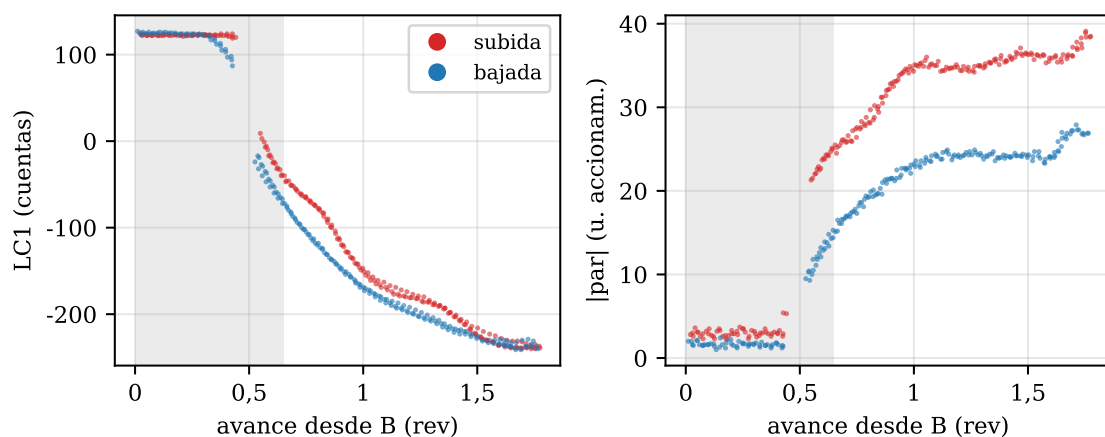


Figura 11.3: Señal de la célula (izquierda) y par del motor (derecha) frente al avance, en los tres ensayos limpios. En gris, el tramo sin carga.

es sistemáticamente mayor en la subida que en la bajada, como corresponde a unas pérdidas que se oponen siempre al movimiento.

Aplicando la separación de la ecuación (4.13) sobre intervalos de 0,05 rev (figura 11.4) se obtiene, en el tramo con carga (de 0,68 a 1,73 rev):

- **par gravitatorio** τ_g : crece de 20,8 a 32,3 u.a., es decir, un 56 %;
- **pérdidas** τ_f : 5,4 u.a. de media, el 19 % de τ_g , y prácticamente constantes a lo largo del recorrido, lo que corresponde a un rozamiento de tipo Coulomb;
- **tensión del modelo** T : *decrece* de 44,2 a 35,5 N, un -20 %, porque al subir la barra el brazo del cable mejora más deprisa de lo que crece el momento del peso.

11.6. Contraste con el modelo: cilindro frente a caracola

Éste es el núcleo del trabajo experimental. Se comparan tres hipótesis de tambor, todas con la misma geometría del mecanismo y el mismo recorrido de cable:

1. **cilindro** de radio constante $r = 34,2$ mm, el que paga el mismo cable en el mismo giro;
2. **caracola configurada**, con el radio decreciendo de 45 a 25 mm;
3. **caracola invertida**, con el radio creciendo de 25 a 45 mm.

11.6.1. Lo que dice la fuerza: nada concluyente

La tabla 11.2 recoge el ajuste de cada hipótesis contra la señal de la célula. Las tres dan coeficientes de determinación altos, entre 0,93 y 0,97. No es que las tres sean correctas: es que la señal de la célula, sin calibrar, no tiene capacidad para distinguirlas. El ajuste absorbe en su pendiente tanto la sensibilidad desconocida como el signo (sección 4.5.1), y lo poco que diferencia a las tres hipótesis en la forma de la curva queda por debajo del ruido y del retardo del filtro.

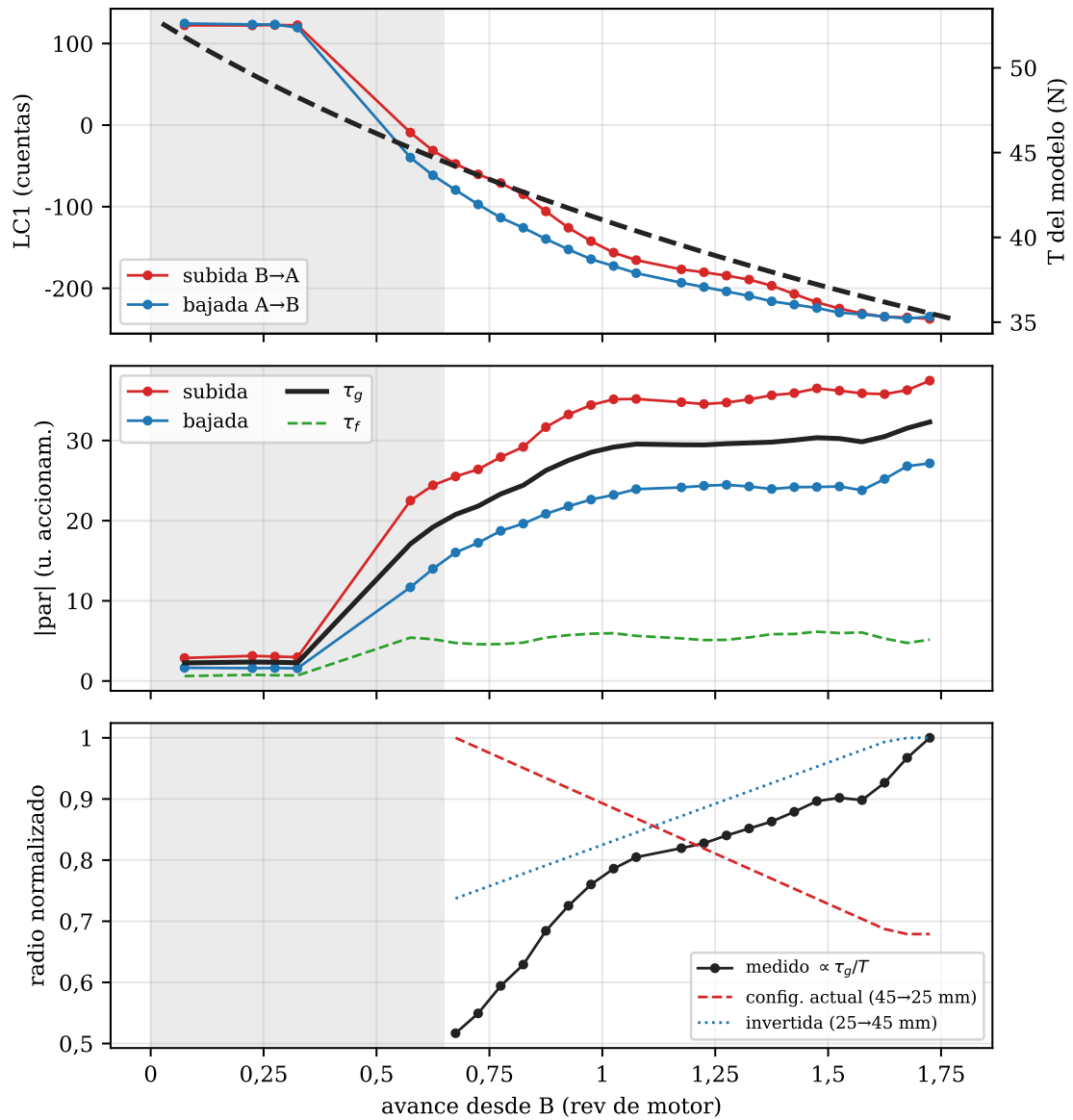


Figura 11.4: Arriba: señal de la célula en cada sentido, con la tensión del modelo. Centro: par en cada sentido, componente gravitatoria τ_g y pérdidas τ_f . Abajo: radio efectivo medido frente a las dos orientaciones posibles de la caracola.

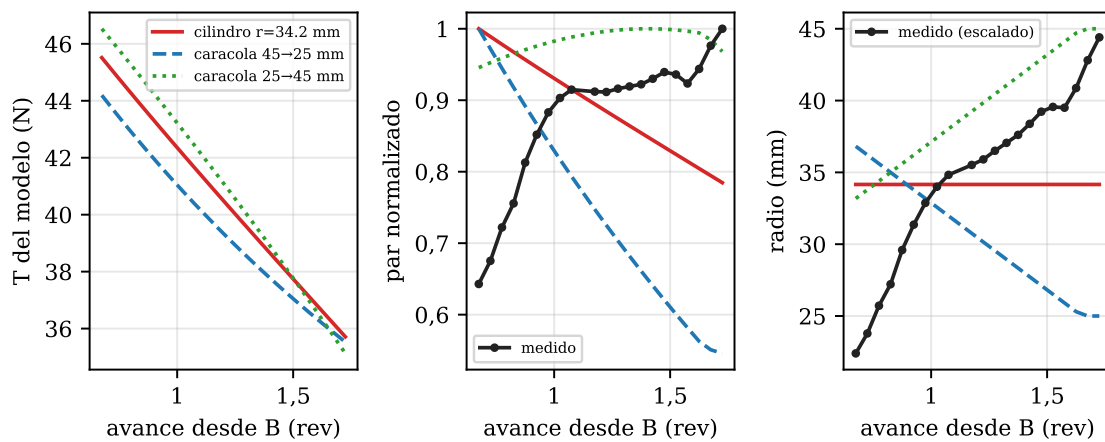


Figura 11.5: Contraste de las tres hipótesis. Izquierda: tensión del modelo. Centro: par normalizado de cada hipótesis frente al par medido. Derecha: radio de cada hipótesis frente al radio efectivo deducido de la medida.

Tabla 11.2: Contraste de las tres hipótesis de tambor sobre el tramo con carga. El R^2 del par no distingue el signo; la columna de la pendiente sí: un valor negativo significa que el modelo y la medida evolucionan en sentidos opuestos.

Hipótesis	Fuerza		Par		Variación de τ del modelo
	R^2	pendiente	R^2	pendiente	
Cilindro, $r = 34,2$ mm	0,947	+	0,780	–	–22 %
Caracola configurada, 45→25 mm	0,966	+	0,809	–	–45 %
Caracola invertida, 25→45 mm	0,930	+	0,673	+	2 %
Medido					56 %

11.6.2. Lo que dice el par: el tambor no es un cilindro

El par del accionamiento no necesita calibración, y con él la comparación sí es concluyente. En un tambor cilíndrico el par es proporcional a la tensión, porque el radio no cambia:

$$\tau_{\text{cil}}(\varphi) = T(\varphi) r_c \implies \frac{\Delta\tau}{\tau} = \frac{\Delta T}{T} = -22\%. \quad (11.1)$$

Es decir, un cilindro obliga al par a *bajar* un 22 % a lo largo del tramo con carga. El par medido *sube* un 56 %. La discrepancia no es de matiz: modelo y medida evolucionan en sentidos opuestos, y el cociente entre ambos comportamientos es de un factor 2,0 sobre el recorrido.

La conclusión es inmediata: **el tambor no se comporta como un cilindro**. Y como la única diferencia entre las dos hipótesis es el radio de enrollamiento, la medida permite estimarlo: el radio efectivo de la ecuación (4.14) pasa de 0,470 a 0,909 u.a./N, es decir, **crece un 93 %** a lo largo del tramo con carga (figura 11.5).

11.6.3. Cuánto compensa, y en qué sentido

Que el radio crezca es, cualitativamente, lo que pide la compensación: como la tensión baja al subir la barra, el radio tiene que crecer para que el producto se mantenga. La cuestión es cuánto. Con el recorrido medido, para que el par fuese exactamente constante bastaría con que el radio creciera un 24 %. El radio efectivo crece un 93 %, casi cuatro veces más.

El resultado, por tanto, tiene dos caras:

- **La caracola actúa, y de forma medible.** Frente al –22 % que impondría un cilindro, el par real recorre un camino completamente distinto. El mecanismo cambia la relación de transmisión a lo largo del recorrido, que es exactamente lo que se quería comprobar.

- **Pero no compensa: sobrecompensa.** En lugar de aplanar el par lo empeora en el otro sentido, pasando de una caída del 22 % a una subida del 56 %. Con estos parámetros, el rizado de par sobre el tramo con carga es peor que el de un cilindro equivalente (tabla 4.2).

Además, el sentido del radio efectivo medido (creciente) es el *contrario* al de la caracola configurada en el banco (decreciente, de 45 a 25 mm), lo que apunta a que la pieza está descrita al revés en la configuración, montada al revés en la máquina, o ambas cosas [VERIFICAR: medir el perfil real de la caracola y comprobar en qué sentido se enrolla el cable].

11.7. Ajuste de los parámetros del mecanismo

Los parámetros de la tabla 4.1 no proceden de una medición, sino de un ajuste sobre los propios ensayos, y conviene ser explícito sobre su alcance. Ajustando sólo contra la señal de la célula se obtienen los valores que usa la configuración actual, con $R^2 \approx 0,93$. Al repetir el ajuste incluyendo el par, el optimizador se marcha a valores sin sentido físico —radio final de 7 mm, desplazamiento de la salida del cable de 200 mm, reducción 3,5— aunque mejore formalmente los coeficientes (R^2 de 0,99 y 0,97).

Esa divergencia es en sí misma un resultado: significa que el modelo, con la parametrización elegida, no puede reproducir a la vez la fuerza y el par medidos. Faltan datos independientes, y los que faltan son precisamente los que no se han medido: las cotas del mecanismo y la calibración de la célula. Mientras no se midan, cualquier juego de parámetros ajustado es una descripción de los datos, no una verificación del modelo.

11.8. Comparación con la campaña preliminar

La campaña del 14 y 15 de septiembre, analizada con el modelo anterior (salida del cable en la vertical, tensión proporcional al cable) y con el ciclo $A \rightarrow B \rightarrow A$, sugería lo contrario: en el tramo central del recorrido la tensión variaba un 17 % y el par gravitatorio sólo un 6,4 %, lo que se interpretó como una compensación parcial. Aquel resultado no se sostiene, por tres motivos que conviene dejar escritos:

1. el modelo de tensión era el del caso particular $dx = 0$, que no corresponde al montaje;
2. el sentido del recorrido era el opuesto y el ciclo no empezaba en el extremo del mecanismo, de modo que el tramo analizado no es el mismo;
3. la calibración de la célula que entonces se daba por buena tenía incluso el signo contrario al que se deduce ahora, lo que confirma que la fuerza no estaba midiendo lo que se creía.

La lección metodológica es la que recoge la sección 10.6: un ajuste con buen R^2 sobre una única magnitud sin calibrar puede confirmar una hipótesis equivocada.

11.9. Limitaciones de las medidas

- **Célula sin calibrar.** La constante cuentas/newton ha salido distinta en cada intento (14,41 en la configuración, 20,5 ajustando contra el modelo, 27 a 28 en los ensayos limpios) y con signo cambiante entre campañas. Los valores absolutos de fuerza de esta memoria son, por tanto, orientativos.
- **Cotas sin medir.** Toda la tensión del modelo depende de L_2 , dx , L_3 , L_4 y α_0 , que son ajustados y no medidos.
- **Escala del par desconocida.** Por eso todos los resultados se expresan en variaciones relativas, que no dependen de ella.
- **Tramo sin carga.** La primera media revolución no aporta información y obliga a recortar el recorrido analizado.

- **Filtro de la célula.** La media móvil de ocho lecturas introduce un retardo que sólo se compensa en primera aproximación al promediar los dos sentidos.
- **Una sola masa y un solo montaje.** No se ha estudiado la dependencia con la carga.

11.10. Trabajo pendiente para cerrar la validación

1. **Medir las cotas** del mecanismo (L_2 , dx , L_3 , L_4) y el ángulo de la barra en B, e introducirlas en la configuración en lugar de los valores ajustados.
2. **Medir el perfil de la caracola:** radios inicial y final, sentido de enrollamiento y barrido, que es la magnitud que el análisis deduce de forma indirecta.
3. **Calibrar la célula** con masas patrón: escalones crecientes y decrecientes que cubran el rango de uso, ajuste por mínimos cuadrados, linealidad, histéresis y repetibilidad, y estimación de la incertidumbre con $k = 2$.
4. **Eliminar la holgura inicial** o, al menos, medirla, para que el recorrido con carga sea el recorrido completo.
5. **Repetir la campaña** con todo lo anterior y contrastar de nuevo cilindro frente a caracola, que es una comparación que el banco ya sabe hacer.

Capítulo 12

Gestión del proyecto

12.1. Ciclo de vida

El proyecto ha seguido un ciclo de vida incremental, con siete iteraciones en las que cada una entrega algo utilizable y se valida antes de pasar a la siguiente. No es una elección de estilo: en un trabajo con tanta incertidumbre sobre el mecanismo y los componentes, cada iteración ha servido para descartar una hipótesis. Las fases de definición y análisis se concentraron en la primera iteración; el diseño, la construcción y las pruebas se repiten en todas.

La validación se ha apoyado sistemáticamente en programas de prueba por subsistema (sección 6.11), de modo que ningún fallo de integración obligara a depurar todo el sistema a la vez.

12.2. Planificación

12.2.1. Planificación real

La planificación se ha reconstruido a partir del historial del repositorio: 36 *commits* entre el 4 de marzo y el 21 de septiembre de 2026, concentrados en nueve semanas (figura 12.1). El historial sólo refleja el trabajo sobre código y diseño electrónico; los periodos sin *commits* corresponden a montaje mecánico, pruebas sin cambios de código y periodo lectivo **[COMPLETAR: describir las actividades de abril a junio y de julio a agosto]**.

F1. Exploración del accionamiento (2–10 de marzo)

Comunicación Modbus con una placa XIAO ESP32-S3: escaneo de direcciones, modos par y velocidad, monitor de terminal, parámetros del accionamiento y resistencia de frenado. Culmina en un firmware v1.0 con regulador de fuerza en modo par.

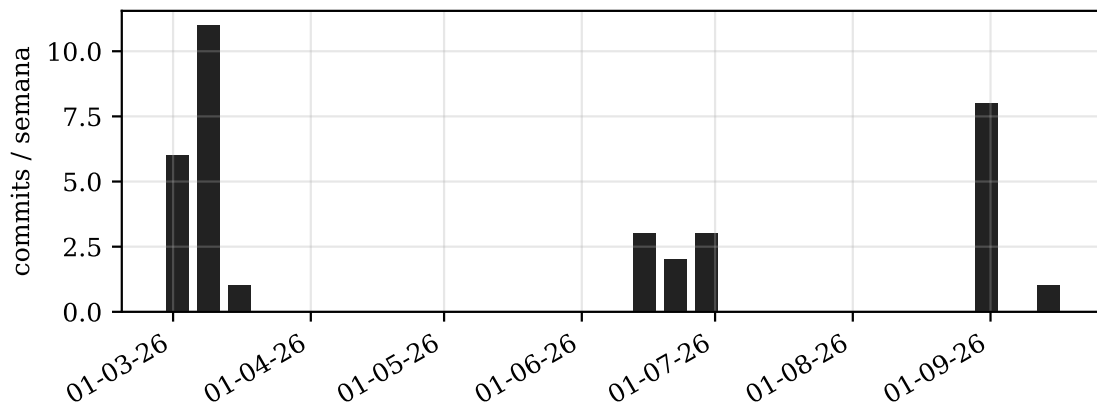


Figura 12.1: *Commits* por semana en el repositorio del proyecto.

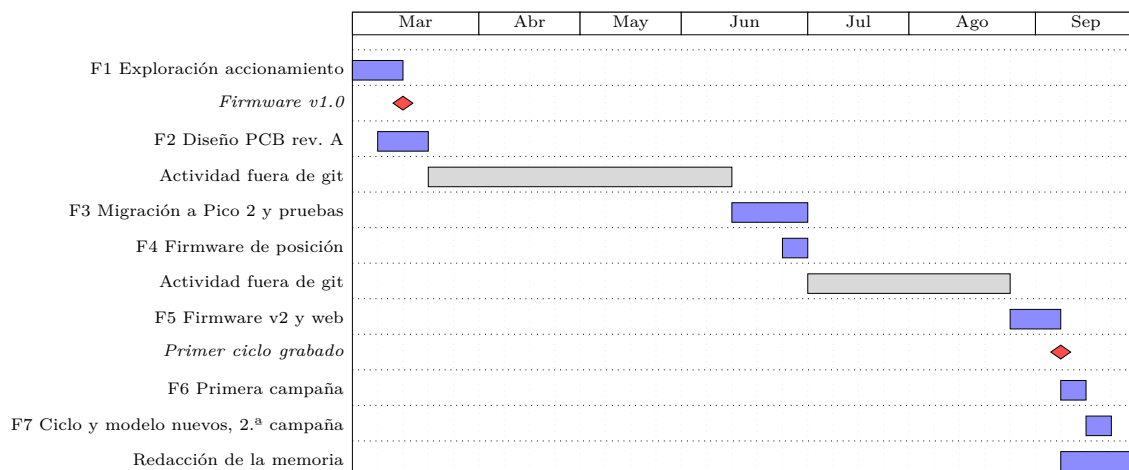


Figura 12.2: Diagrama de Gantt reconstruido (semanas desde el 2 de marzo de 2026).

F2. Diseño de la placa adaptadora (12–17 de marzo)

Esquema y colocación de la revisión A.

F3. Migración a Raspberry Pi Pico 2 y validación de la placa (20–30 de junio)

Programas de prueba de entradas y salidas, células de carga, RS-485 y escaneo Modbus. Versión de doble núcleo.

F4. Firmware de posición en modo velocidad (4 de julio)

Movimiento entre extremos con integración de la posición y monitor asociado.

F5. Firmware v2 y supervisión web (2–6 de septiembre)

Homing con dos finales de carrera, servidor FastAPI, página de supervisión, ciclos automáticos y grabación de ensayos.

F6. Primera campaña de ensayos (14–15 de septiembre)

Quince ensayos con el ciclo $A \rightarrow B \rightarrow A$, visor de ensayos con curva teórica y primer análisis.

F7. Rediseño del ciclo y del modelo, y segunda campaña (21 de septiembre)

Ciclo $B \rightarrow A \rightarrow B$ con extremos por finales de carrera, modelo con la salida del cable desplazada, configuración única del mecanismo con simulador, segunda campaña de ensayos y análisis definitivo.

12.3. Presupuesto

12.3.1. Personal

La dedicación de un TFG se fija en créditos ECTS, cada uno de 25 a 30 horas de trabajo [5]. La tabla 12.1 reparte una dedicación estimada por fases [VERIFICAR: ajustar las horas y los créditos del TFG en la titulación]. Para valorar el trabajo se toma como supuesto un coste de 30 €/h para un ingeniero o ingeniera junior y de 45 €/h para la tutoría.

Tabla 12.1: Dedicación estimada y coste de personal.

Actividad	Horas	€/h	Coste (€)
Estudio y documentación	30	30	900
Exploración del accionamiento (F1)	35	30	1.050
Diseño de la placa (F2)	45	30	1.350
Pruebas de placa y comunicaciones (F3)	45	30	1.350
Firmware (F4–F5)	70	30	2.100
Software de supervisión (F5, F7)	60	30	1.800
Ensayos y análisis (F6, F7)	40	30	1.200
Redacción de la memoria	50	30	1.500
Subtotal trabajo técnico	375		11.250
Tutoría	25	45	1.125
Total recursos humanos			12.375

12.3.2. Material

Tabla 12.2: Material del sistema.

Elemento	Cant.	Precio unit. (€)	Total (€)
Raspberry Pi Pico 2	1	[COMPLETAR:]	
Transceptor RS-485 THVD1400	1	[COMPLETAR:]	
Interruptor de lado alto TPS4H000-Q1	1	[COMPLETAR:]	
Amplificador operacional MCP6294	1	[COMPLETAR:]	
Diodos zéner BZX384	4	[COMPLETAR:]	
Resistencias y condensadores SMD 0805	25	[COMPLETAR:]	
Bornas de tornillo y tiras de pines	–	[COMPLETAR:]	
Fabricación de la PCB	1	[COMPLETAR:]	
Load Cell 2 Click (ZSC31014)	1	[COMPLETAR:]	
Célula de carga	1	[COMPLETAR:]	
Servomotor paso a paso de lazo cerrado con accionamiento	1	[COMPLETAR:]	
Fuente de alimentación del accionamiento	1	[COMPLETAR:]	
Finales de carrera	2	[COMPLETAR:]	
Caracola, barra, cable, estructura y masa de 2 kg	–	[COMPLETAR:]	
Total material			[COMPLETAR:]

12.3.3. Equipos y software

Todo el software empleado es libre o gratuito (KiCad, Arduino IDE con el núcleo Arduino-Pico, Python, FastAPI, Chart.js, git y $\text{\LaTeX}$), por lo que su coste es nulo. Los equipos se imputan por amortización lineal,

$$A = C \frac{t_{\text{uso}}}{t_{\text{vida}}}, \quad (12.1)$$

con C el coste de adquisición, t_{uso} el tiempo de uso y t_{vida} la vida útil.

Tabla 12.3: Amortización de equipos.

Equipo	Coste (€)	Vida útil (meses)	Uso (meses)	Amortización (€)
Ordenador portátil	[COMPLETAR:]	48	7	
Polímetro	[COMPLETAR:]	60	7	
Fuente de laboratorio	[COMPLETAR:]	60	7	
Total				[COMPLETAR:]

12.3.4. Resumen de costes

Tabla 12.4: Presupuesto total.

Concepto	Importe (€)
Recursos humanos	12.375
Material	[COMPLETAR:]
Amortización de equipos	[COMPLETAR:]
Subtotal	[COMPLETAR:]
Costes indirectos (10 %, supuesto)	[COMPLETAR:]
Base imponible	[COMPLETAR:]
IVA (21 %)	[COMPLETAR:]
Total	[COMPLETAR:]

12.4. Impacto

12.4.1. Impacto técnico y económico

La consecuencia práctica del capítulo 4 es de dimensionado: para un recorrido dado, un perfil de par constante reduce el par de pico que debe dar el motor. Sobre el recorrido con carga de los ensayos, el perfil ideal rebaja el pico un 11 % respecto a un cilindro equivalente, lo que permitiría un motor y un accionamiento algo menores. A velocidad constante, las pérdidas por efecto Joule son proporcionales a $\int \tau^2 d\varphi$, mínima cuando el par es constante (desigualdad de Cauchy-Schwarz), aunque en este caso la mejora es de apenas un 1 %.

Esas cifras deben leerse junto con el resultado del capítulo 11: el tambor instalado no aplanaba el par, sino que lo empeora respecto a un cilindro. El margen de mejora que ofrece el rediseño del perfil es, por tanto, bastante mayor que ese 11 %.

La plataforma desarrollada es además reutilizable: la placa, el firmware y la aplicación web forman un banco genérico para cualquier eje accionado por Modbus con medida de fuerza, con un coste de material muy inferior al de un sistema de adquisición comercial.

12.4.2. Objetivos de Desarrollo Sostenible

Tabla 12.5: Relación del trabajo con los Objetivos de Desarrollo Sostenible [21].

ODS	Contribución
4. Educación de calidad	Banco de ensayo abierto y documentado, útil como práctica de laboratorio de mecanismos, accionamientos y adquisición de datos.
7. Energía asequible y no contaminante	Un perfil de par uniforme reduce las pérdidas en el motor.
9. Industria, innovación e infraestructura	Integración de tecnologías industriales (Modbus, RS-485) con herramientas libres de bajo coste.
12. Producción y consumo responsables	Dimensionado de actuadores más pequeños; módulo microcontrolador en zócalo, reutilizable; datos brutos conservados para no repetir ensayos.

12.4.3. Aspectos éticos, legales y de seguridad

El prototipo es un equipo de laboratorio que no se comercializa. Su evolución hacia una máquina de uso por terceros tendría que considerar la Directiva 2006/42/CE de máquinas, que el Reglamento (UE) 2023/1230 sustituye a partir de enero de 2027, la Directiva 2014/30/UE de compatibilidad electromagnética y la Directiva 2011/65/UE sobre sustancias peligrosas (RoHS). La sección 5.5 discute las limitaciones de seguridad del diseño actual frente a ISO 13849-1 [11].

En cuanto a la ética profesional, se ha priorizado la trazabilidad: se conservan las medidas brutas, las figuras se generan con programas reproducibles y las limitaciones se señalan de forma explícita, incluso cuando el resultado contradice la hipótesis de partida.

Capítulo 13

Conclusiones

13.1. Conclusión

El objetivo general era construir y validar un banco de ensayos para un mecanismo de elevación con caracola de radio variable, y usarlo para contrastar el modelo del mecanismo. El banco está construido y funciona; el contraste ha dado un resultado distinto del esperado, y de ahí salen las conclusiones más interesantes del trabajo.

13.1.1. Sobre el banco de ensayos

1. El sistema cumple los requisitos de temporización: muestrea a 20,0 Hz con una desviación típica del periodo de 0,06 ms, y el trabajo del lazo ocupa el 22 % del periodo. Llegar a ese determinismo exigió abandonar la versión de doble núcleo en favor de un lazo de periodo fijo con un único dueño de cada bus.
2. El ciclo de ensayo $B \rightarrow A \rightarrow B$, con los extremos reconocidos por los finales de carrera y no por la posición integrada, repite el recorrido dentro del 0,6 % y cierra sobre el punto de partida. Esa decisión hace que el banco sea utilizable pese a la incertidumbre de la medida de posición.
3. Guardar los ensayos en cuentas brutas, con las constantes en la cabecera, ha permitido reanalizar por completo dos campañas después de cambiar el modelo y la calibración, sin repetir un solo ensayo. Es la decisión de diseño que más ha rendido.
4. Unificar el modelo en una sola implementación compartida por el simulador y el visor eliminó una fuente real de error: las páginas habían divergido hasta calcular con geometrías distintas.

13.1.2. Sobre el modelo del mecanismo

5. La posición de la salida del cable respecto a la articulación no es un detalle de dibujo. Con la salida en la vertical, la tensión resulta proporcional a la longitud libre de cable, un resultado simple y atractivo; con la salida desplazada, esa cancelación desaparece y la tensión pasa a depender de dos ángulos. El modelo del banco trabaja ya con el caso general.
6. El perfil de par constante es el que minimiza el par de pico para un recorrido y un giro de tambor dados. La demostración es independiente de la geometría: el trabajo está fijado, luego el máximo del par no puede ser menor que su media.

13.1.3. Sobre la validación experimental

7. **Medir sólo la fuerza no permite validar el modelo.** Con la célula sin calibrar, el ajuste incluye la sensibilidad como incógnita y el coeficiente de determinación es el cuadrado de la correlación: no distingue el signo. Tres hipótesis de tambor muy distintas —cilindro, caracola y caracola invertida— ajustan la señal de la célula con R^2 entre 0,93 y 0,97. Un buen ajuste no es una validación.

8. **El par del accionamiento sí decide**, porque no depende de ninguna calibración pendiente. Y al usarlo, la comparación es concluyente: un tambor cilíndrico obligaría al par a bajar un 22 % a lo largo del tramo con carga, mientras que el par medido sube un 56 %. **El tambor no se comporta como un cilindro**: su radio efectivo crece un 93 % a lo largo del recorrido.
9. **La caracola, sin embargo, no compensa: sobrecompensa**. Para mantener el par constante bastaría con que el radio creciera un 24 %. Con el crecimiento real, el par queda peor repartido que con un cilindro equivalente, y además en el sentido contrario al que describe la configuración del banco.
10. Las pérdidas del mecanismo son de tipo Coulomb: constantes a lo largo del recorrido y del orden del 19 % del par gravitatorio.
11. El resultado anterior no cierra la validación, y es importante decirlo con claridad: mientras no se midan las cotas del mecanismo y no se calibre la célula con masas patrón, cualquier juego de parámetros es una descripción de los datos y no una verificación del modelo. El trabajo deja el procedimiento montado y el diagnóstico hecho.

13.1.4. Grado de cumplimiento de los objetivos

Obj.	Resultado	Estado
O1	Modelo estático general, perfil ideal de par constante y demostración de que minimiza el par de pico (capítulo 4).	Cumplido
O2	Placa adaptadora con RS-485, entradas y salidas y dos zócalos mikroBUS; revisión A analizada y corregida en la revisión B (capítulo 6).	Cumplido; falta versionar la revisión B
O3	Firmware determinista con Modbus, célula, <i>homing</i> , ciclo B→A→B y protecciones (capítulo 7).	Cumplido
O4	Protocolo binario con CRC-8, telemetría por lotes y compatibilidad entre versiones (capítulo 8).	Cumplido
O5	Servidor y tres páginas web: monitor, configuración con simulador y visor de ensayos (capítulo 9).	Cumplido
O6	Caracterización del banco y contraste cilindro/caracola sobre dos campañas (capítulo 11).	Cumplido; validación abierta a falta de cotas y calibración
O7	Problemas y soluciones, planificación, presupuesto e impacto (capítulos 10 y 12).	Cumplido

13.2. Desarrollos futuros

13.2.1. Para cerrar la validación

- Medir las cotas del mecanismo y el perfil real de la caracola (radios, sentido y barrido), y sustituir con ellas los parámetros ajustados.
- Calibrar la célula de carga con masas patrón y estimar su incertidumbre; con ello, los valores de fuerza pasarían de orientativos a trazables.
- Confirmar la escala del registro de par del accionamiento para poder expresar los pares en Nm.
- Eliminar o medir la holgura inicial del cable, para que el recorrido con carga coincida con el recorrido completo.
- Repetir la campaña y rehacer el contraste cilindro frente a caracola, que el banco ya automatiza.

13.2.2. Sobre el mecanismo

- Fabricar, por ejemplo por impresión 3D, un tambor cilíndrico y otro con el perfil ideal calculado en la sección 4, y ensayar los tres en el mismo banco. Sería la comparación definitiva, con el mecanismo como única variable.
- Revisar el sentido de montaje de la caracola actual, que según la medida trabaja al contrario de lo que describe la configuración.
- Ensayar con varias masas para comprobar la linealidad del modelo con la carga.

13.2.3. Sobre el sistema de medida

- Leer la posición del codificador del accionamiento por Modbus en lugar de integrar la velocidad.
- Leer velocidad y par en una sola transacción Modbus (registros 16385 a 16387), lo que ahorraría unos 3 ms por periodo y permitiría subir la frecuencia de muestreo.
- Enviar en cada muestra la posición del firmware y la lectura sin filtrar de la célula, y sincronizar la lectura con la señal de interrupción del acondicionador.
- Corregir la codificación de la ganancia y los bits de estado del ZSC31014 (sección 10.9).

13.2.4. Sobre la seguridad y la explotación

- Cablear los finales de carrera y una seta de emergencia a la entrada STO del accionamiento, de modo que la parada no dependa del software.
- Versionar y fabricar la revisión B de la placa, con los ajustes de resistencias propuestos.
- Hacer que el servidor escuche sólo en la máquina local o exija autenticación, y añadir pruebas automáticas del protocolo.

13.3. Valoración personal

[**COMPLETAR:** Valoración personal: qué se ha aprendido, qué dificultades han sido más formativas y qué se haría de otra manera.]

Apéndice A

Asignación de pines

A.1. Raspberry Pi Pico 2 (revisión B de la placa)

Tabla A.1: Asignación de GPIO según el esquema de la revisión B y el firmware.

GPIO	Pin	Señal	Función
GP0	1	INT1	Interrupción de la célula 1 (MD1)
GP1	2	EN1	Habilitación / reinicio de las placas Click (LC1_EN_GPIO)
GP2	4	SDA1	I ² C1 SDA (LC_SDA_PIN)
GP3	5	SCL1	I ² C1 SCL (LC_SCL_PIN)
GP4	6	INT2	Interrupción de la célula 2 (MD2)
GP5, GP6	7, 9	–	Libres
GP7	10	SIN4	Entrada del canal 4 del TPS4H000
GP8	11	SIN3	Entrada del canal 3
GP9	12	SIN2	Entrada del canal 2
GP10	14	SIN1	Entrada del canal 1
GP11	15	DST	Fallo global del TPS4H000 (drenador abierto, <i>pull-up</i>)
GP12	16	DE	Dirección del transceptor RS-485 (MB_EN_GPIO)
GP13	17	RO	Recepción RS-485, UART PIO (MB_RX_GPIO)
GP14	19	DI	Transmisión RS-485, UART PIO (MB_TX_GPIO)
GP15	20	DIAG_EN	Habilitación del diagnóstico del TPS4H000
GP16	21	DIN1	Final de carrera A, arriba (PIN_LIMIT_A)
GP17	22	DIN2	Final de carrera B, abajo (PIN_LIMIT_B)
GP18	24	DIN3	Entrada digital libre
GP19	25	DIN4	Entrada digital libre
GP20	26	COMP	Salida del comparador (umbral 2,5 V)
GP21, GP22	27, 29	–	Libres
GP26	31	AOUT3	ADC0: entrada analógica AIN3
GP27	32	AOUT1	ADC1: entrada analógica AIN1
GP28	34	AOUT2	ADC2: entrada analógica AIN2
VBUS	40	+5V	Alimentación de los amplificadores
3V3	36	+3.3V	Transceptor RS-485 y placas Click

Nota. El firmware define LC1_INT_GPIO = 4, mientras que en el esquema de la revisión B la interrupción de la célula 1 llega a GP0 y GP4 corresponde a la célula 2. El firmware sólo usa

ese pin para llevarlo a nivel bajo durante el apagado de la célula (sección 10.2) **[VERIFICAR: comprobar a qué GPIO está conectado INT en la placa montada]**.

A.2. Bornas de campo

Tabla A.2: Bornas de la placa (revisión B).

Borna	Pin	Señal
J5 (RS-485)	1 / 2 / 3	A / B / GND
Terminal de salidas	1–8	SOUT4, SOUT3, –, VS_IN, SOUT2, SOUT1, AIN2, AIN1
Terminal de entradas	1–8	AIN3, AIN4, VS_IN, –, DI1, DI2, DI3, DI4
Filas de masa	1–8	GND

[VERIFICAR: comprobar el orden de las bornas sobre la placa fabricada]

Apéndice B

Especificación del protocolo binario

Formato común: 0xAA 0x55 LEN carga_útil[LEN] CRC8. Enteros en *little-endian*. CRC-8 con polinomio 0x07 y valor inicial 0x00 sobre la carga útil. Los desplazamientos se dan en bytes dentro de la carga útil.

B.1. Paquete 0x02: telemetría por lotes

Tabla B.1: Cabecera del lote (14 bytes).

Despl.	Campo	Tipo	Descripción
0	packet_id	u8	0x02
1	base_t	u32	millis() de la primera muestra del lote
5	bridge	u16	Lectura bruta del ZSC31014 (14 bits)
7	lc_status	u8	Bits de estado de la última lectura
8	flags	u8	b0: célula válida; b1: servo conectado; b2: configuración aplicada
9	error	u8	Código de error (tabla B.4)
10	servo_id	u8	Dirección Modbus en uso
11	rpm_status	u8	Resultado de la lectura de velocidad (tabla B.5)
12	trq_status	u8	Resultado de la lectura de par
13	n	u8	Número de muestras (1–8)

Tabla B.2: Muestra (14 bytes), repetida n veces.

Despl.	Campo	Tipo	Descripción
+0	dt_ms	u16	Milisegundos desde base_t
+2	rpm	i16	Velocidad del motor (registro 16385)
+4	current_x10	i16	Par (registro 16387), décimas de u. a.
+6	ref_cmd	i16	Consigna de velocidad escrita (rpm); negativa hacia A
+8	base_read	i16	Lectura filtrada menos tara (cuentas)
+10	io	u8	b0: final A (arriba); b1: final B (abajo)
+11	state	u8	Estado de la máquina (tabla B.3)
+12	work_us	u16	Tiempo de trabajo de la vuelta (μ s, saturado a 65535)

B.2. Paquete 0x03: estado de *homíng* (11 bytes)

0	packet_id	u8	0x03
1	pos_mrev	i32	Posición $\times 1000$
5	range_mrev	i32	Recorrido A $\rightarrow$ B medido $\times 1000$
9	home_phase	u8	0 reposo, 1 buscando A, 2 buscando B, 3 yendo al origen, 4 completado, 5 fallo
10	flags	u8	b0: recorrido válido

B.3. Paquete 0x04: resultado de la célula (13 bytes)

0	packet_id	u8	0x04
1	req	u8	Orden respondida (0x09, 0x0A o 0x0B)
2	result	u8	Resultado (tabla B.6)
3	gain	u8	Ganancia grabada (código 0–7, no lineal: sección 10.9)
4	offset	u8	Cero del A/D grabado (0–15)
5	cfg_before	u16	B_Config antes
7	cfg_after	u16	B_Config después
9	ack	u8	Primer byte devuelto por el chip
10	diag	u8	b0: el reinicio corta; b1: hubo ACK a Start_CM; b7–b4: intento del barrido
11	cfg1	u16	ZMDI_Config1

B.4. Paquete 0x05: estado del ciclo (18 bytes)

0	packet_id	u8	0x05
1	phase	u8	0 reposo, 1 colocándose en B, 2 hacia A, 3 volviendo a B, 4 completado, 5 abortado
2	done	u16	Repeticiones completadas
4	target	u16	Repeticiones pedidas (0 = sin fin)
6	pos_mrev	i32	Posición actual $\times 1000$
10	target_mrev	i32	Recorrido B $\rightarrow$ A pedido $\times 1000$
14	pos_b_mrev	i32	Posición en la que ha quedado fijado B $\times 1000$

La versión anterior de este paquete ocupaba 14 bytes y no llevaba el último campo. El servidor acepta las dos longitudes: con la trama corta asume `pos_b_mrev` = 0, de modo que un firmware y un servidor desparejados siguen informando de la fase (sección 10.4).

B.5. Órdenes

Orden	Carga útil	Notas
INIT	01 [addr]	addr opcional, 1-247
STOP	02	
SET_PARAM	03 id vv vv	valor i16
MOVE_A / MOVE_B	04 / 05	A es arriba; B, abajo
SHUTDOWN	06	
SCAN	07	bloqueante
HOME	08	
LC_PROGRAM	09 bb bb cc cc	B_Config u16, ZMDI_Config1 u16
LC_QUERY	0A	
LC_HOLD	0B park	0: alta impedancia; 1: líneas a masa
CYCLE_START	0C mm tt tt vv vv rr rr	mm = 0: hasta el final de carrera; 1: hasta la distancia indicada. Recorrido i16 (centi-rev), velocidad i16, repeticiones u16
CYCLE_STOP	0D	

Atajos de terminal (sólo con el decodificador en reposo): i INIT, a MOVE_A, b MOVE_B, x STOP, h HOME, q SHUTDOWN.

B.6. Tablas de códigos

Tabla B.3: Estados de la máquina.

0	IDLE (servo sin habilitar)
1	SCANNING
2	CONFIGURING
3	STOPPED (habilitado, consigna 0)
4	MOVING_A
5	MOVING_B
6	ERROR
7	HOMING
8	PROGRAMMING (célula)

Tabla B.4: Códigos de error.

0x00	Sin error
0x01	Célula no encontrada
0x02	Fallo en el escaneo del servo
0x03	Fallo al configurar el servo
0x04	Límite de fuerza

Tabla B.5: Resultados de ModbusMaster.

0x00	Correcto
0x01-0x04	Excepción del esclavo
0xE0	Dirección de esclavo incorrecta
0xE1	Función incorrecta
0xE2	Tiempo de espera agotado
0xE3	CRC incorrecto

Tabla B.6: Resultados de las órdenes de la célula.

0	Nunca se ha intentado
1	Grabado y verificado tras un ciclo de alimentación
2	Ya estaba así: no se ha escrito la EEPROM
3	Rechazado: el eje no está parado
4	Rechazado: bus ocupado
5	El chip se reinicia pero no entra en modo comando
6	Escrito, pero la relectura no coincide
7	Sólo lectura de la configuración
8	Sigue alimentada con el reinicio activo: hay otro camino de alimentación
9	La célula no contesta: sin tensión o sin bus
10	Reinicio mantenido para medir

Apéndice C

Registros Modbus del accionamiento

Dirección = 256·grupo + índice (índice en hexadecimal). Esclavo 1, 115200 baudios, 8N1.
[**VERIFICAR:** contrastar denominaciones y unidades con el manual del accionamiento]

Tabla C.1: Registros utilizados por el firmware.

Dirección	Parámetro	Acceso	Descripción	Valor escrito
0	C00.00	E	Modo de control (1 = velocidad, 2 = par)	1
5	C00.05	E	Nivel de rigidez	10
16	C00.10	E	Selección de resistencia de frenado (1 = externa)	1
17	C00.11	E	Potencia de la resistencia de frenado	1000
18	C00.12	E	Valor óhmico de la resistencia de frenado	235
19	C00.13	E	Disipación de la resistencia de frenado	30
801	C03.21	E	Consigna de velocidad (rpm, con signo: negativa hacia A)	cada periodo
802	C03.22	E	Aceleración	5
804	C03.24	E	Deceleración	5
1041	C04.11	E	Habilitación del servo (0/1)	0 → 1
16385	0x4001	L	Velocidad del motor (rpm)	—
16387	0x4003	L	Par del motor (décimas de u. a.)	—

Registros usados en versiones anteriores, con el accionamiento en modo par: 833 (consigna de par, C03.41), 839 y 840 (límites de par), 1542 (C06.06, apantallamiento de STO) y 1576 (C06.28, retardo de desconexión tras STO).

C.1. Secuencia de configuración

Al recibir la orden INIT, el firmware escribe en este orden: C04.11 = 0 (deshabilitar), C00.00 = 1 (modo velocidad), C03.21 = 0 (consigna nula), C00.05 (rigidez), C03.22 y C03.24 (rampas), C00.10 a C00.13 (resistencia de frenado) y C04.11 = 1 (habilitar). Entre escrituras se deja un silencio de 700 μs. Si falla una escritura esencial se repite la secuencia completa, y tras tres intentos el sistema pasa a estado de error con el código 0x03.

Apéndice D

Formato de los ficheros de ensayo

Cada ensayo se graba en `cycles/AAAAMDD-HHMMSS_ciclo.csv`, codificado en UTF-8. Las líneas que empiezan por # forman la cabecera, con las constantes en pares `clave=valor`; después viene una línea de nombres de columna y una fila por muestra.

```
# ensayo 2026-09-21 18:08:46
# lc1_cuentas_por_N=14.41 lc1_cero_N=51.78 lc1_int16=0
# repeticiones_pedidas=1 B=1.000 rev (posicion)
# lc1_base ya lleva restada la tara del firmware; lc1_raw es cruda
t_ms,t_rel_s,rep,fase,pos_rev,lc1_raw,lc1_base,rpm,par_x10,ref_rpm,lim_a,lim_b,estado,work_us
```

Código D.1: Comienzo de un fichero de la campaña del 21 de septiembre.

Tabla D.1: Columnas del fichero de ensayo.

Columna	Unidad	Descripción
t_ms	ms	Reloj del microcontrolador
t_rel_s	s	Tiempo desde la primera muestra del ensayo
rep	–	Repetición en curso (desde 1)
fase	–	hacia_A (subida) o volviendo_a_B (bajada). En los ensayos de la campaña preliminar, hacia_B y volviendo_a_A
pos_rev	rev	Posición del eje del motor integrada por el servidor
lc1_raw	cuentas	Lectura bruta del ZSC31014 (una por lote)
lc1_base	cuentas	Media móvil de 8 lecturas menos la tara del firmware
rpm	rpm	Velocidad medida por el accionamiento
par_x10	0,1 u. a.	Par medido por el accionamiento
ref_rpm	rpm	Consigna de velocidad (negativa hacia A, positiva hacia B)
lim_a, lim_b	0/1	Estado de los finales de carrera
estado	–	Estado de la máquina (tabla B.3)
work_us	µs	Tiempo de trabajo de la vuelta del lazo

D.1. Conversión y análisis

La conversión a fuerza es $F [N] = lc1_base/k + F_0$, con $k = lc1_cuentas_por_N$ y $F_0 = lc1_cero_N$. Conviene recordar que esa constante es provisional (sección 11.9).

Para el análisis, el *avance del ciclo* se obtiene restando la posición de la primera muestra, que es el punto B donde arranca el ensayo, y corrigiendo el signo con el de la muestra más alejada:

$$p_i = (pos_rev_i - pos_rev_0) \cdot \text{signo}(pos_rev_{lejana} - pos_rev_0). \quad (D.1)$$

Así el avance es siempre positivo hacia A, y los ensayos de las dos campañas se pueden comparar pese a tener sentidos de recorrido opuestos.

D.2. Notas sobre las cabeceras

- Los ensayos de la campaña preliminar se grabaron con una versión del servidor que todavía no escribía la calibración: en ellos aparece `lc1_cuentas_por_N=0`.
- El texto “(posicion)” de la tercera línea se escribe siempre que el recorrido pedido sea mayor que cero, aunque el ensayo se haya hecho contra los finales de carrera, que es el caso de todos los ensayos de esta memoria. Es un defecto conocido de la cabecera y no afecta a los datos.

Apéndice E

Manual de uso

E.1. Instalación

E.1.1. Firmware

1. Instalar Arduino IDE y añadir en *Preferencias* el gestor de placas de Arduino-Pico:
`https://github.com/earlephilhower/arduino-pico/releases/download/global/packagerp2040\_index.json`
2. Instalar el paquete *Raspberry Pi Pico/RP2040/RP2350* y la biblioteca *ModbusMaster* desde el gestor de bibliotecas.
3. Abrir `firmware_posicion_v2/firmware_posicion_v2.ino` y seleccionar la placa *Raspberry Pi Pico 2*.
4. Conectar la Pico con el botón BOOTSEL pulsado y cargar el programa.

E.1.2. Aplicación de supervisión

```
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi "uvicorn[standard]" pyserial
python web_monitor.py           # o: python web_monitor.py /dev/ttyACMO
```

En Linux, la cuenta de usuario debe pertenecer al grupo `dialout` para acceder al puerto. La interfaz se abre en `http://localhost:8080`, con tres páginas enlazadas por la barra inferior: *monitor (/)*, *ensayos (/ensayos)* y *mecanismo (/sim)*.

E.2. Configuración del mecanismo

Antes de ensayar conviene revisar la geometría en la página *mecanismo*: tipo de tambor (caracola o cilindro), radios y barrido, reducción, sentido, cotas L_2 , dx , L_3 , L_4 , masa y ángulo α_0 en el punto B. Los cambios no se aplican hasta pulsar *Guardar y propagar*; desde ese momento los usan tanto el simulador como el visor de ensayos.

La página avisa si la geometría es imposible (por ejemplo, si el recorrido pedido obligaría a la barra a pasar de sus topes) e indica, mediante la inversa, cuántas revoluciones de motor hacen falta para llevar la barra a un ángulo concreto: es el recorrido que tendría que hacer el ciclo.

E.3. Procedimiento de ensayo

1. **Comprobaciones previas:** masa colgada, cable sin cocas y correctamente asentado en la caracola, finales de carrera en reposo (indicadores apagados) y zona de movimiento despejada.

2. **Inicializar:** pestaña *Servo*, dirección 1, botón *Init*. El estado pasa a STOPPED y el indicador del servo a conectado.
3. **Homing:** fijar la velocidad de *homing* y el desplazamiento del origen, pulsar *Aplicar parámetros* y después *Home*. Al terminar se muestra el recorrido A→B. El eje queda en A, arriba.
4. **Tara:** con el mecanismo en reposo, pulsar *Tara* para poner a cero la célula.
5. **Ciclo:** en la pestaña *Ciclo B→A→B*, elegir hasta dónde sube (final de carrera A, recomendado, o una distancia desde B), la velocidad y las repeticiones, y pulsar *Iniciar ciclo*. El eje baja primero a B para colocarse —ese tramo no se graba— y a partir de ahí empieza el ensayo. La grabación se abre y se cierra sola.
6. **Análisis:** en la página *ensayos*, abrir el fichero y activar la curva teórica para compararla con la medida.
7. **Fin:** *Shutdown* para deshabilitar el accionamiento.

E.4. Programación de la célula de carga

Con el eje parado, abrir *Configuración avanzada* en la pestaña de la célula, elegir la ganancia y el cero del A/D y pulsar *Programar*. El eje queda bloqueado cerca de un segundo. El resultado aparece debajo; si es “sigue alimentada con el reinicio activo”, usar *Mantener reset* y medir la tensión de alimentación de la placa Click, que debe caer a 0 V. Téngase en cuenta que el código de ganancia de la interfaz no coincide con la ganancia real para los valores intermedios (sección 10.9).

E.5. Diagnóstico

Síntoma	Causa probable y acción
Servo desconectado, lecturas con código 0xE2	Tiempo de espera Modbus: revisar el cableado A/B, la masa común, la dirección y la velocidad del accionamiento. Probar con <code>test_modbus_scan</code> .
Código 0xE3	CRC incorrecto: ruido o polaridad A/B invertida.
Estado ERROR con código 0x03	Tres fallos de configuración: el accionamiento no acepta las escrituras (¿alimentado?, ¿alarma activa?).
Final de carrera siempre activo	Contacto abierto o cable roto (entrada NC).
El ciclo termina enseguida y no graba nada	El eje ya estaba en el extremo o la fase de colocación no llegó a B: revisar en <code>web_monitor.log</code> las líneas de cambio de fase.
La interfaz no muestra la fase del ciclo	Firmware y servidor desparejados: recargar el firmware o actualizar el servidor.
Célula no válida	Sin lecturas frescas en 150 ms: revisar la placa Click y el bus I ² C con <code>test_loadcell</code> .
La curva teórica no se parece a la medida	Revisar la geometría en <i>mecanismo</i> : el aviso de escala y el de ángulo indican qué parámetro no cuadra.

E.6. Precauciones

- Las protecciones son de software: mantener las manos fuera del recorrido con el servo habilitado y tener a mano el corte de alimentación del accionamiento.
- No programar la célula ni usar *Mantener reset* con carga en movimiento (el firmware lo impide, pero la medida de fuerza queda interrumpida).
- El servidor acepta órdenes de cualquier equipo de la red: usarlo en una red de confianza.

Apéndice F

Estructura del repositorio

Ruta	Contenido
Firmware	
firmware_posicion_v2/	Firmware final (Pico 2): Modbus, célula, <i>homing</i> , ciclo B→A→B y protocolo. <code>i2c_bus.h</code> : árbitro del bus I ² C.
firmware_posicion/	Versión anterior en modo velocidad entre extremos.
firmwarerasp/	Primera versión para Pico 2, con doble núcleo.
firmware/	Versión para XIAO ESP32-S3 con regulador de fuerza en modo par.
Programas de prueba	
pcbtest/	Entradas analógicas, entradas digitales y salidas de potencia.
test_limit_switches/	Finales de carrera.
test_rs485*/	Bucle local PIO, transmisión, recepción y comunicación Modbus.
test_modbus_scan/	ZSC31014 y lectura dual con NAU7802.
test_loadcell*/	Barrido Modbus desde el PC con adaptador USB-RS485.
test_servo_modbus.py	
Ordenador	
web_monitor.py	Servidor FastAPI: puente serie-WebSocket, grabación y configuración del mecanismo.
mecanismo.json	Geometría del mecanismo (la escribe el servidor).
static/index.html	Monitor en tiempo real.
static/ensayos.html	Visor de ensayos con curva teórica y ajuste.
static/sim.html	Configuración del mecanismo y simulador.
static/mecanismo.js	Modelo del mecanismo compartido por las páginas.
static/tema.css, tema.js, nav.js	Tema visual, colores de las gráficas y navegación común.
static/fui/	Kit de interfaz externo (copia sin modificar).
leer_ciclo.py	Resumen de un ensayo en consola.
monitor.py, monitor_posicion.py	Monitores de terminal de versiones anteriores.
cycles/	Ficheros de ensayo (CSV).
Hardware y documentación	
tfg_adaptador/	Proyecto KiCad de la placa (revisión A) y copias de seguridad.
image (1).png	Esquema de la revisión B.
README.MD	Notas sobre la UART de la Pico y el doble núcleo.
memoria/	Esta memoria: <code>TFG.tex</code> , <code>capitulos/</code> , <code>figuras/</code> y <code>bibliografia/</code> .

Ruta	Contenido
<code>figuras/generar_figuras.py</code>	Figuras y cifras de la campaña preliminar.
<code>figuras/generar_figuras21.py</code>	Figuras y cifras de la campaña del 21 de septiembre.
<code>figuras/modelo_mecanismo.py</code>	Modelo del capítulo 4 portado a Python.
<code>memoria/_v1/</code>	Versión anterior de la memoria, antes de adoptar la plantilla de la ETSIDI.
<code>old/</code>	Pruebas iniciales en modo par.

F.1. Reproducción de los resultados

Con el repositorio y Python con `numpy`, `scipy` y `matplotlib` instalados:

```
python memoria/figuras/generar_figuras.py      # campaña preliminar
python memoria/figuras/generar_figuras21.py    # campaña del 21 de septiembre
```

Ambos programas leen `cycles/*.csv`, escriben las figuras en `memoria/figuras/` y dejan las cifras en `datos.json` y `datos21.json`, que son las que cita el capítulo 11. Los parámetros del mecanismo están al principio de `modelo_mecanismo.py` y deben coincidir con los de `mecanismo.json`.

Bibliografía

- [1] J. Bieber, D. Bernstein, and M. Beitelschmidt. Variable radius drum design for cable-driven parallel robots based on maximum load profile. In *Cable-Driven Parallel Robots (CableCon 2023), Mechanisms and Machine Science*, volume 132, Cham, 2023. Springer.
- [2] Chart.js contributors. Chart.js.
- [3] G. Endo, H. Yamada, A. Yajima, M. Ogata, and S. Hirose. A passive weight compensation mechanism with a non-circular pulley and a spring. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2010.
- [4] I. Fette and A. Melnikov. The websocket protocol, rfc 6455. IETF, 2011.
- [5] Gobierno de España. Real decreto 1125/2003, por el que se establece el sistema europeo de créditos y el sistema de calificaciones en las titulaciones universitarias. BOE n.º 224, 2003.
- [6] Just L. Herder. *Energy-free systems: theory, conception and design of statically balanced spring mechanisms*. PhD thesis, Delft University of Technology, 2001.
- [7] K. Hoffmann. *An Introduction to Stress Analysis and Transducer Design using Strain Gauges*. HBM, Darmstadt, 2012.
- [8] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [9] IEC. Iec 61800-5-2: Adjustable speed electrical power drive systems — part 5-2: Safety requirements — functional, 2016.
- [10] E. F. Philhower III et al. Arduino-pico: Raspberry pi pico arduino core for rp2040 and rp2350 boards.
- [11] ISO. Iso 13849-1: Safety of machinery — safety-related parts of control systems — part 1: General principles for design, 2023.
- [12] T. Kenjo and A. Sugawara. *Stepping Motors and Their Microprocessor Controls*. Oxford University Press, 2nd edition, 1994.
- [13] KiCad Project. Kicad eda 9.0.
- [14] David S. Landes. *Revolution in Time: Clocks and the Making of the Modern World*. Harvard University Press, Cambridge, MA, 1983.
- [15] C. Liechti. pyserial.
- [16] Microchip Technology. *MCP6291/1R/2/3/4/5: 1.0 mA, 10 MHz rail-to-rail op amps — Datasheet*.
- [17] MikroElektronika. Load cell 2 click — documentación del producto.
- [18] MikroElektronika. mikrobus standard specification, v2.00.
- [19] Modbus Organization. Modbus over serial line: Specification and implementation guide v1.02, 2006.

- [20] Modbus Organization. Modbus application protocol specification v1.1b3, 2012.
- [21] Naciones Unidas. Transformar nuestro mundo: la agenda 2030 para el desarrollo sostenible. resolución a/res/70/1, 2015.
- [22] Nexperia. *BZX384 series: Voltage regulator diodes — Datasheet*.
- [23] Nuvoton Technology. *NAU7802: 24-bit dual-channel ADC for bridge sensors — Datasheet*.
- [24] NXP Semiconductors. *UM10204: I2C-bus specification and user manual, rev. 7.0*, 2021.
- [25] R. Pallàs-Areny and J. G. Webster. *Sensors and Signal Conditioning*. Wiley, Nueva York, 2nd edition, 2001.
- [26] S. Ramírez. Fastapi.
- [27] Raspberry Pi Ltd. *RP2350 Datasheet y Raspberry Pi Pico 2 product brief*.
- [28] Renesas Electronics. *ZSC31014 RBicLite Digital Output Sensor Signal Conditioner — Datasheet*, 2016.
- [29] S. Seriani and P. Gallina. Variable radius drum mechanisms. *Journal of Mechanisms and Robotics*, 8(2):021016, 2016.
- [30] T. Shen and K. Yano. An innovative spiral pulley that optimizes cable tension variation for superior balancing performance. *Journal of Robotics and Mechatronics*, 34(3):599–606, 2022.
- [31] Telecommunications Industry Association. Tia/eia-485-a: Electrical characteristics of generators and receivers for use in balanced digital multipoint systems, 1998.
- [32] Texas Instruments. *THVD1400: 3.3-V to 5-V RS-485 transceiver with IEC ESD protection — Datasheet*.
- [33] Texas Instruments. *TPS4H000-Q1: 40-V, 1000-m Ω 4-channel smart high-side switch — Datasheet*.
- [34] D. Walker. Modbusmaster: Enlighten your arduino to be a modbus master.